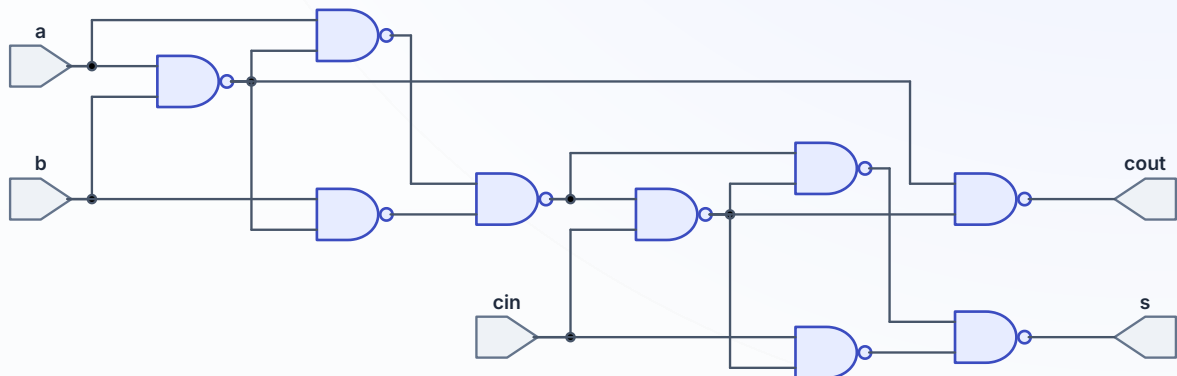


From NAND to a Pipelined CPU

Gates, arithmetic, state, and a verified five-stage RISC-V processor — with multiplication and division in the appendices, every circuit written in SystemVerilog and drawn from its own synthesized netlist.



Nine-NAND full adder, synthesized from gates.sv (Chapter 4)

Abstract

This primer builds a working computer from a single logic primitive. Starting from the two-input NAND gate, it derives the remaining Boolean operators, assembles them into adders, multiplexers and an arithmetic-logic unit (ALU), closes feedback loops to obtain latches, flip-flops and a register file, and combines those parts into a central processing unit (CPU) for a subset of RV32I, the base integer instruction set of the open RISC-V architecture. That processor is developed as a single-cycle machine whose clock period must cover its slowest instruction, and as a five-stage pipeline with forwarding, load-use stalling and branch flushing. It is later extended with dynamic branch prediction and rebuilt as a multicycle machine, and the clock periods of all four versions are measured. A chapter on number systems explains how the processor represents and computes with signed and unsigned integers. A chapter on floating point builds an IEEE 754 FPU, verifies it bit-exactly against a reference, and attaches it to all four processors as a memory-mapped coprocessor. Four appendices extend the core. Two treat the arithmetic left out of it: multiplication (array, shift-and-add, Booth, and Dadda/Wallace trees) and division (restoring, non-restoring, SRT), including the RISC-V M-extension conventions for division by zero and signed overflow. A third builds an addressable 16-word $\times$ 8-bit memory from decoders, registers and a multiplexer tree. The fourth implements the same multicycle control unit as a hardwired state machine and as a microprogram, compares the two, and completes the multicycle processor they control. Further sections add canonical forms and Karnaugh maps, a Kogge–Stone parallel-prefix adder, dynamic branch prediction, and a direct-mapped cache. Appendix E opens every word-level operator symbol down to gates, Appendix F explains how to reproduce every result, and Appendix G is a glossary of terms and notation.

Every circuit in the text exists as synthesizable SystemVerilog. Figures tagged YOSYS $\rightarrow$ NETLISTSVG are not drawings: each is the output of synthesizing the listed module with Yosys, exporting the netlist, and laying it out automatically with netlistsvg, so what the figure shows is what the code describes. All designs were simulated with Icarus Verilog. The gate library, adders, ALU and storage elements were checked exhaustively; the multipliers and dividers were checked for every pair of 8-bit operands; and four processors — single-cycle, multicycle, pipelined, and pipelined with branch prediction — were run against a Python instruction-set simulator, both on a 26-instruction directed program and on 2 000 random programs, with every register and memory write compared. Ten key properties were also proved formally with a Boolean satisfiability (SAT) solver, and deliberately injected bugs confirmed that both the tests and the proofs detect errors. Quantitative statements — gate depth of a ripple adder, cycle counts, stall and flush counts — are measured from those netlists and simulations, not estimated.

267 576

unit checks, 0 failures
(exhaustive where stated)

2N+3

NAND levels to carry-out, N-bit ripple adder (measured N = 4...64)

0 / 249 472

mismatches over randomly generated instructions on 4 processors

10 / 10

properties proved formally (adders, operators, multipliers, ALU, hazard unit, controllers)

Keywords: Boolean algebra; functional completeness; CMOS; carry-lookahead; sequential logic; static timing; RISC-V; single-cycle datapath; pipelining; hazards; forwarding; Booth recoding; Dadda multiplier; restoring and non-restoring division; SRT; memory addressing; address decoding; microprogramming; hardwired control; two's complement; overflow; IEEE 754 floating point; FPU; Karnaugh maps; parallel-prefix adders; branch prediction; caches; random co-simulation; mutation testing; formal verification.

Contents

1	Introduction	6
1.1	Scope and approach	7
1.2	How the figures and numbers were produced	8
2	Boolean Logic and the Universality of NAND	10
2.1	Boolean algebra	10
2.2	Functional completeness	10
2.3	The NAND gate in CMOS	10
2.4	Deriving the other gates	11
2.5	The multiplexer	12
2.6	Canonical forms: sum of products and product of sums	13
2.7	Karnaugh maps	14
2.8	From two-level forms to gates	16
3	Number Systems and Computer Arithmetic	18
3.1	Positional number systems	18
3.2	Unsigned integers	18
3.3	Signed integers	18
3.4	Working with two's complement	20
3.5	Overflow	20
3.6	How a RISC-V processor handles arithmetic	21
3.7	Demonstration on the four processors	21
3.8	Fixed-point fractions	22
4	Combinational Arithmetic	24
4.1	Binary numbers and two's complement	24
4.2	Half and full adders	24
4.3	The ripple-carry adder and its delay	25
4.4	Carry-lookahead	26
4.5	The arithmetic-logic unit	28
4.6	Parallel-prefix adders: Kogge–Stone	29
5	Sequential Logic	33
5.1	Feedback and the SR latch	33
5.2	The gated D latch	33
5.3	The edge-triggered flip-flop	34
5.4	The synchronous timing discipline	35
5.5	Registers and counters	35
5.6	Register files	36
6	The Instruction Set	38
6.1	Why RISC-V	38
6.2	Instruction formats	38
6.3	The test program	39
7	The Single-Cycle Processor	41
7.1	Datapath	41
7.2	Control	43
7.3	Critical path and performance	45
8	The Pipelined Processor	46

8.1	Throughput versus latency	46
8.2	Five stages	46
8.3	Watching the pipeline run	48
8.4	Data hazards and forwarding	48
8.5	The load-use stall	49
8.6	Control hazards	49
8.7	The hazard unit	50
8.8	Measured behaviour	51
8.9	Where real processors go next	52
8.10	Adding dynamic branch prediction	52
8.11	Measured clock periods: does pipelining pay?	54
9	Floating Point and an FPU	57
9.1	The IEEE 754 formats	57
9.2	Precision, rounding and special values	58
9.3	The algorithms	59
9.4	A synthesizable FPU	59
9.5	What floating point costs in hardware	63
9.6	Using the FPU as a memory-mapped coprocessor	64
9.7	Extending the processor: the RISC-V F extension	66
10	Verification	67
10.1	Random-program co-simulation	68
10.2	Mutation testing: does the test bench have teeth?	68
10.3	Formal verification	69
11	Summary	71
A	Multiplication	72
A.1	Partial products	72
A.2	The array multiplier	72
A.3	Sequential shift-and-add	73
A.4	Booth recoding	75
A.5	Tree multipliers	76
A.6	RISC-V M-extension semantics	77
B	Division	78
B.1	Restoring division	78
B.2	Non-restoring division	80
B.3	SRT division	80
B.4	RISC-V division semantics	81
C	Memory Addressing	82
C.1	Addresses and capacity	82
C.2	The write path: decoding	82
C.3	The storage words	85
C.4	The read path: multiplexing	85
C.5	The complete memory	86
C.6	Timing of a write and a read	88
C.7	Verification	88
C.8	From registers to real memory	88
C.9	Caches	89
D	Microcode and Hardwired Control	92
D.1	Why sequencing is needed	92

D.2	The hardwired implementation	93
D.3	The microprogrammed implementation	94
D.4	Equivalence and cost	96
D.5	Hardcoded versus microcoded instructions in practice	97
D.6	A complete multicycle processor	97
E	Word-Level Operators at Gate Level	101
E.1	Why some operations are drawn as circles	101
E.2	The adder (+)	101
E.3	The subtractor (−)	102
E.4	Equality (=)	103
E.5	Comparison (<, ≤, >, ≥)	104
E.6	Cells drawn with ordinary gate symbols	105
E.7	Cost and verification	107
F	Reproducing the Results	109
G	Glossary and Notation	112
G.1	Terms and abbreviations	112
G.2	Notation	115
	References	116

CHAPTER 1

Introduction

A modern processor is the most intricate object most engineers will ever use, yet nothing in it is more complicated than a switch that conducts or does not. The distance between those two facts is bridged by abstraction: a small, reliable component is characterized once, and from then on is used only through its interface. The ladder in Figure 1.1 is the route this primer climbs. Each rung is built only from the rungs beneath it, and each is checked before the next one is added.

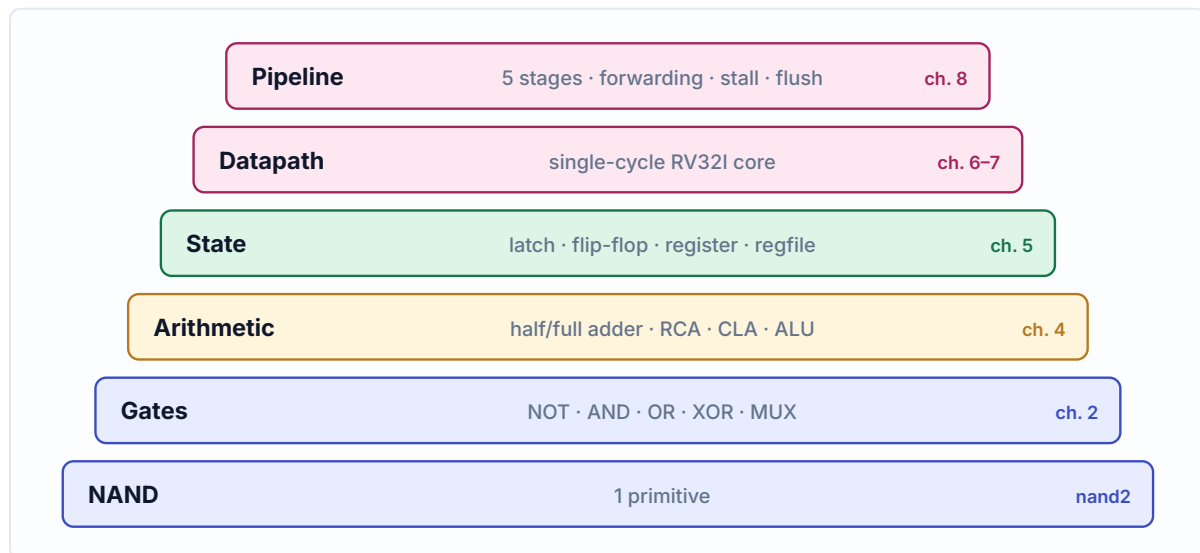


Figure 1.1. The abstraction ladder followed in this primer. Every level is expressed in SystemVerilog using only the level beneath it (for the lowest levels) or ordinary register-transfer constructs (from the arithmetic level upward).

Figure 1.2 shows the same climb as a dependency tree drawn from the actual modules. Read top-down, it shows what the pipelined processor is made of. Read bottom-up, it shows every path by which a single primitive, `nand2`, becomes part of the processor. Several blocks, such as the multiplexer and the register, sit on many paths at once, which is why they are designed and verified first.

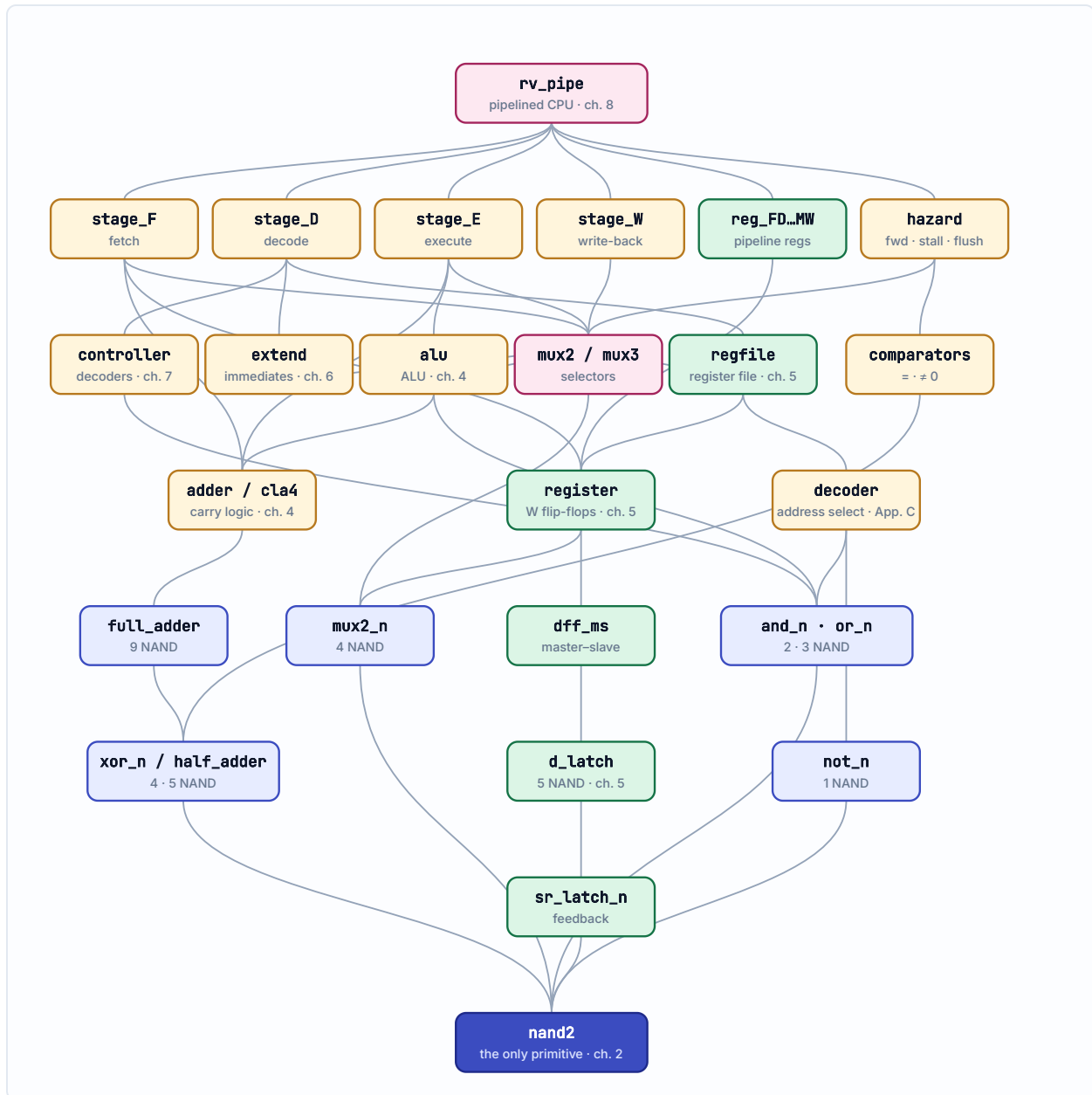


Figure 1.2. Dependency tree from NAND to the pipelined CPU. Each box is a module in the accompanying sources (colours as in the netlist figures: amber = combinational blocks, green = state, pink = selection, blue = gate-level cells). An edge means "is built from". Blocks above *register* are written at RTL, and the edge shows the construction that synthesis would use.

1.1 Scope and approach

The destination is a five-stage pipelined processor for a subset of the RV32I base instruction set [13]: the register-register operations `add` `sub` and `or` `sll`, their immediate counterparts `addi` `andi` `ori` `slli`, the memory operations `lw` `sw`, the conditional branch `beq`, and the jump-and-link `jal`. That subset is small enough to trace by hand and large enough to exhibit every classical pipeline hazard. Chapter 3 first fixes how bits represent numbers. Along the way the primer builds four verified versions of the processor — single-cycle (Chapter 7), pipelined (Chapter 8), pipelined with branch prediction (Section 8.10) and multicycle (Appendix D.6) — and compares their measured clock periods and run times (Section 8.11). Chapter 9 adds floating point: IEEE 754 arithmetic, a verified FPU, and how it attaches to the processor. The appendices cover multiplication, division, memory addressing and caches, and microprogrammed control. Appendix E shows every word-level operator symbol at gate level, and Appendix G collects the terms and notation used throughout. The design lineage is the textbook one popularized by Patterson and

Hennessy [1] and by Harris and Harris [2]; the gate-up progression follows the spirit of Nisan and Schocken's *Elements of Computing Systems* [3], but in an industry hardware description language (HDL), SystemVerilog, rather than a teaching HDL.

Two conventions run through the text. First, below the arithmetic level, circuits are written *structurally*: they instantiate a `nand2` module and nothing else, so the synthesized netlist contains only NAND gates and the figures show the real gate topology. Above that level, circuits are written at register-transfer level (RTL), which describes registers and the operations between them, — `assign y = a + b;` — because once the reader has seen an adder built from NANDs, redrawing it inside every datapath adds nothing. Second, performance is expressed in quantities that can be counted from the design: NAND levels, gate counts and clock cycles. Absolute delays depend on a process technology this primer does not assume.

1.2 How the figures and numbers were produced

The toolchain is entirely open source. Yosys 0.69 [16] parses and elaborates each module, maps processes to cells, and writes a JSON netlist; submodules that should appear as boxes are marked as black boxes so that hierarchy is preserved. netlistsvg 1.0.2 [17] lays the netlist out with the Eclipse Layout Kernel's layered algorithm, and a custom skin and stylesheet give gates, multiplexers, flip-flops and module instances distinct colours. Thick lines are multi-bit buses; the dark vertical bars are bus splits and joins, labelled with the bit ranges they carry. Icarus Verilog 12.0 [18] simulates every module. A small Python assembler encodes the test program, and a Python instruction-set simulator (ISS) provides the reference architectural state. Formal proofs use Yosys' built-in SAT solver, and whole-processor timing is computed by a longest-path analysis of the synthesized gate netlists (Section 8.11). Chapter 10 summarizes the verification, and Appendix F lists the commands needed to reproduce every artefact.

READING A NETLIST FIGURE

Yosys keeps some operations as whole-word cells rather than gates, and netlistsvg draws each cell type with its own symbol (Figure 1.3):

- **Ports.** Pentagons on the left are module inputs; those on the right are outputs.
- **Gates.** Blue shapes are gates; a bubble means inversion.
- **Operators.** Pale blue **circles** are word-level arithmetic and comparison operators. `+` is an adder (`$add`) and `-` a subtractor (`$sub`). `=` is an equality comparator (`$eq`), whose single-bit output is 1 when its two input words are identical; it is how `a = b` or `state = 4'd3` appears. `<` is a less-than comparator.
- **Other cells.** Pink trapezoids are multiplexers (input **0** is the upper one), and green boxes are banks of D flip-flops.
- **Instances and constants.** Amber boxes are instances of other modules, labelled *instance : module*. Small white boxes are constants.
- **Wiring.** Thick lines are buses, and the dark bars split or join them.

At the NAND level every one of these cells would dissolve into gates. An `$add` becomes the ripple or lookahead adder of Chapter 4, and an `$eq` becomes one XOR per bit, an OR tree and a NOT. Appendix E shows each of these symbols at gate level.

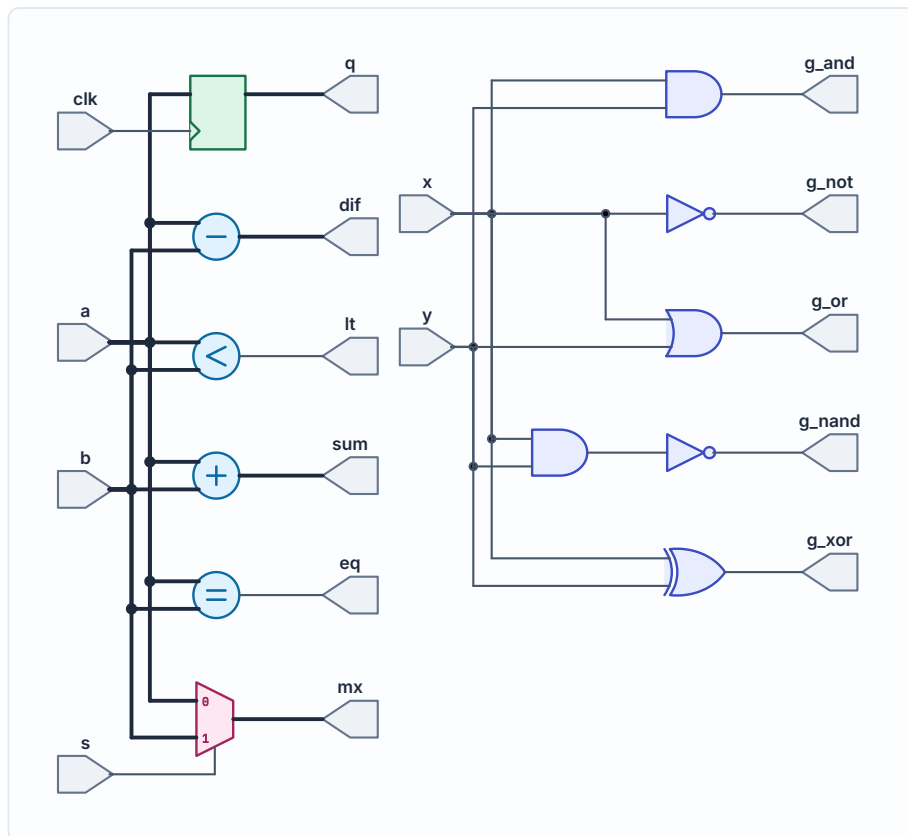


Figure 1.3. Symbol legend, synthesized from `Legend.sv`, which contains one of each cell type: flip-flop bank, subtractor, less-than, adder, equality comparator and multiplexer (left); AND, NOT, OR, NAND and XOR gates (right).

YOSYS → NETLISTSVG

CHAPTER 2

Boolean Logic and the Universality of NAND

2.1 Boolean algebra

Boolean algebra works over the set $\{0, 1\}$ with conjunction ($\cdot$, AND), disjunction ($+$, OR) and complement ($\bar{x}$, NOT). Its laws mirror ordinary algebra with two important differences: both operations distribute over each other, and every element has a complement. The identities used throughout this primer are:

$$\begin{aligned}
 x + 0 &= x, & x \cdot 1 &= x && \text{(identity)} \\
 x + \bar{x} &= 1, & x \cdot \bar{x} &= 0 && \text{(complement)} \\
 x \cdot (y + z) &= xy + xz, & x + yz &= (x + y)(x + z) && \text{(distribution)} \\
 \overline{x \cdot y} &= \bar{x} + \bar{y}, & \overline{x + y} &= \bar{x} \cdot \bar{y} && \text{(De Morgan)}
 \end{aligned}$$

Claude Shannon's 1938 master's thesis showed that this algebra, developed by Boole for logic, describes networks of relay contacts exactly [5]. That observation turned circuit design into a symbolic discipline: a switching network can be written down, simplified algebraically, and built.

2.2 Functional completeness

A set of operators is *functionally complete* if every Boolean function can be expressed with it. The set $\{\text{AND, OR, NOT}\}$ is complete by construction. Any function $f(x_1, \dots, x_n)$ is determined by its truth table, and it equals the OR of one minterm for each row where $f = 1$ (the sum-of-products, or disjunctive normal, form):

$$f(\mathbf{x}) = \sum_{\mathbf{m}: f(\mathbf{m})=1} \prod_{i=1}^n \ell_i(\mathbf{m}), \quad \ell_i(\mathbf{m}) = \begin{cases} x_i & m_i = 1 \\ \bar{x}_i & m_i = 0 \end{cases}$$

It follows that any single operator from which AND, OR and NOT can all be built is complete on its own. NAND, $\overline{a \cdot b}$, qualifies:

$$\bar{a} = \overline{a \cdot a}, \quad a \cdot b = \overline{\overline{a \cdot b} \cdot \overline{a \cdot b}}, \quad a + b = \overline{\bar{a} \cdot \bar{b}} = \overline{\overline{a \cdot a} \cdot \overline{b \cdot b}}.$$

The last identity is De Morgan's law read backwards. NOR is complete by the dual argument. Henry Sheffer published the result for the "stroke" operator in 1913 [4]; Charles Sanders Peirce had noted it decades earlier in manuscripts that were published only posthumously.

2.3 The NAND gate in CMOS

NAND is also the natural gate of complementary metal-oxide-semiconductor (CMOS) logic, which is why it makes a sensible primitive and not just a mathematical curiosity. A CMOS gate pairs a pull-up network of p-channel transistors, which conduct when their gate is low, with a dual pull-down network of n-channel transistors, which conduct when their gate is high. For NAND (Figure 2.1), the output is pulled to ground only when both series NMOS devices conduct, meaning $A = B = 1$; otherwise at least one parallel PMOS device connects the output to V_{DD} . Exactly one network conducts for every input, so the output is always driven to a full logic level. This *restoring* property lets gates be cascaded indefinitely without the signal degrading [8].

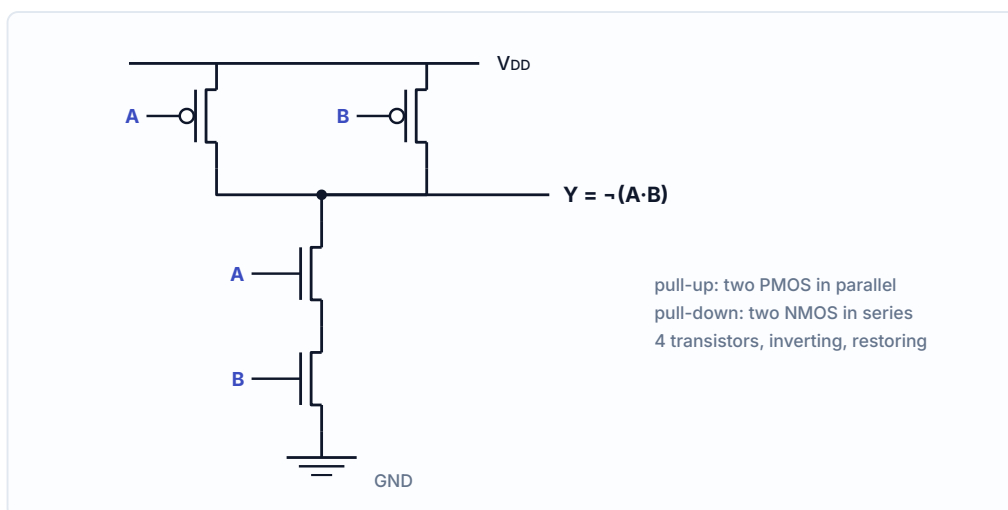


Figure 2.1. Static CMOS NAND gate: four transistors, series NMOS pull-down, parallel PMOS pull-up. Every CMOS gate is naturally inverting; an AND therefore costs a NAND plus an inverter (six transistors).

2.4 Deriving the other gates

Listing 2.1 defines the primitive and the derived gates. Under the `SIM` macro, `nand2` carries a one-unit propagation delay. That delay matters for simulating the feedback circuits of Chapter 5 and is omitted for synthesis. Figures 2.2–2.5 are the synthesized results: in each, the only cell type is the NAND gate. Table 2.1 summarizes the cost of every derived function.

Listing 2.1. The primitive and the first three derived gates.

gates.sv

```
// gates.sv - everything below is built from a single primitive: the 2-input NAND.
module nand2 (input logic a, b, output logic y);
`ifdef SIM
    assign #1 y = ~(a & b);          // unit gate delay for simulation only
`else
    assign y = ~(a & b);
`endif
endmodule

module not_n (input logic a, output logic y);
    nand2 g1 (.a(a), .b(a), .y(y));
endmodule

module and_n (input logic a, b, output logic y);
    logic n;
    nand2 g1 (.a(a), .b(b), .y(n));
    nand2 g2 (.a(n), .b(n), .y(y));
endmodule

module or_n (input logic a, b, output logic y);
    logic na, nb;
    nand2 g1 (.a(a), .b(a), .y(na));
    nand2 g2 (.a(b), .b(b), .y(nb));
    nand2 g3 (.a(na), .b(nb), .y(y));
endmodule
```

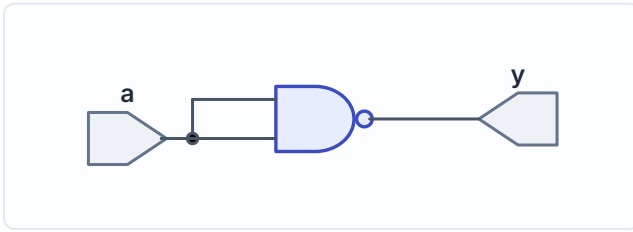


Figure 2.2. NOT: a NAND with its inputs tied together.

YOSYS → NETLISTSVG

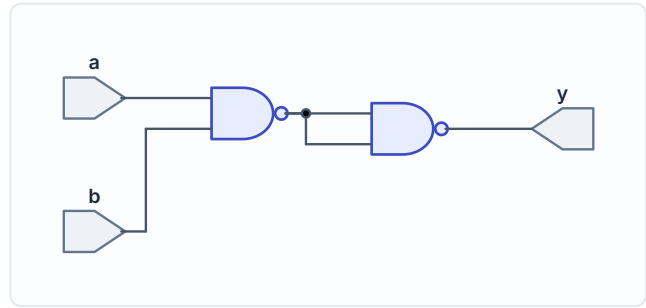


Figure 2.3. AND: NAND followed by NAND-as-inverter.

YOSYS → NETLISTSVG

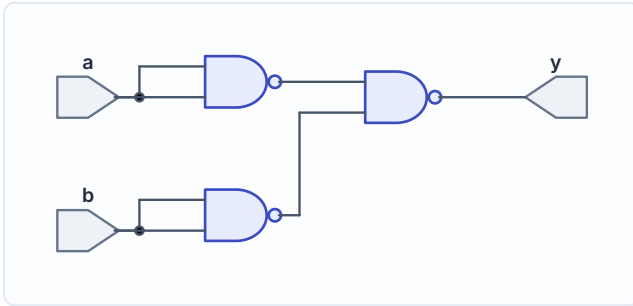


Figure 2.4. OR via De Morgan: invert both inputs, then NAND.

YOSYS → NETLISTSVG

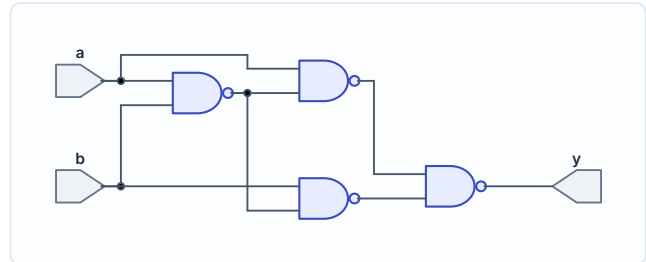


Figure 2.5. XOR in four NANDs. The first gate's output $m = \overline{ab}$ is shared by the two middle gates.

YOSYS → NETLISTSVG

The four-gate XOR merits a derivation, because its trick — reusing one intermediate term twice — reappears in the full adder. With $m = \overline{ab}$, the middle gates compute $p = \overline{a}m$ and $q = \overline{b}m$, and the output is

$$y = \overline{p}q = am + bm = (a + b)\overline{ab} = \overline{a}b + a\overline{b} = a \oplus b.$$

Table 2.1. Cost of each derived function in NAND gates, as synthesized. A CMOS NAND uses four transistors.

Function	NAND gates	Logic depth	Construction
NOT	1	1	$\overline{aa}$
AND	2	2	NAND, then NOT
OR	3	2	$\overline{\overline{a}b}$
XOR	4	3	shared $\overline{ab}$ term
2:1 MUX	4	3 (from s), 2 (from data)	AND-OR with inverted select
Half adder	5	3	XOR plus $\overline{m}$ as carry
Full adder	9	6 (sum), 5 (carry)	two XORs sharing carry terms

2.5 The multiplexer

The multiplexer is the combinational "if". A 2:1 multiplexer computes $y = \overline{s}a + sb$. Applying De Morgan's law to this sum of products turns it into NAND-NAND form: two first-level NANDs gate the data inputs with $\overline{s}$ and s , and a second-level NAND combines them. One extra NAND supplies $\overline{s}$ (Figure 2.6). Wider multiplexers follow the same pattern: decode the select lines into one-hot enables, AND each enable with its data input, and OR the results. The alternative is a tri-state bus, where each enabled driver connects to a shared wire. Tri-state buses are now largely confined to chip I/O, because field-programmable gate arrays (FPGAs) and standard-cell flows prefer the AND-OR form. Both appear in the synthesized processor: every `mux2` and `mux3` instance of Chapter 7 is a multiplexer tree of this kind.

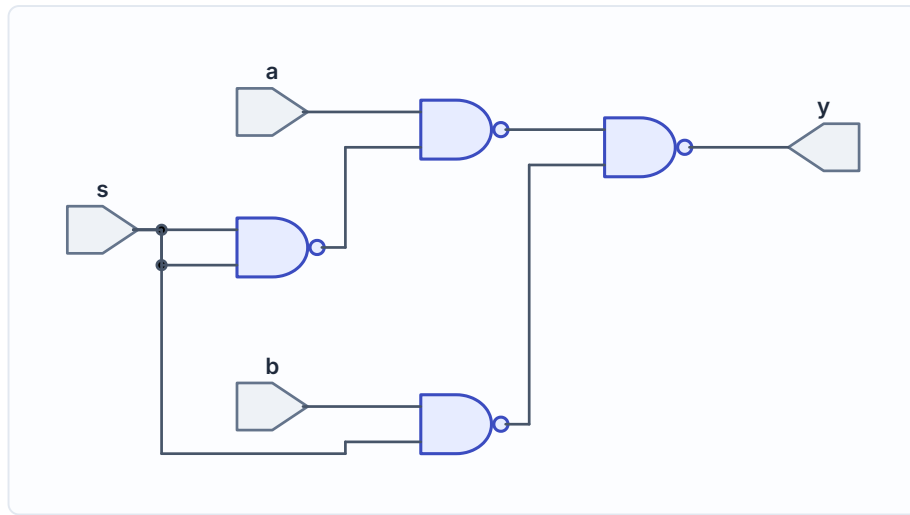


Figure 2.6. 2:1 multiplexer from four NANDs: $y = \overline{\overline{a}\overline{s} \cdot \overline{b}s}$.

YOSYS → NETLISTSVG

2.6 Canonical forms: sum of products and product of sums

Section 2.2 used the fact that every function can be written as an OR of *minterms*. A minterm is a product that contains every variable once, either plain or complemented, and it is 1 for exactly one input row. The dual object is the *maxterm*: a sum that contains every variable once and is 0 for exactly one row. Writing m_i and M_i for the minterm and maxterm of row i , so that $M_i = \overline{m_i}$, every function has two canonical forms:

$$f = \sum_{i: f(i)=1} m_i \quad (\text{canonical sum of products, SoP}), \quad f = \prod_{i: f(i)=0} M_i \quad (\text{canonical product of sums, PoS}).$$

The majority function, which is 1 when at least two of its three inputs are 1, makes a good running example (Table 2.2). It is exactly the full adder's carry. Its canonical forms are

$$y = \sum m(3, 5, 6, 7) = \overline{a}bc + a\overline{b}c + ab\overline{c} + abc, \quad y = \prod M(0, 1, 2, 4) = (a+b+c)(a+b+\overline{c})(a+\overline{b}+c)(\overline{a}+b+c).$$

Table 2.2. Truth table of the majority function with its minterm and maxterm names.

row	a b c	y	minterm m_i	maxterm M_i
0	0 0 0	0	$\overline{a}\overline{b}\overline{c}$	$a + b + c$
1	0 0 1	0	$\overline{a}\overline{b}c$	$a + b + \overline{c}$
2	0 1 0	0	$\overline{a}b\overline{c}$	$a + \overline{b} + c$
3	0 1 1	1	$\overline{a}bc$	$a + \overline{b} + \overline{c}$
4	1 0 0	0	$a\overline{b}\overline{c}$	$\overline{a} + b + c$
5	1 0 1	1	$a\overline{b}c$	$\overline{a} + b + \overline{c}$
6	1 1 0	1	$ab\overline{c}$	$\overline{a} + \overline{b} + c$
7	1 1 1	1	abc	$\overline{a} + \overline{b} + \overline{c}$

Canonical forms are unique, which makes them the natural reference for checking a design, but they are rarely the cheapest circuit. Minimization looks for an equivalent two-level form with fewer terms and literals. It relies on the *combining theorem*:

$$xy + x\overline{y} = x \quad \text{and dually} \quad (x + y)(x + \overline{y}) = x.$$

Two terms that differ in exactly one variable merge into one term without that variable.

2.7 Karnaugh maps

A Karnaugh map [27] arranges a truth table so that the combining theorem becomes visible. Rows and columns are labelled in Gray-code order (00, 01, 11, 10), so cells that touch — including across the left/right and top/bottom edges — differ in exactly one variable. Minimizing a sum of products then becomes a covering problem:

1. Circle groups of 1s. Each group must be rectangular, contain 2^k cells, and may wrap around the edges.
2. Make every group as large as possible. Each maximal group is a *prime implicant*, and a group of 2^k cells eliminates k variables.
3. Cover every 1 using as few groups as possible. A prime implicant that is the only cover of some 1 is *essential* and must be included.
4. Read each group as the product of the variables that stay constant across it.

For a product of sums, group the 0s instead and read each group as the sum of the *complemented* constant variables. *Don't-care* cells (×) are input combinations that can never occur. They may be included in a group whenever that makes the group larger, and otherwise ignored.

Figure 2.7 applies both procedures to two functions. The majority function needs three overlapping groups in either form. The second example is segment e of a decoder from binary-coded decimal (BCD) to a seven-segment display: the lower-left bar of the display, lit for digits 0, 2, 6 and 8. Codes 10–15 never occur in BCD, so they are don't-cares. With them, the four corner cells form one wrap-around group ($\overline{B}\overline{D}$), and the right-hand column forms another ($C\overline{D}$). Table 2.3 compares the costs. The PoS map shows that grouping the 0s is cheaper still: the two middle columns (all $D = 1$) and a 2×2 block give $e = \overline{D}(\overline{B} + C)$. Every group in the figure is checked programmatically — it covers only allowed cells, and the groups together cover every required one — and the minimal forms were confirmed independently with SymPy's `SOPform` and `POSform`.

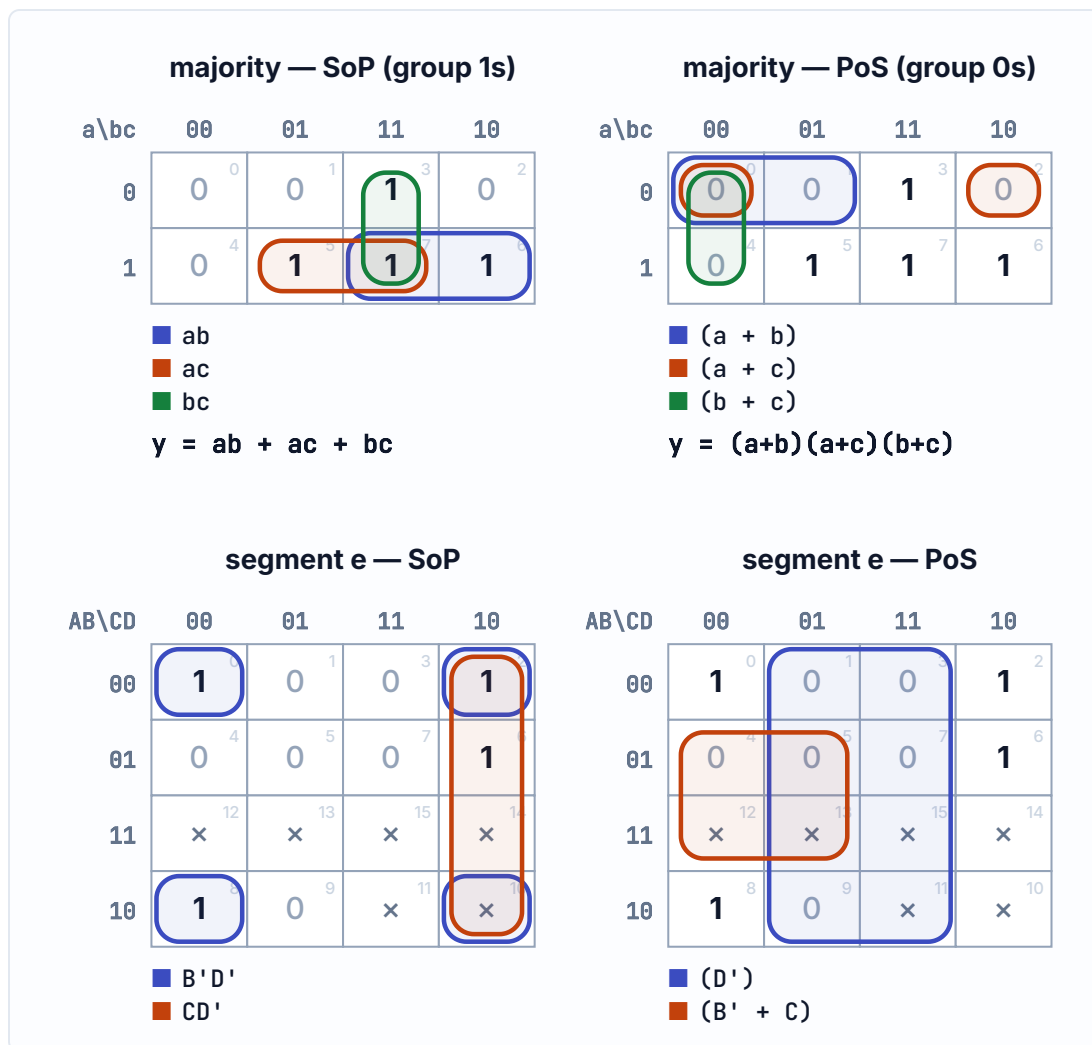


Figure 2.7. Karnaugh maps with prime-implicant groupings. Top: the majority function as SoP (grouping 1s) and PoS (grouping 0s). Bottom: BCD segment *e*, whose don't-cares (x) let the groups grow. Small grey numbers are minterm indices; groups drawn in two pieces wrap around the map edge.

Table 2.3. Cost of the two-level forms of segment *e* (literals counted over all terms).

Form	Expression	Terms	Literals
canonical SoP	$\sum m(0, 2, 6, 8)$	4	16
minimal SoP, no don't-cares	$\overline{A}C\overline{D} + \overline{B}C\overline{D}$	2	6
minimal PoS, no don't-cares	$\overline{D}(\overline{B} + C)(\overline{A} + \overline{C})$	3	5
minimal SoP, with don't-cares	$\overline{B}\overline{D} + C\overline{D}$	2	4
minimal PoS, with don't-cares	$\overline{D}(\overline{B} + C)$	2	3

Maps are practical up to about five or six variables. Beyond that, the same prime-implicant idea is applied algorithmically: Quine–McCluskey tabulation [26, 28] gives exact minima, and heuristic minimizers such as Espresso [29] handle functions with many inputs and outputs. Modern synthesis tools, including Yosys and the ABC logic optimizer it calls, work with multi-level logic rather than strictly two-level forms.

2.8 From two-level forms to gates

A sum of products maps directly onto the NAND-only discipline of this primer. By De Morgan's law, an AND-OR network is identical to a NAND-NAND network, so each product becomes a first-level NAND and the OR becomes a second-level NAND. Dually, a product of sums is a NOR-NOR network. Listing 2.2 gives segment e three ways, and Figures 2.8–2.9 show the synthesized minimal forms. The NAND-NAND version needs five gates: two inverters, two product NANDs and the output NAND. Figures 2.10 and 2.11 show the two minimal majority forms as drawn by Yosys.

Listing 2.2. Segment e : canonical SoP, minimal SoP as NAND-NAND, and minimal PoS.

kmap.sv

```
// BCD-to-7-segment, segment e (lit for digits 0, 2, 6, 8; codes 10-15 never occur).
// (1) canonical sum of products: one minterm per digit that lights the segment
module seg_e_canonical (input logic [3:0] d, output logic e); // d = {A,B,C,D}
    assign e = (~d[3] & ~d[2] & ~d[1] & ~d[0]) // m0
              | (~d[3] & ~d[2] & d[1] & ~d[0]) // m2
              | (~d[3] & d[2] & d[1] & ~d[0]) // m6
              | (d[3] & ~d[2] & ~d[1] & ~d[0]); // m8
endmodule

// (2) minimal SoP from the K-map with don't-cares, e = B'D' + CD', as NAND-NAND
module seg_e_sop_nand (input logic [3:0] d, output logic e);
    logic nb, nd, t1, t2;
    nand2 i1 (.a(d[2]), .b(d[2]), .y(nb)); // B'
    nand2 i2 (.a(d[0]), .b(d[0]), .y(nd)); // D'
    nand2 p1 (.a(nb), .b(nd), .y(t1)); // NOT(B'D')
    nand2 p2 (.a(d[1]), .b(nd), .y(t2)); // NOT(CD')
    nand2 s (.a(t1), .b(t2), .y(e)); // OR of the two products
endmodule

// (3) minimal PoS from the K-map (grouping the zeros), e = D'(B' + C)
module seg_e_pos (input logic [3:0] d, output logic e);
    assign e = ~d[0] & (~d[2] | d[1]);
endmodule
```

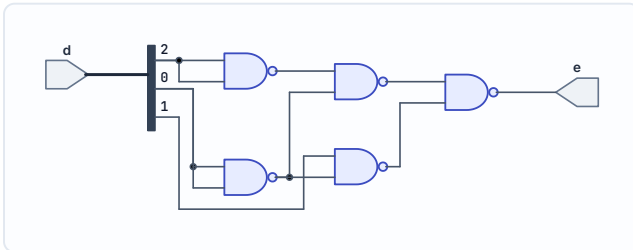


Figure 2.8. Segment e , minimal SoP realized as NAND-NAND: $\overline{B}$ and $\overline{D}$ come from NAND inverters.

YOSYS → NETLISTSVG

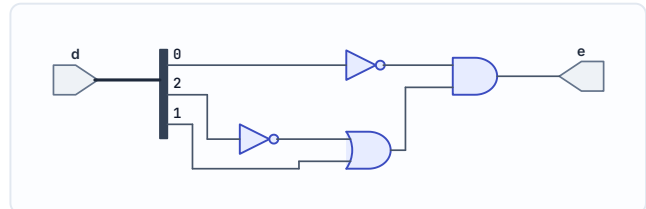


Figure 2.9. Segment e , minimal PoS: $\overline{D} \cdot (\overline{B} + C)$.

YOSYS → NETLISTSVG

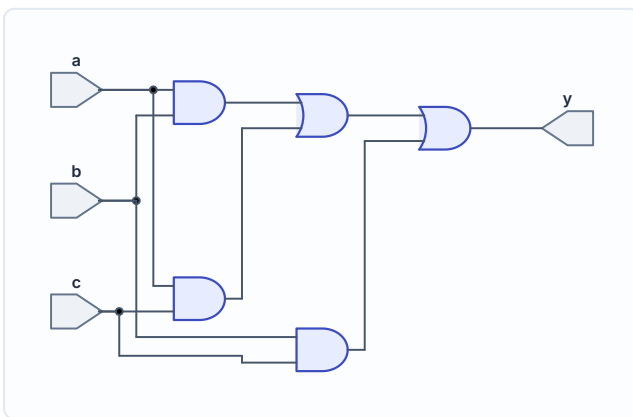


Figure 2.10. Majority, minimal SoP: $ab + ac + bc$.

YOSYS → NETLISTSVG

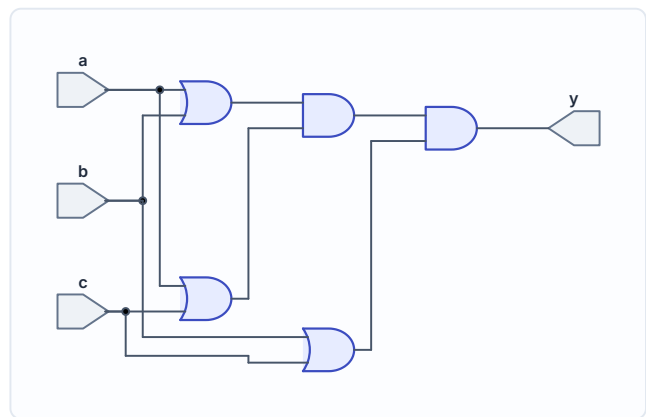


Figure 2.11. Majority, minimal PoS: $(a + b)(a + c)(b + c)$.

YOSYS → NETLISTSVG

Two formal proofs (Section 10.3) close the loop. On every valid BCD code, the two minimized forms of segment e equal the canonical form; codes 10–15 are deliberately left unconstrained, because they are don't-cares. And the minimal SoP and PoS majority forms both equal the carry output of the nine-NAND full adder of Chapter 4, which computes the same function in a different, multi-level structure.

CHAPTER 3

Number Systems and Computer Arithmetic

Every signal in the processor is a bit, and every number it handles is a pattern of bits that means something only under a convention. This chapter fixes those conventions before any arithmetic hardware is built:

- positional binary and hexadecimal notation;
- unsigned integers, and three ways of writing signed ones, of which two's complement wins;
- what sign extension, negation and overflow mean for a fixed word width;
- how a RISC-V processor, which has no condition flags, deals with all of this;
- fixed-point fractions, which lead to the floating-point numbers of Chapter 9.

3.1 Positional number systems

A number written with digits $d_{n-1} \dots d_1 d_0$ in base β has the value

$$v = \sum_{i=0}^{n-1} d_i \beta^i, \quad 0 \leq d_i < \beta.$$

Digital hardware uses base 2, because a wire carries one of two levels. Binary strings are long, so people write them in hexadecimal (base 16, digits 0–9 and A–F), where each hex digit stands for exactly four bits. For example, $45 = 32 + 8 + 4 + 1 = 101101_2 = 0x2D$.

- **Decimal to binary.** Divide repeatedly by 2 and read the remainders from last to first.
- **Binary to hexadecimal.** Group the bits in fours from the right.

Four bits make a *nibble*, eight a *byte*, and the processor's natural width, 32 bits for RV32, is its *word*. An n -bit word has 2^n distinct patterns; everything below is a choice about which numbers those patterns stand for.

3.2 Unsigned integers

Read as an unsigned integer, an n -bit word takes the positional value above, so it covers $0 \dots 2^n - 1$: 0 to 15 for 4 bits, and 0 to 4 294 967 295 for 32 bits. Hardware addition keeps only n bits, so unsigned arithmetic is arithmetic *modulo* 2^n . When the true sum does not fit, the carry out of the top bit is 1 and the stored result has wrapped around. For example, in 4 bits $13 + 6 = 19$, which is stored as $19 - 16 = 3$ with carry 1. The carry-out is therefore the unsigned overflow indicator (Table 3.2).

3.3 Signed integers

Three conventions have been used to give bit patterns a sign (Table 3.1):

- **Sign-magnitude.** The top bit is the sign and the rest is the magnitude, as in handwriting.
- **Ones' complement.** A negative number is the bitwise inverse of its magnitude.
- **Two's complement.** The top bit carries a *negative* weight:

$$\langle x \rangle = -x_{n-1} 2^{n-1} + \sum_{i=0}^{n-2} x_i 2^i, \quad -2^{n-1} \leq \langle x \rangle \leq 2^{n-1} - 1.$$

Table 3.1. All 4-bit patterns under the four interpretations.

bits	hex	unsigned	sign-magnitude	ones' complement	two's complement
0000	0	0	0	0	0
0001	1	1	+1	1	1
0010	2	2	+2	2	2
0011	3	3	+3	3	3
0100	4	4	+4	4	4
0101	5	5	+5	5	5
0110	6	6	+6	6	6
0111	7	7	+7	7	7
1000	8	8	-0	-7	-8
1001	9	9	-1	-6	-7
1010	A	10	-2	-5	-6
1011	B	11	-3	-4	-5
1100	C	12	-4	-3	-4
1101	D	13	-5	-2	-3
1110	E	14	-6	-1	-2
1111	F	15	-7	-0	-1

Sign-magnitude and ones' complement both have two zeros (+0 and -0), and neither can be added with a plain binary adder: sign-magnitude needs to compare magnitudes, and ones' complement needs an "end-around" carry. Two's complement has a single zero, and its patterns are the unsigned patterns with the upper half relabelled as negative numbers (Figure 3.1). As a result, *the same adder circuit adds signed and unsigned numbers*, because both are arithmetic modulo 2^n . Only the interpretation of the result, and of overflow, differs. Every modern processor uses two's complement for integers, and so does the rest of this primer.

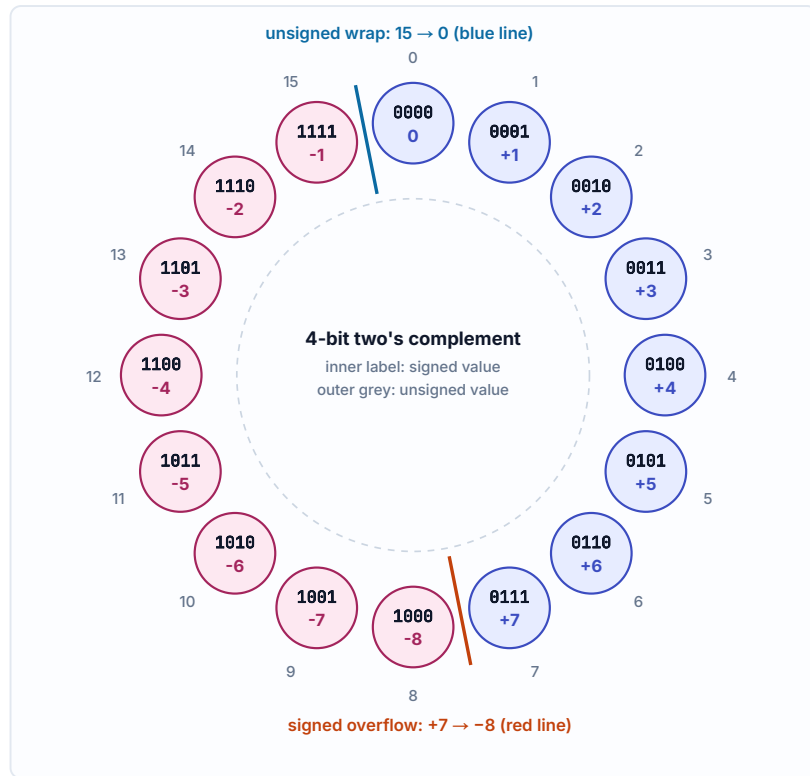


Figure 3.1. The sixteen 4-bit patterns arranged as a wheel. Counting up moves clockwise. Unsigned arithmetic wraps between 15 and 0 (blue); signed arithmetic overflows between +7 and -8 (red).

3.4 Working with two's complement

- **Negation.** $-x = \bar{x} + 1$: invert every bit and add one. Negating the most negative number, -2^{n-1} , gives the same pattern back, because $+2^{n-1}$ has no representation. The 32-bit instance appears in the program of Section 3.7.
- **Subtraction.** $a - b = a + \bar{b} + 1$, so a subtractor is an adder with inverted second input and carry-in 1. This is exactly how the ALU of Section 4.5 works.
- **Sign extension.** A signed value is widened by copying its sign bit into the new upper bits: 1000_2 (-8) becomes 11111000_2 in 8 bits. RISC-V immediates are stored in 12 bits and sign-extended to 32 by the `extend` block of Chapter 6, so `addi x9, x0, -2048` loads `0xFFFFF800`. Unsigned values are widened with zeros instead (zero extension).
- **Comparison.** The same bits order differently under the two readings: 1111_2 is $15 > 1$ unsigned but $-1 < 1$ signed. A processor therefore needs separate signed and unsigned comparisons. RV32I has `slt`/`blt` (signed) and `sltu`/`bltu` (unsigned); Appendix E shows the gate-level difference.

3.5 Overflow

An n -bit adder produces the right answer modulo 2^n . Whether that answer is the *true* answer depends on the interpretation:

- **Unsigned overflow.** The carry-out c_n is 1.
- **Signed overflow (V).** The two operands have the same sign and the result has the opposite sign; equivalently, the carries into and out of the sign position differ:

$$V = \overline{(a_{n-1} \oplus b_{n-1})} (a_{n-1} \oplus s_{n-1}) = c_n \oplus c_{n-1}.$$

Adding two numbers of opposite sign can never overflow. Table 3.2 shows that the two flags are independent: each combination occurs.

Table 3.2. 4-bit additions read both ways. C = carry out (unsigned overflow), V = signed overflow.

bits	sum bits	unsigned reading	signed reading	C	V
0101 + 0010	0111	$5 + 2 = 7 \rightarrow 7$	$5 + 2 = 7 \rightarrow 7$	0	0
0111 + 0001	1000	$7 + 1 = 8 \rightarrow 8$	$7 + 1 = 8 \rightarrow -8$	0	1
1111 + 0001	0000	$15 + 1 = 16 \rightarrow 0$	$-1 + 1 = 0 \rightarrow 0$	1	0
1000 + 1111	0111	$8 + 15 = 23 \rightarrow 7$	$-8 + -1 = -9 \rightarrow 7$	1	1
1101 + 0110	0011	$13 + 6 = 19 \rightarrow 3$	$-3 + 6 = 3 \rightarrow 3$	1	0
1100 + 1100	1000	$12 + 12 = 24 \rightarrow 8$	$-4 + -4 = -8 \rightarrow -8$	1	0

3.6 How a RISC-V processor handles arithmetic

Many architectures record the outcome of every arithmetic operation in condition flags: x86 keeps carry, overflow, zero and sign flags, and Arm keeps N, Z, C and V. Later instructions test those flags. RISC-V deliberately has no flags register [13], and the processors of this primer follow suit:

- **One adder for both.** `add`, `sub` and `addi` produce the same bits whether the operands are meant as signed or unsigned. There is no "signed add".
- **Overflow is ignored by the hardware.** The result simply wraps; no trap and no flag. Software that cares checks explicitly, using idioms given in the RISC-V specification: unsigned `add t0, t1, t2`; `bltu t0, t1, overflow`, and signed `add t0, t1, t2`; `slti t3, t2, 0`; `slt t4, t0, t1`; `bne t3, t4, overflow`. The second works because adding a negative number must make the sum smaller, and adding a non-negative one must not.
- **Comparisons are instructions.** `slt`, `slti` and `beq` (in this subset), plus `sltu`, `bne`, `blt`, `bge`, `bltu` and `bgeu` in full RV32I, compare their operands directly instead of inspecting flags left behind by an earlier subtraction. The ALU's SLT output (Section 4.5) is exactly the signed comparison, including the overflow correction $s_{n-1} \oplus V$.
- **Shifts come in two kinds.** Full RV32I has logical right shifts (`srl`), which fill with zeros, and arithmetic right shifts (`sra`), which copy the sign bit. `sra` by k divides a signed number by 2^k , rounding toward $-\infty$.
- **Wider results.** Multiplication and division, with their signed and unsigned variants, are in the M extension (Appendices A–B). Real numbers need the floating-point representation of Chapter 9.

3.7 Demonstration on the four processors

The program in Listing 3.1 exercises these rules on real hardware. It builds 2^{31} by 31 doublings, then shows that:

- the largest positive number plus one wraps to the most negative;
- negating the most negative number gives it back;
- 12-bit immediates are sign-extended;
- `slt` orders the numbers as signed values.

Finally it applies the specification's signed-overflow test to a deliberate overflow. The `bne` of the idiom is rewritten with this subset's `beq`. The program ran on the reference simulator and on all four processors of this primer — single-cycle, pipelined, pipelined with prediction, and multicycle with either controller — and every register and the stored memory word agreed. Table 3.3 lists the results.

Listing 3.1. Number-system demonstration program (23 instructions).

sim/numdemo.py

```

    addi x1, x0, 1      # x1 = 1
    addi x31, x0, 31    # loop counter
dbl:  add  x1, x1, x1     # 31 doublings: x1 = 2^31 = 0x80000000
      addi x31, x31, -1
      beq  x31, x0, done
      jal  x0, dbl
done:  addi x2, x1, -1    # 0x7FFFFFFF: largest positive number
      addi x3, x2, 1     # +1 wraps to 0x80000000: most negative number
      slt  x4, x3, x2     # 1: most negative < largest positive
      slt  x5, x2, x3     # 0
      sub  x6, x0, x3     # -(most negative) overflows back to itself
      addi x7, x0, -1     # 12-bit immediate -1 sign-extends to 0xFFFFFFFF
      slt  x8, x7, x0     # 1: -1 < 0 as signed numbers
      addi x9, x0, -2048  # most negative 12-bit immediate: 0xFFFFF800
      add  x10, x7, x7    # -1 + -1 = -2 (the carry out of bit 31 is discarded)
      sub  x11, x0, x7    # -(-1) = 1
      add  x12, x2, x2    # positive + positive overflows to -2
      slti x13, x2, 0     # overflow test (RISC-V spec idiom): is the addend negative?
      slt  x14, x12, x2   # ... is the sum less than the other operand?
      beq  x13, x14, nov  # equal answers: no overflow
      addi x15, x0, 1     # x15 = 1 flags the signed overflow
nov:   sw   x12, 0(x0)    # store the wrapped sum in memory word 0
halt:  beq  x0, x0, halt

```

Table 3.3. Final register values, identical on the reference simulator and all four processors.

reg	bits	unsigned	signed	meaning
x1	0x80000000	2 147 483 648	-2 147 483 648	2^{31} built by 31 doublings
x2	0x7FFFFFFF	2 147 483 647	2 147 483 647	largest positive
x3	0x80000000	2 147 483 648	-2 147 483 648	$2^{31} - 1 + 1$ wraps to the most negative number
x4	0x00000001	1	1	most negative < largest positive
x5	0x00000000	0	0	...and not the other way round
x6	0x80000000	2 147 483 648	-2 147 483 648	$-(-2^{31})$ overflows back to itself
x7	0xFFFFFFFF	4 294 967 295	-1	12-bit immediate -1, sign-extended
x8	0x00000001	1	1	$-1 < 0$ as signed numbers
x9	0xFFFFF800	4 294 965 248	-2 048	most negative 12-bit immediate
x10	0xFFFFFFF0	4 294 967 294	-2	$-1 + (-1)$; the carry out of bit 31 is discarded
x11	0x00000001	1	1	$-(-1)$
x12	0xFFFFFFF0	4 294 967 294	-2	largest positive $\times 2$: signed overflow
x13	0x00000000	0	0	overflow test: addend negative?
x14	0x00000001	1	1	overflow test: sum < other operand?
x15	0x00000001	1	1	overflow detected (the two tests disagree)

3.8 Fixed-point fractions

Integers can also represent fractions if both sides agree on an implied binary point. In *fixed-point* $Qm.n$ format, an integer k stands for $k \cdot 2^{-n}$. For example, 16-bit $Q8.8$ stores 1.5 as 0x0180.

- **Addition** of equal formats is plain integer addition.
- **Multiplication** doubles the fraction bits, so the product must be shifted right by n .
- **Division** must pre-shift the dividend.

Fixed point is fast and exact within its range, and it is common in signal processing and small microcontrollers. Its range and resolution are fixed together, though: Q8.8 cannot hold both 10^{-6} and 10^6 . Scientific computation needs a representation whose binary point *floats*, which is the subject of Chapter 9.

CHAPTER 4

Combinational Arithmetic

4.1 Binary numbers and two's complement

Chapter 3 introduced the number representations; this section collects the facts the adders below rely on.

An n -bit vector $x_{n-1} \dots x_0$ denotes the unsigned integer $\sum_i x_i 2^i$ or, in two's complement, the signed integer

$$\langle x \rangle = -x_{n-1} 2^{n-1} + \sum_{i=0}^{n-2} x_i 2^i, \quad -2^{n-1} \leq \langle x \rangle \leq 2^{n-1} - 1.$$

The attraction of two's complement is that addition works identically on signed and unsigned values, since both are arithmetic modulo 2^n . Negation is $-x = \bar{x} + 1$, so a subtractor is simply an adder with inverted second operand and carry-in 1. A signed sum overflows exactly when both operands have the same sign and the result has the other sign; equivalently, when the carries into and out of the sign position differ:

$$V = \overline{(a_{n-1} \oplus b_{n-1})} \cdot (a_{n-1} \oplus s_{n-1}) = c_n \oplus c_{n-1}.$$

4.2 Half and full adders

A half adder adds two bits: $s = a \oplus b$ and $c = ab$. In NAND form the carry comes almost for free, because the XOR's first gate already computes $\overline{ab}$ and one more NAND inverts it (Figure 4.1). A full adder also accepts a carry-in:

$$s = a \oplus b \oplus c_{in}, \quad c_{out} = ab + (a \oplus b) c_{in}.$$

Two cascaded XORs give the sum. The carry needs only one more gate, because each XOR's first NAND already produces one of the two carry terms in complemented form, $\overline{ab}$ and $\overline{(a \oplus b)c_{in}}$. A final NAND of those two signals yields c_{out} by De Morgan's law. The result is the classic nine-NAND full adder (Listing 4.1, Figure 4.2).

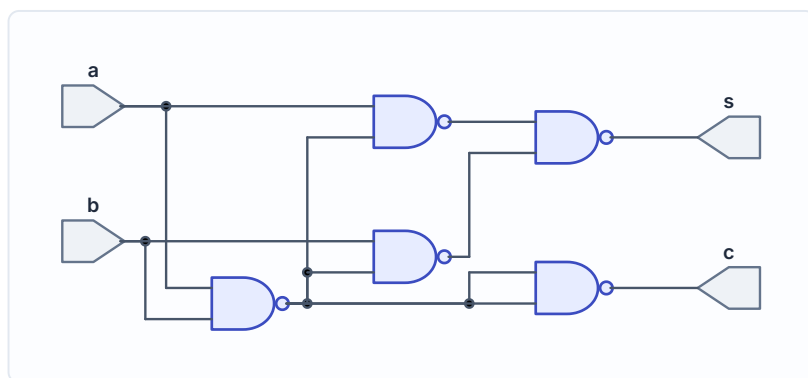


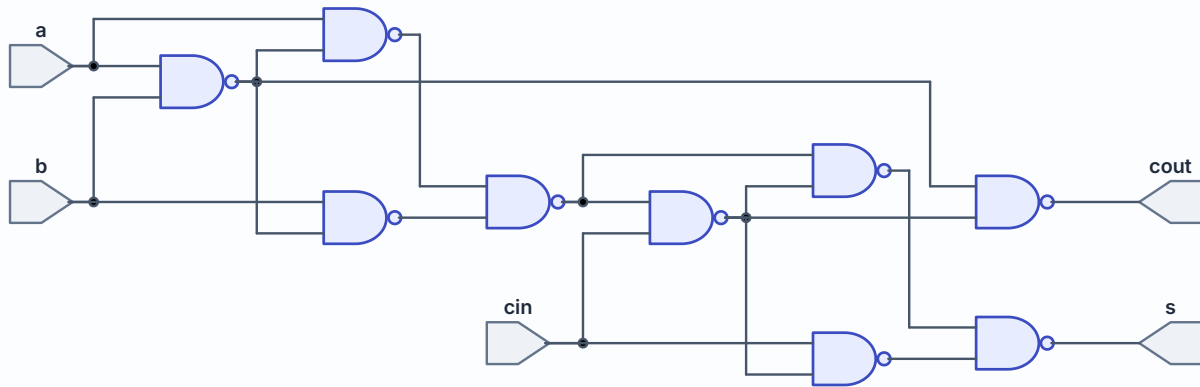
Figure 4.1. Half adder, five NANDs. Gate $g5$ inverts the shared $\overline{ab}$ term to form the carry.

YOSYS → NETLISTSVG

Listing 4.1. Nine-NAND full adder. Gates g1 and g5 each serve both an XOR and the carry.

gates.sv

```
// Nine-NAND full adder: the carry reuses the first NAND of each XOR cell.
module full_adder (input logic a, b, cin, output logic s, cout);
  logic m1, p1, q1, x1, m2, p2, q2;
  nand2 g1 (.a(a), .b(b), .y(m1)); // m1 = ~(a·b)
  nand2 g2 (.a(a), .b(m1), .y(p1));
  nand2 g3 (.a(b), .b(m1), .y(q1));
  nand2 g4 (.a(p1), .b(q1), .y(x1)); // x1 = a ^ b
  nand2 g5 (.a(x1), .b(cin), .y(m2)); // m2 = ~(x1·cin)
  nand2 g6 (.a(x1), .b(m2), .y(p2));
  nand2 g7 (.a(cin), .b(m2), .y(q2));
  nand2 g8 (.a(p2), .b(q2), .y(s)); // s = x1 ^ cin
  nand2 g9 (.a(m1), .b(m2), .y(cout)); // cout = a·b + x1·cin
endmodule
```

**Figure 4.2.** Synthesized nine-NAND full adder. The NAND driving *cout* combines the first NAND of each of the two XOR stages.

YOSYS → NETLISTSVG

4.3 The ripple-carry adder and its delay

Chaining N full adders, each carry-out feeding the next carry-in, produces an N -bit ripple-carry adder (RCA; Figure 4.3). It is as small as an adder can be — $9N$ NANDs — and as slow: the carry may have to propagate through every stage. The gate depth can be read off the full adder. From the operand inputs, c_1 is at most five NANDs deep (g1–g2–g4–g5–g9); each later stage adds two (g5–g9); and a sum bit lies three levels beyond its incoming carry (g5–g6–g8). Hence

$$D(c_N) = 5 + 2(N - 1) = 2N + 3, \quad D(s_{N-1}) = D(c_{N-1}) + 3 = 2N + 4.$$

These closed forms were checked by static timing analysis — a longest-path search over the flattened NAND netlist — for $N = 4, 8, 16, 32, 64$. The measured depths match exactly (Figure 4.4, Table 4.1). Delay is therefore linear in word length, which is unacceptable for a 32- or 64-bit datapath.

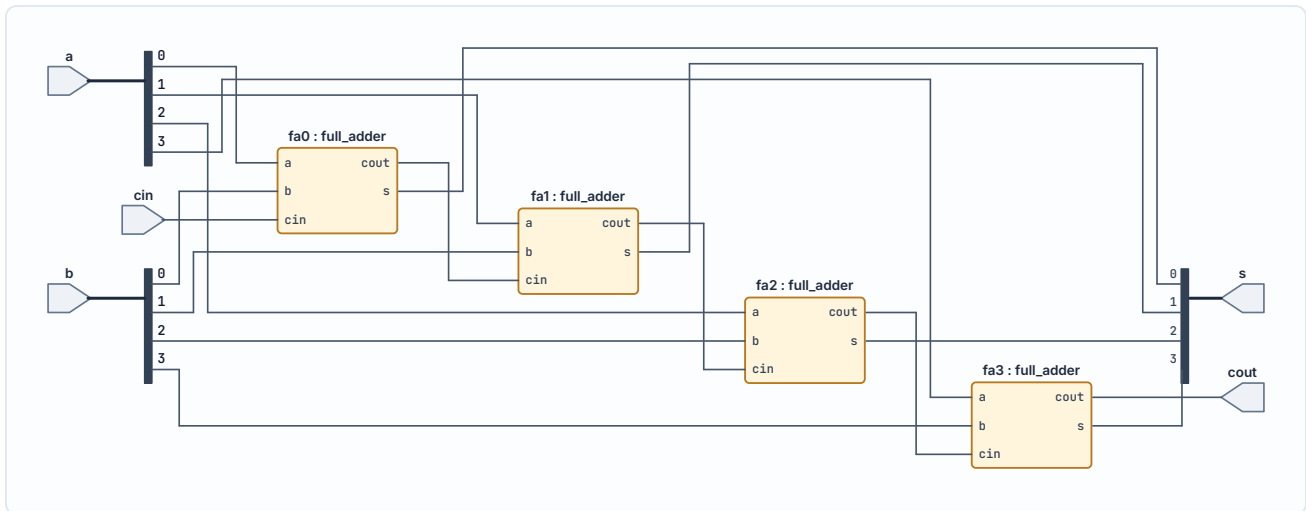


Figure 4.3. 4-bit ripple-carry adder. The four *full_adder* instances are kept as black boxes; the carry chain threads through them from *cin* to *cout*. YOSYS → NETLISTSVG

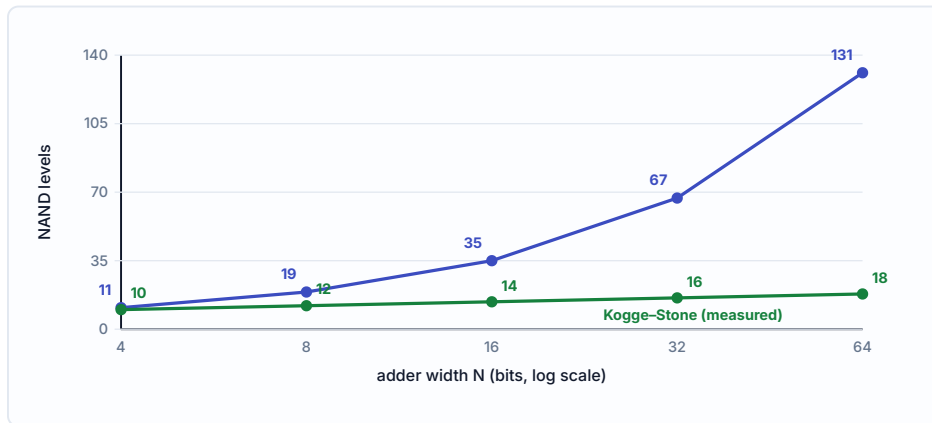


Figure 4.4. Measured NAND depth versus adder width: ripple-carry to carry-out (blue, $2N+3$) and the Kogge–Stone adder of Section 4.6 to its slowest output (green).

Table 4.1. Static timing of the flattened NAND ripple adder.

N	NAND gates	depth to c_N	depth to s_{N-1}
4	36	11	12
8	72	19	20
16	144	35	36
32	288	67	68
64	576	131	132

4.4 Carry-lookahead

Carry-lookahead removes the ripple by computing each carry directly from the inputs. Every bit position either *generates* a carry regardless of its carry-in, or *propagates* an incoming carry:

$$g_i = a_i b_i, \quad p_i = a_i \oplus b_i, \quad c_{i+1} = g_i + p_i c_i.$$

Unrolling the recurrence expresses every carry in two levels of AND-OR logic:

$$\begin{aligned}
c_1 &= g_0 + p_0 c_0 \\
c_2 &= g_1 + p_1 g_0 + p_1 p_0 c_0 \\
c_3 &= g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0 \\
c_4 &= g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0
\end{aligned}$$

Figure 4.5 is the synthesized 4-bit carry-lookahead adder (CLA). Like the ripple adder, it was checked against the arithmetic sum for all 512 combinations of operands and carry-in. The flat form cannot be scaled directly, because the AND and OR fan-in grow linearly with width. Practical adders instead combine (g, p) pairs with the associative operator

$$(g, p) \circ (g', p') = (g + p g', p p'),$$

arranged as a tree. Because the operator is associative, all N carries can be computed in $O(\log N)$ levels: this is the family of *parallel-prefix* adders. Kogge and Stone's construction [10], built and measured in Section 4.6, minimizes depth at the cost of wiring; other members of the family trade depth for area and fan-out [9]. Weinberger and Smith's 1958 carry-lookahead scheme [19] is the historical starting point.

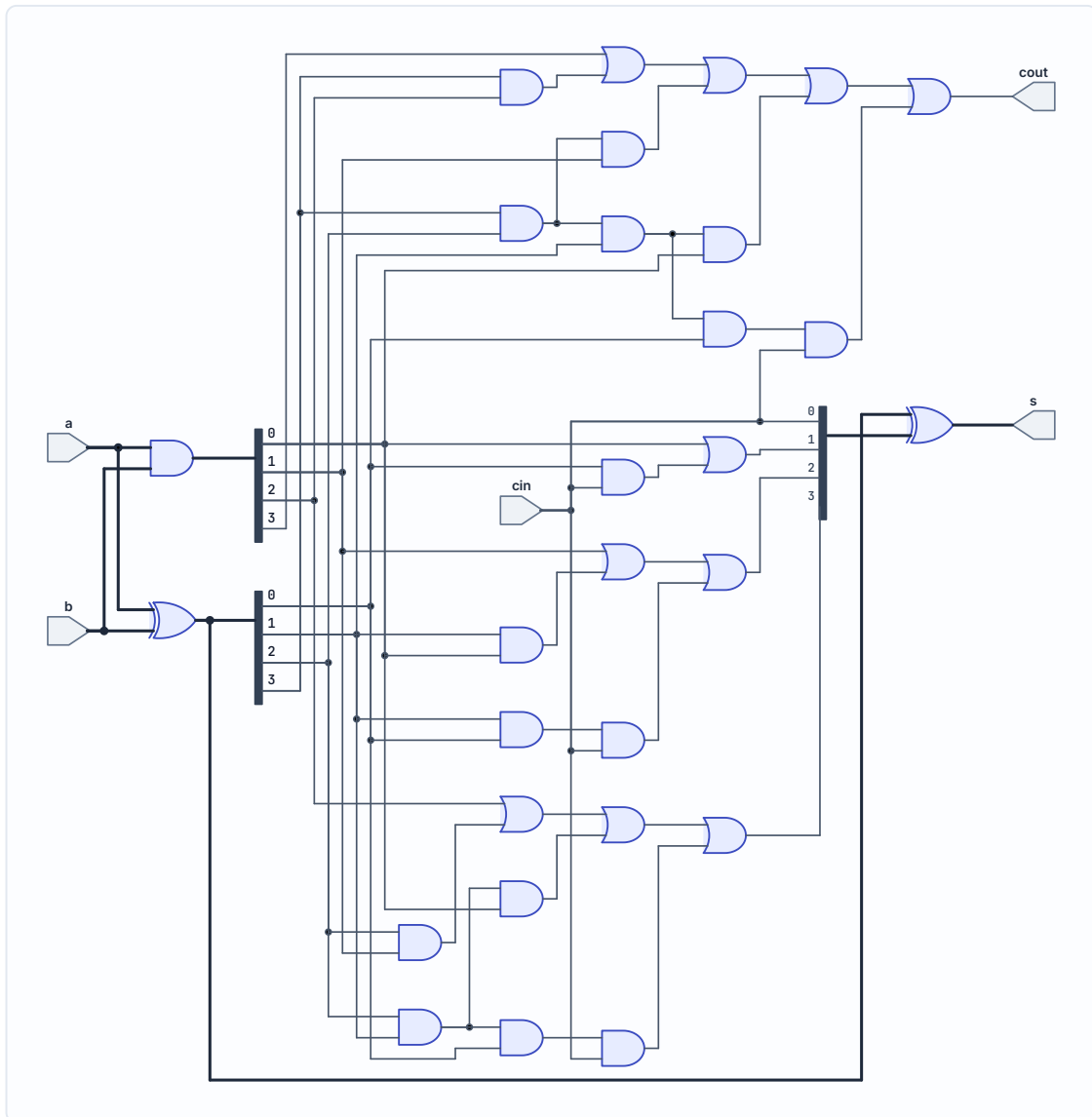


Figure 4.5. 4-bit carry-lookahead adder. The AND and XOR at the far left form the 4-bit vectors g and p , which are split into bits for the AND-OR network of unrolled carry equations; the XOR at the right forms $s = p \oplus c$.

YOSYS → NETLISTSVG

4.5 The arithmetic-logic unit

The ALU is the processor's calculator. Ours implements the five operations the instruction subset needs, selected by a 3-bit control code (Table 4.2). Subtraction reuses the adder: bit 0 of the control code inverts b and supplies the $+1$ carry-in. Set-less-than (SLT) also subtracts, then outputs the true sign of $a - b$. That sign is not simply s_{n-1} , because the subtraction can overflow. When it does, the sign bit is wrong, and the correction is

$$lt = s_{n-1} \oplus V.$$

For example, with 4 bits, $a = -8$ and $b = 1$ give $a - b = -9$, which wraps to $+7$ ($s_3 = 0$) with $V = 1$, so $lt = 1$ as it should be. The `zero` output drives the branch decision. Listing 4.2 is the parameterized RTL. Figure 4.6 is the 4-bit instance, which was checked exhaustively: all 256 operand pairs for each of the eight control codes.

Table 4.2. ALU control encoding (after Harris & Harris [2]).

ctrl	Operation	Result	Used by
000	add	$a + b$	add, addi, lw, sw
001	subtract	$a + \bar{b} + 1$	sub, beq
010	and	$a \& b$	and, andi
011	or	$a b$	or, ori
101	set less than	$s_{n-1} \oplus V$	slt, slti

Listing 4.2. Parameterized ALU with signed-overflow-corrected SLT.

rv_blocks.sv

```
// ALUControl: 000 add, 001 sub, 010 and, 011 or, 101 slt
module alu #(parameter int W = 32) (
  input  logic [W-1:0] a, b,
  input  logic [2:0]   ctrl,
  output logic [W-1:0] y,
  output logic        zero);
  logic [W-1:0] bb, sum;
  logic [W:0]   ext;
  logic        v, lt;
  assign bb = ctrl[0] ? ~b : b; // invert for sub / slt
  // One adder: the extra low bit turns ctrl[0] into the carry-in (+1 completes two's
  // complement). Writing a + bb + ctrl[0] instead synthesizes two chained adders.
  assign ext = {a, 1'b1} + {bb, ctrl[0]};
  assign sum = ext[W:1];
  assign v = ~(a[W-1] ^ bb[W-1]) & (a[W-1] ^ sum[W-1]); // signed overflow
  assign lt = sum[W-1] ^ v; // true sign of a-b
  always_comb
    case (ctrl)
      3'b000, 3'b001: y = sum;
      3'b010:        y = a & b;
      3'b011:        y = a | b;
      3'b101:        y = {{(W-1){1'b0}}, lt};
      default:       y = '0;
    endcase
  assign zero = (y == '0);
endmodule
```

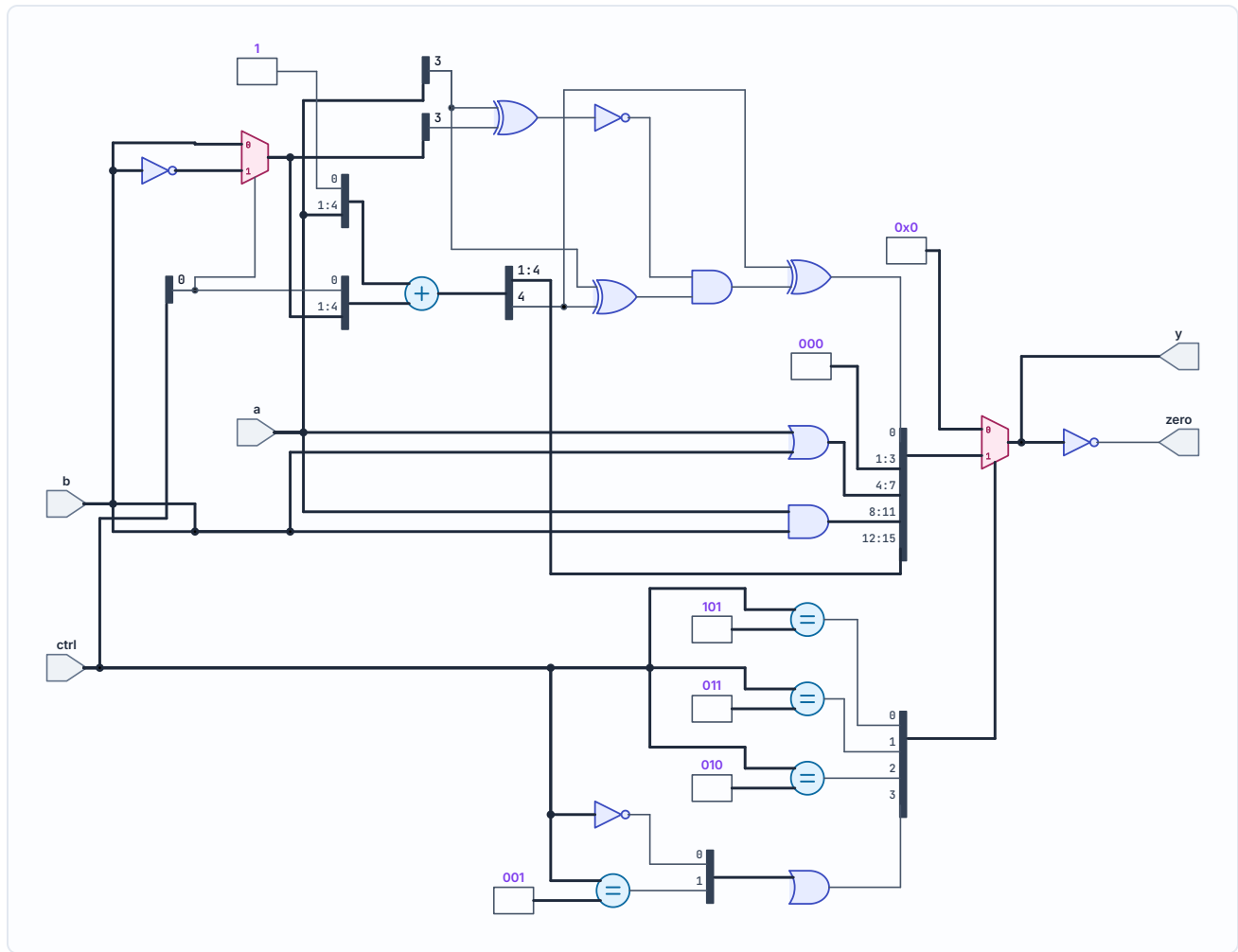


Figure 4.6. The ALU synthesized at $W = 4$. The conditional inverter (top left) and the adder path are shared by add, sub and slt. A single adder does both: an extra low-order bit feeds $ctrl_0$ in as the carry-in. Writing the sum as $a + \tilde{b} + ctrl_0$ would synthesize two chained adders instead (Section 8.11). The comparators at the bottom decode $ctrl$ into one-hot selects for the output multiplexer, and the logical NOT at the right forms *zero*. YOSYS → NETLISTSVG

4.6 Parallel-prefix adders: Kogge–Stone

Section 4.4 introduced the associative operator $(g, p) \circ (g', p')$. A parallel-prefix adder evaluates it over all bit positions as a tree, so the carry into every position is available after $O(\log N)$ levels. The Kogge–Stone construction [10] is the fastest regular arrangement:

- At level l , every position $k \geq 2^l$ combines its own (G, P) with that of position $k - 2^l$.
- The spans therefore double at each level, so $\lceil \log_2(N+1) \rceil$ levels suffice.
- The carry-in enters as an extra position with $g = c_{in}$ and $p = 0$.

Figure 4.7 shows the resulting prefix graph for $N = 8$. Listing 4.3 builds the adder from the same NAND-level cells as the ripple adder, so their depths are directly comparable. Figure 4.9 shows the 4-bit instance as synthesized. The group-generate combine $G = g_{hi} + p_{hi} g_{lo}$ costs three NANDs in two levels (Figure 4.8): an inverter on g_{hi} in parallel with $\overline{p_{hi} g_{lo}}$, then an output NAND.

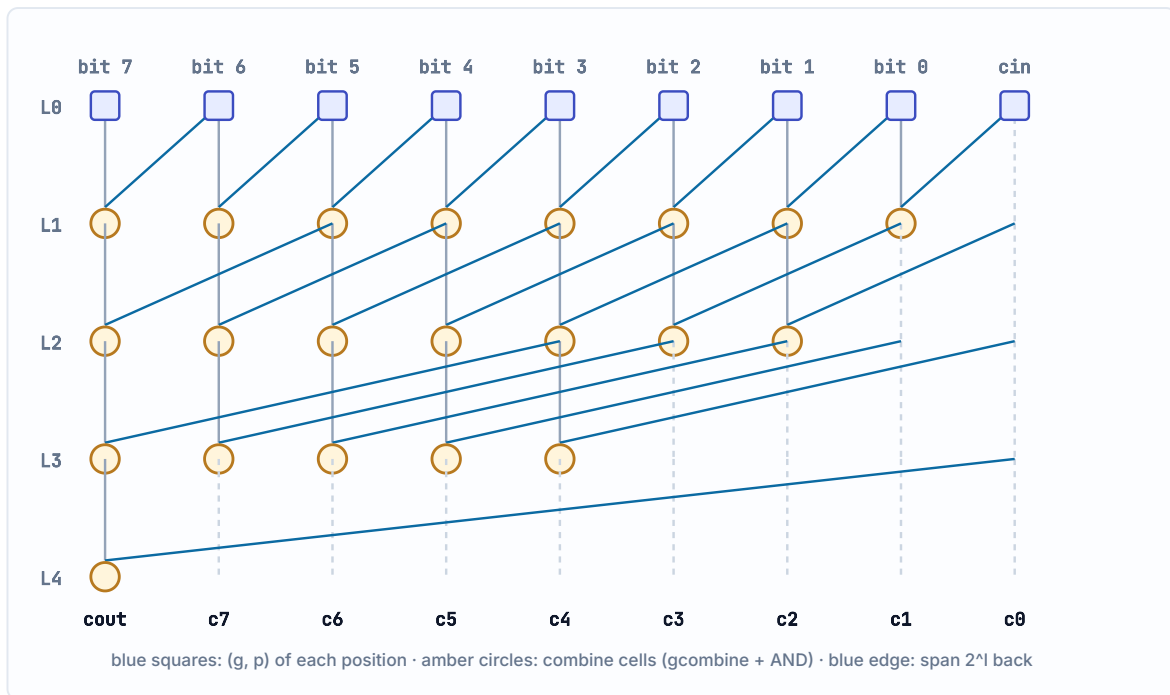


Figure 4.7. Kogge–Stone prefix graph for $N = 8$. Each level doubles the span of the (G, P) pairs; after four levels every position holds the group generate from the carry-in, which is its carry.

Listing 4.3. The combine cell and the parameterized Kogge–Stone adder.

prefix.sv

```
// Group-generate combine in two NAND levels: G = g_hi | (p_hi & g_lo)
// = NAND( NOT g_hi , NAND(p_hi, g_lo) )
module gcombine (input logic g_hi, p_hi, g_lo, output logic g);
    logic ng, t;
    nand2 u1 (.a(g_hi), .b(g_hi), .y(ng));
    nand2 u2 (.a(p_hi), .b(g_lo), .y(t));
    nand2 u3 (.a(ng), .b(t), .y(g));
endmodule

module ks_adder #(parameter int N = 16) (
    input logic [N-1:0] a, b,
    input logic cin,
    output logic [N-1:0] s,
    output logic cout);
    localparam int K = N + 1; // position 0 carries cin (g = cin, p = 0)
    localparam int L = $clog2(K); // number of prefix levels
    logic [N-1:0] g0, p0;
    logic [K-1:0] G [L+1];
    logic [K-1:0] P [L+1];
    for (genvar j = 0; j < N; j++) begin : g_bit
        and_n ug (.a(a[j]), .b(b[j]), .y(g0[j]));
        xor_n up (.a(a[j]), .b(b[j]), .y(p0[j]));
    end
    assign G[0] = {g0, cin};
    assign P[0] = {p0, 1'b0};
    for (genvar l = 0; l < L; l++) begin : g_lv1
        for (genvar k = 0; k < K; k++) begin : g_pos
            if (k >= (1 << l)) begin : g_comb
                gcombine uc (.g_hi(G[l][k]), .p_hi(P[l][k]), .g_lo(G[l][k-(1<<l)]), .g(G[l+1][k]));
                and_n ua (.a(P[l][k]), .b(P[l][k-(1<<l)]), .y(P[l+1][k]));
            end else begin : g_pass
                assign G[l+1][k] = G[l][k];
                assign P[l+1][k] = P[l][k];
            end
        end
    end
    // G[L][k] is the carry into bit k (group generate over positions k..0)
    for (genvar j = 0; j < N; j++) begin : g_sum
        xor_n us (.a(p0[j]), .b(G[L][j]), .y(s[j]));
    end
end
```

```
assign cout = G[L][N];
endmodule
```

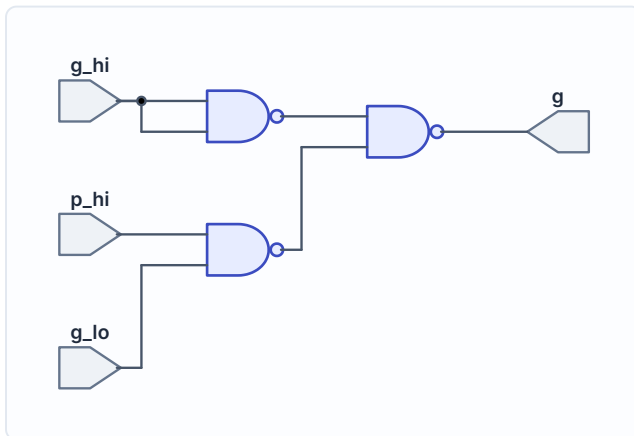


Figure 4.8. The group-generate combine cell: two NAND levels.

YOSYS → NETLISTSVG

Table 4.3. Ripple versus Kogge–Stone, measured on flattened NAND netlists (depth = NAND levels to the slowest output).

N	RCA gates	RCA depth	KS gates	KS depth	KS/RCA gates
4	36	12	72	10	2.0×
8	72	20	169	12	2.3×
16	144	36	398	14	2.8×
32	288	68	931	16	3.2×
64	576	132	2152	18	3.7×

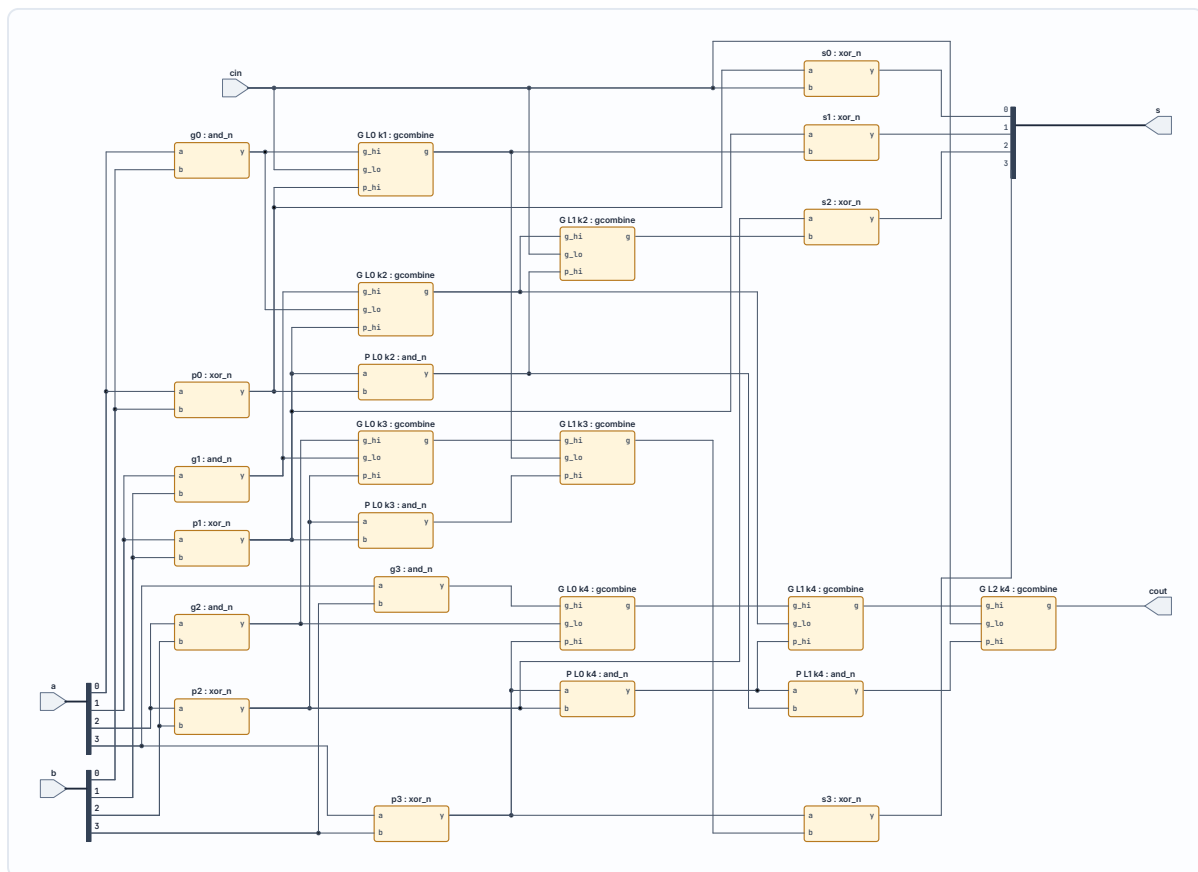


Figure 4.9. The 4-bit Kogge–Stone adder with its cells as black boxes: generate/propagate cells (g , p), group-generate ($G L I k k$) and group-propagate ($P L I k k$) combines per level, and the sum XORs (s).

YOSYS → NETLISTSVG

The measurements in Table 4.3 and the green curve of Figure 4.4 show the trade.

- **Depth.** Kogge–Stone depth grows by two NAND levels per doubling of width, fitting $2 \log_2 N + 6$ over the measured range: 18 levels at 64 bits against the ripple adder's 132.
- **Area.** The price is paid in gates and wiring: 2.0× the ripple adder's gates at 4 bits, rising to 3.7× at 64 bits.
- **Correctness.** The adder was tested exhaustively at 8 bits (131 072 cases) and on 200 000 random 32-bit cases. It was also proved formally equal to the ripple adder and to the $+$ operator for all 2^{65} inputs at 32

bits (Section 10.3).

Other prefix networks — Brent-Kung, Sklansky, Han-Carlson — trade some of that depth for fewer cells or lower fan-out [\[9\]](#).

CHAPTER 5

Sequential Logic

5.1 Feedback and the SR latch

Every circuit so far is combinational: its outputs are a function of its present inputs alone. Memory requires *feedback*. Cross-coupling two NAND gates (Figure 5.1) gives a bistable element, the set-reset (SR) latch with active-low inputs. With $\overline{S} = \overline{R} = 1$, each NAND acts as an inverter of the other's output, and the pair holds whichever state it is in. Pulling $\overline{S}$ low forces $Q = 1$; pulling $\overline{R}$ low forces $Q = 0$. Driving both low forces both outputs high, and releasing them together leaves the final state to a race. That is the "forbidden" input.

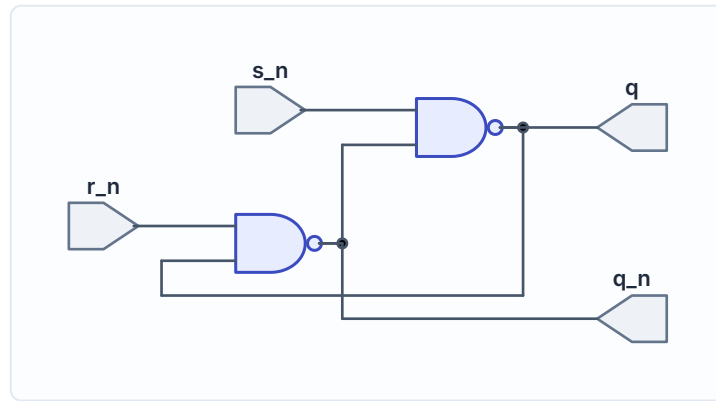


Figure 5.1. Cross-coupled NAND ($\overline{S}\overline{R}$) latch. Each gate's output is one input of the other.
YOSYS → NETLISTSVG

5.2 The gated D latch

The forbidden state is eliminated by deriving $\overline{S}$ and $\overline{R}$ from a single data bit, and a level-sensitive enable is added in the same step:

$$\overline{S} = \overline{D \cdot EN}, \quad \overline{R} = \overline{\overline{D} \cdot EN}.$$

While $EN = 1$ the latch is *transparent*: Q follows D . While $EN = 0$ both set and reset are inactive and the latch holds (Figure 5.2). This five-NAND cell is the classic one-bit storage element used in small hand-built memory arrays. The simulation checks transparency and hold directly.

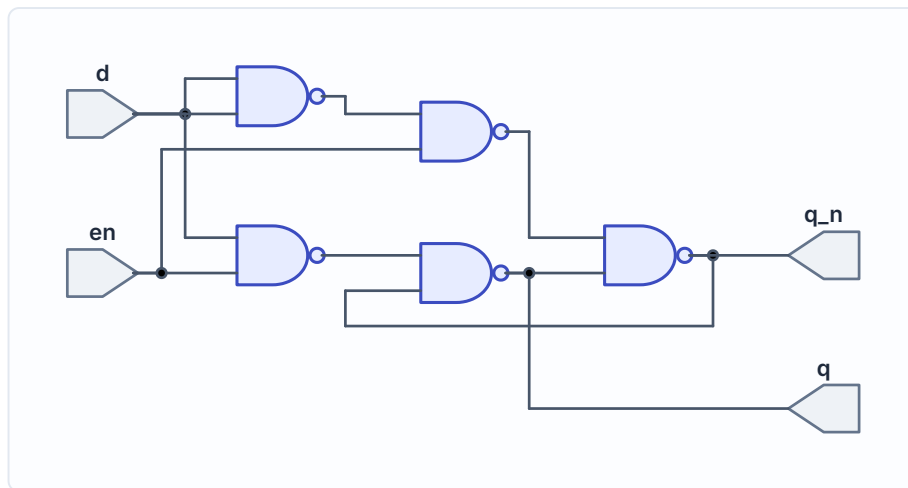


Figure 5.2. Gated D latch in five NANDs: an input stage (inverter plus two gating NANDs) drives the $\bar{S}\bar{R}$ core.

YOSYS → NETLISTSVG

5.3 The edge-triggered flip-flop

A transparent latch is awkward in a synchronous system: while it is open, a change at its input can race through it and through downstream logic within the same clock phase. The master-slave flip-flop closes that path. It uses two latches enabled on opposite clock phases. While the clock is low, the master is transparent and the slave holds. When the clock rises, the master closes on the value present just before the edge, and the slave opens to pass that value to Q (Listing 5.1, Figure 5.3). The composite samples its input only at the rising edge. The testbench confirms both properties: Q does not change when D changes while the clock is low, and it takes the new value at the next rising edge.

Listing 5.1. Gated D latch and positive-edge master-slave flip-flop.

seq.sv

```
module d_latch (input logic d, en, output logic q, q_n);
    logic d_n, s_n, r_n;
    nand2 g0 (.a(d), .b(d), .y(d_n));
    nand2 g1 (.a(d), .b(en), .y(s_n));
    nand2 g2 (.a(d_n), .b(en), .y(r_n));
    nand2 g3 (.a(s_n), .b(q_n), .y(q));
    nand2 g4 (.a(r_n), .b(q), .y(q_n));
endmodule

// Positive-edge master-slave flip-flop from two gated latches
module dff_ms (input logic d, clk, output logic q);
    logic clk_n, m, m_n, q_n;
    nand2 gi (.a(clk), .b(clk), .y(clk_n));
    d_latch master (.d(d), .en(clk_n), .q(m), .q_n(m_n));
    d_latch slave (.d(m), .en(clk), .q(q), .q_n(q_n));
endmodule
```

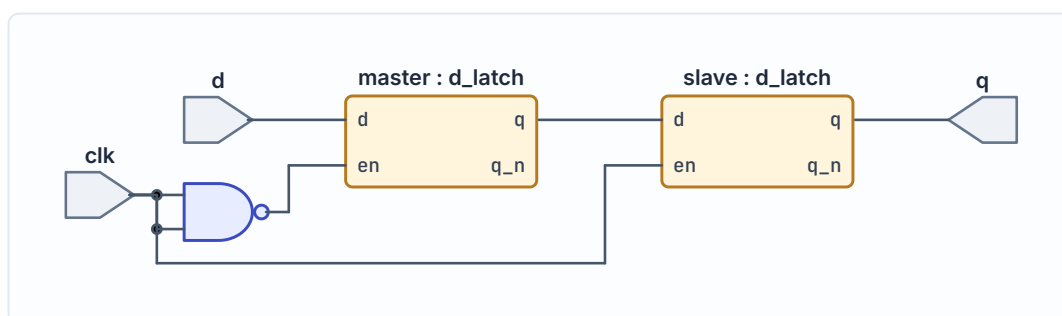


Figure 5.3. Master-slave flip-flop: one NAND inverts the clock for the master latch. YOSYS → NETLISTSVG

5.4 The synchronous timing discipline

A flip-flop has three timing parameters (Figure 5.4). The *setup time* t_{setup} and *hold time* t_{hold} define a window around the clock edge during which D must be stable, and the *clock-to-Q delay* t_{pcq} is the time after the edge before Q is valid. Suppose a combinational path with worst-case propagation delay t_{pd} and best-case contamination delay t_{cd} connects two flip-flops. Correct operation then imposes one constraint on each side of the edge:

$$T_c \geq t_{pcq} + t_{pd} + t_{setup}, \quad t_{ccq} + t_{cd} \geq t_{hold}.$$

The first inequality (setup) sets the maximum clock frequency, and the designer satisfies it by shortening the critical path or lengthening the period. The second (hold) does not involve the clock period at all. A hold violation cannot be fixed by slowing the clock; it must be repaired by adding delay to the short path. Clock skew t_{skew} tightens both inequalities: in the worst case it adds to the right-hand side of each, giving $T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew}$ and $t_{ccq} + t_{cd} \geq t_{hold} + t_{skew}$ [2].

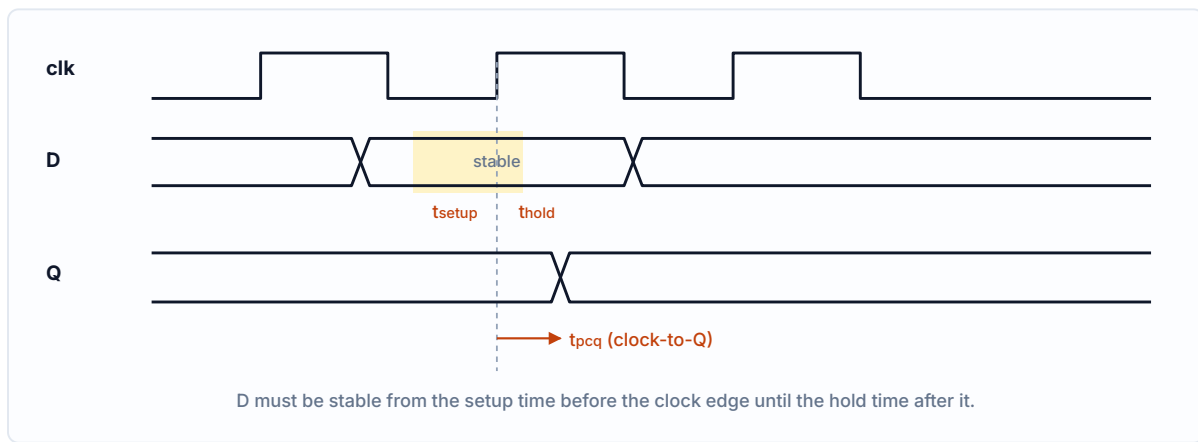


Figure 5.4. Setup and hold window and clock-to-Q delay for a rising-edge flip-flop.

If D changes inside the window, the flip-flop can become *metastable*, balancing between 0 and 1 for an unbounded time. Chaney and Molnar's 1973 measurements [11] are the classic experimental demonstration that such anomalous behaviour occurs in practice. The resolution time is exponentially distributed, so for an asynchronous input the standard synchronizer model gives a mean time between failures

$$\text{MTBF} = \frac{e^{t_r/\tau}}{T_0 f_{clk} f_{data}},$$

where t_r is the resolution time allowed, and τ and T_0 are device constants. A second flip-flop in series roughly doubles t_r and therefore squares the exponential factor. That is why a two-flop synchronizer is the default for every signal crossing into a clock domain.

5.5 Registers and counters

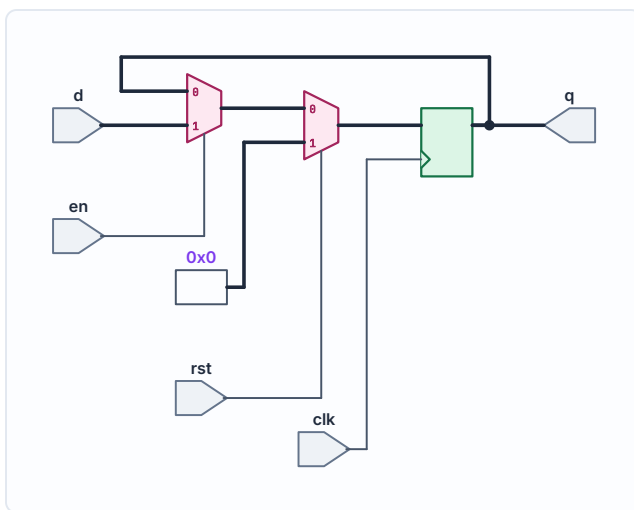
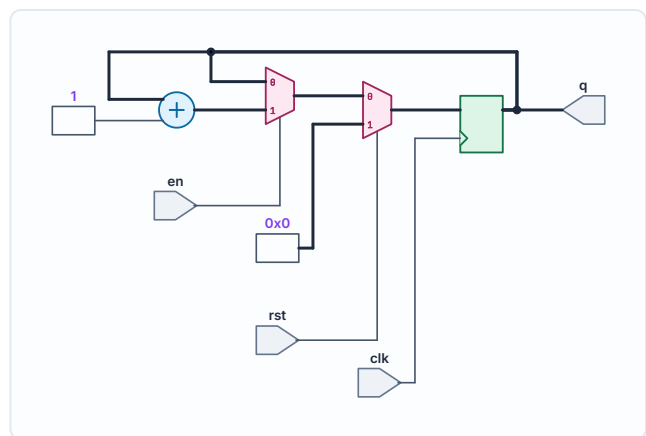
A W -bit register is W flip-flops sharing a clock. Two refinements appear throughout the processor. A *load enable* is a multiplexer that recirculates Q when disabled. A *synchronous reset* is a second multiplexer that selects zero. Figure 5.5 shows both as Yosys builds them from Listing 5.2. A counter adds an incrementer in the feedback path (Figure 5.6). The program counter of Chapter 7 is this structure, except that the next value comes either from $PC + 4$ or from a branch target.

Listing 5.2. Register with synchronous reset and enable; counter.

seq.sv

```
// Behavioral register: synchronous reset, load enable
module register #(parameter int W = 4) (
    input logic      clk, rst, en,
    input logic [W-1:0] d,
    output logic [W-1:0] q);
    always_ff @(posedge clk)
        if (rst)    q ≤ '0;
        else if (en) q ≤ d;
endmodule

module counter #(parameter int W = 4) (
    input logic      clk, rst, en,
    output logic [W-1:0] q);
    always_ff @(posedge clk)
        if (rst)    q ≤ '0;
        else if (en) q ≤ q + 1'b1;
endmodule
```

**Figure 5.5.** 4-bit register: enable multiplexer, then reset multiplexer, then D flip-flops. YOSYS → NETLISTSVG**Figure 5.6.** 4-bit counter: the register of the left figure with $q + 1$ as data. YOSYS → NETLISTSVG

5.6 Register files

A register file is a small addressable memory. Writing uses a decoder: the write address is compared with each row index, and the match, ANDed with the write enable, becomes that row's load enable. Reading uses one multiplexer per read port, selecting a row by address. Figure 5.7 is a 4×4-bit instance with two read ports. It is the same structure as a hand-built grid memory, with a one-hot select per row. The RISC-V register file (Listing 7.3) differs in three ways. It has 32 registers of 32 bits. Register `x0` is hard-wired to zero on read. And its write edge is a parameter. The single-cycle and multicycle cores write on the rising edge. The pipeline writes on the *falling* edge, so a value written in the first half of a cycle can be read in the second half, and Chapter 8 relies on that property.

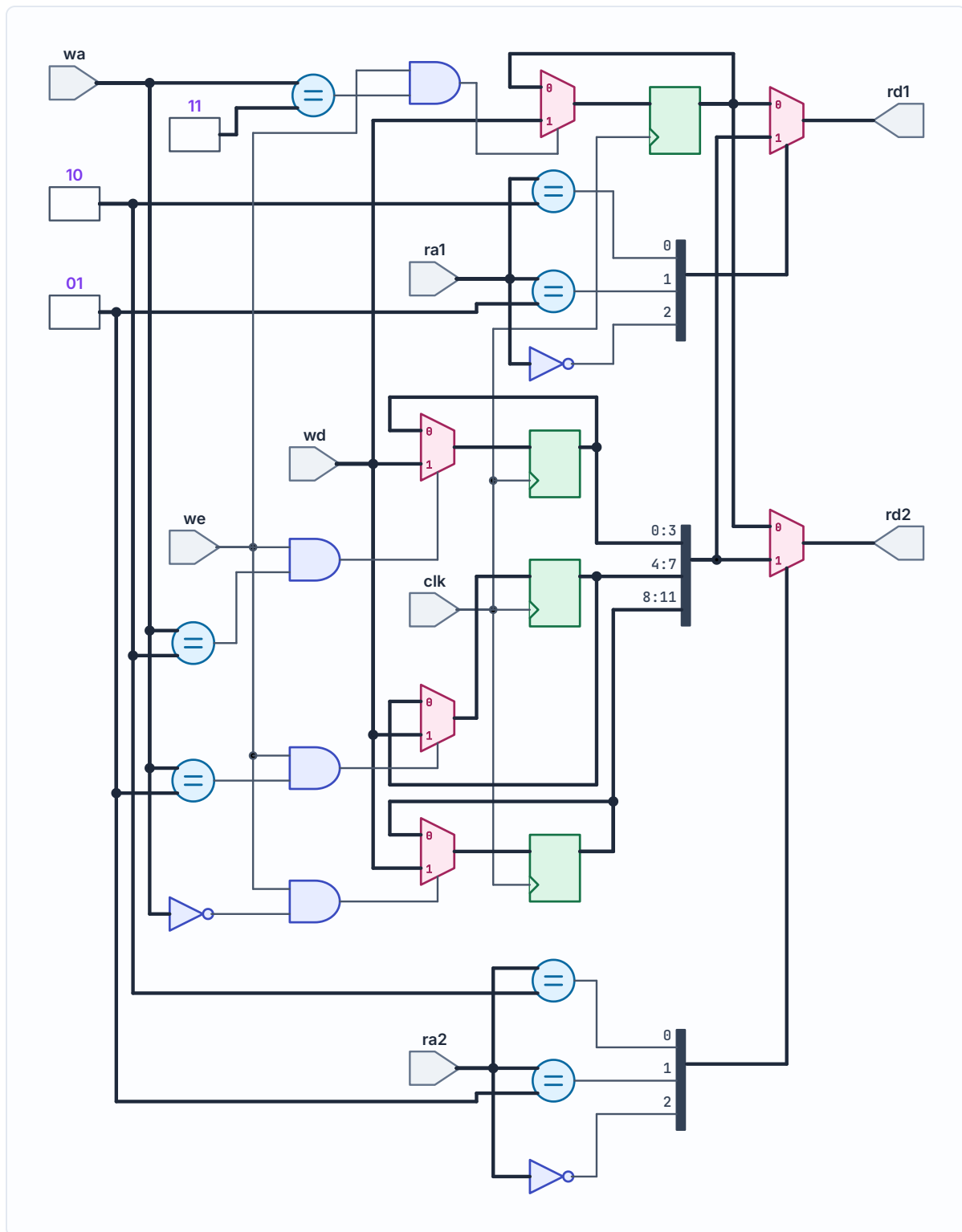


Figure 5.7. 4×4 register file: equality comparators decode the write address; the enabled multiplexers load the selected row; two parallel multiplexers (drawn as one-hot selectors) implement the read ports. YOSYS → NETLISTSVG

CHAPTER 6

The Instruction Set

6.1 Why RISC-V

An instruction set architecture (ISA) is the contract between software and hardware. It fixes the visible state — here a program counter, 32 general registers with `x0` fixed at zero, and a byte-addressed memory — and specifies how each instruction transforms that state. RISC-V is an open, modular ISA from UC Berkeley [13]. Its base integer set RV32I has regular instruction formats that make decoding almost mechanical, and optional extensions such as M (multiply/divide), treated in the appendices, add capability without disturbing the base. Those properties make it the natural teaching target, and they are why recent textbook editions have adopted it [1, 2].

6.2 Instruction formats

Every RV32I instruction is 32 bits long, and the formats share their field positions wherever possible (Figure 6.1). The opcode is always in bits 6–0. The register fields `rd`, `rs1` and `rs2` never move, so the register file can start reading before the instruction has been decoded. The sign bit of every immediate is always bit 31, so sign extension needs no decoding either. The price is that the S, B and J immediates are scattered across the word. The `extend` module (Listing 6.1) reassembles them according to Table 6.1. The B and J immediates are always even, and their implicit zero LSB doubles the reach of branches and jumps.

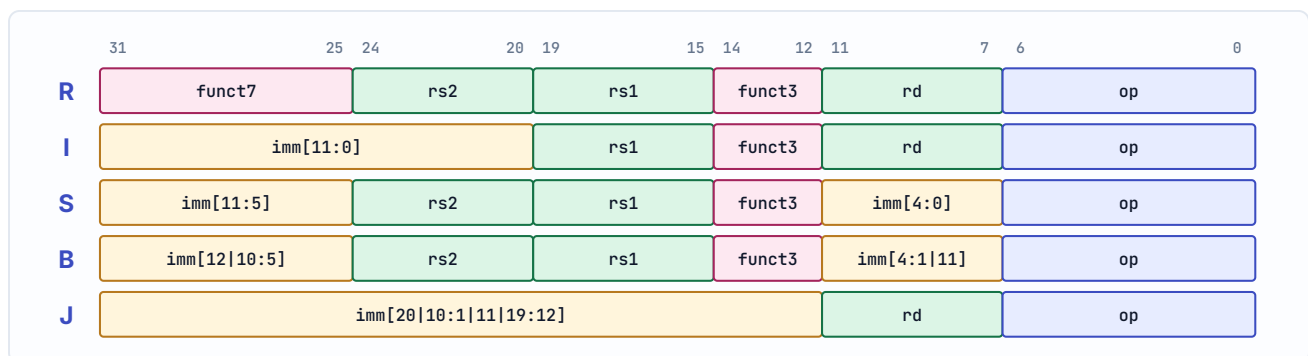


Figure 6.1. The five formats used by the subset. Green fields are register indices, pink are function codes, blue is the opcode, amber are immediate bits (bracketed indices give their position in the assembled immediate).

Table 6.1. Immediate assembly, selected by the 2-bit `ImmSrc`. *s* denotes replication of bit 31.

ImmSrc	Type	ImmExt (32 bits)	Instructions
00	I	{20 s, instr[31:20]}	addi andi ori slti lw
01	S	{20 s, instr[31:25], instr[11:7]}	sw
10	B	{20 s, instr[7], instr[30:25], instr[11:8], 0}	beq
11	J	{12 s, instr[19:12], instr[20], instr[30:21], 0}	jal

Listing 6.1. Immediate extension, written as parallel decode plus a multiplexer.

rv_blocks.sv

```
// ImmSrc: 00 I, 01 S, 10 B, 11 J
module extend (input logic [31:7] instr, input logic [1:0] immsrc,
               output logic [31:0] immext);
    logic [31:0] immI, immS, immB, immJ;
    assign immI = {{20{instr[31]}}, instr[31:20]};
    assign immS = {{20{instr[31]}}, instr[31:25], instr[11:7]};
    assign immB = {{20{instr[31]}}, instr[7], instr[30:25], instr[11:8], 1'b0};
    assign immJ = {{12{instr[31]}}, instr[19:12], instr[20], instr[30:21], 1'b0};
    assign immext = immsrc[1] ? (immsrc[0] ? immJ : immB)
                          : (immsrc[0] ? immS : immI);
endmodule
```

6.3 The test program

The same program (Table 6.2) runs on all four processors. It sums 1 to 10 in a loop with a backward `jal` and a forward `beq`. It then stores the result, reloads it into `x4` and immediately uses `x4`, which is a load-use hazard. It exercises every R- and I-type operation, including a signed comparison against a negative number. It calls forward with `jal x11` over an instruction that must never execute, and it ends in a one-instruction infinite loop. The encodings come from the Python assembler; the reference results come from the Python instruction-set simulator.

Table 6.2. Test program as assembled
(address, machine code, source).

addr	word	instruction
0x0000	00000093	addi x1, x0, 0
0x0004	00100113	addi x2, x0, 1
0x0008	00b00193	addi x3, x0, 11
0x000c	002080b3	add x1, x1, x2
0x0010	00110113	addi x2, x2, 1
0x0014	00310463	beq x2, x3, done
0x0018	ff5ff06f	jal x0, loop
0x001c	06102223	sw x1, 100(x0)
0x0020	06402203	lw x4, 100(x0)
0x0024	004202b3	add x5, x4, x4
0x0028	40128333	sub x6, x5, x1
0x002c	005323b3	slt x7, x6, x5
0x0030	0062f433	and x8, x5, x6
0x0034	0062e4b3	or x9, x5, x6
0x0038	ff900513	addi x10, x0, -7
0x003c	00052633	slt x12, x10, x0
0x0040	fff32693	slti x13, x6, -1
0x0044	00f4f713	andi x14, x9, 0x0f
0x0048	10046793	ori x15, x8, 0x100
0x004c	06502423	sw x5, 104(x0)
0x0050	008005ef	jal x11, skip
0x0054	00000293	addi x5, x0, 0
0x0058	06702623	sw x7, 108(x0)
0x005c	40300833	sub x16, x0, x3
0x0060	07002823	sw x16, 112(x0)
0x0064	00000063	beq x0, x0, halt

The reference final state is $x_1 = 55$, $x_4 = 55$, $x_5 = 110$, $x_6 = 55$, $x_7 = 1$, $x_8 = 38$, $x_9 = 127$, $x_{10} = -7$, $x_{11} = 0x54$, $x_{12} = 1$, $x_{13} = 0$, $x_{14} = 15$, $x_{15} = 294$ and $x_{16} = -11$, with memory words 55, 110, 1 and -11 at addresses 100, 104, 108 and 112. The value $x_5 = 110$ rather than 0 shows that the skipped `addi` did not execute. The program executes 60 instructions, counting the first execution of the halt branch.

CHAPTER 7

The Single-Cycle Processor

7.1 Datapath

A single-cycle processor completes each instruction within one clock period. Its datapath (Listing 7.1, Figure 7.1) is a direct transcription of what an instruction needs, in five steps.

1. **Fetch.** The program counter register addresses instruction memory, and an adder forms $PC + 4$.
2. **Decode.** The `rs1` and `rs2` fields address the register file's read ports, the controller decodes `op`, `funct3` and `funct7[5]`, and `extend` assembles the immediate.
3. **Execute.** A multiplexer selects the ALU's second operand (register or immediate). A second adder computes the branch or jump target $PC + \text{ImmExt}$.
4. **Memory.** The ALU result addresses data memory. `sw` writes the second register value there, and `lw` reads from it.
5. **Write-back.** A three-way multiplexer chooses what is written to `rd`: the ALU result, the memory read, or $PC + 4$ (the link address for `jal`). The next PC is the branch target when $\text{PCSrc} = \text{Branch} \cdot \text{Zero} + \text{Jump}$ is true, and $PC + 4$ otherwise.

Listing 7.1. The complete single-cycle core. Memories are external (see `top_single` in the source).

`rv_single.sv`

```
// rv_single.sv - single-cycle RV32I-subset processor
module rv_single (
    input  logic      clk, rst,
    output logic [31:0] pc,
    input  logic [31:0] instr,
    input  logic [31:0] readdata,
    output logic      memwrite,
    output logic [31:0] aluresult, writedata);

    logic      regwrite, alusrc, branch, jump, zero, pcsrc, memwrite_d;
    logic [1:0] resultsrc, immsrc;
    logic [2:0] alucontrol;
    logic [31:0] pcnext, pcplus4, pctarget, immext, srca, srcb, result;

    controller ctl (.op(instr[6:0]), .funct3(instr[14:12]), .funct7b5(instr[30]),
        .regwrite, .alusrc, .memwrite(memwrite_d), .branch, .jump,
        .resultsrc, .immsrc, .alucontrol);
    assign pcsrc = (branch & zero) | jump;
    assign memwrite = memwrite_d & ~rst; // no architectural writes during reset

    flopr    #(32) pcreg (.clk, .rst, .d(pcnext), .q(pc));
    adder    #(32) pcadd4(.a(pc), .b(32'd4), .y(pcplus4));
    adder    #(32) pcaddb(.a(pc), .b(immext), .y(pctarget));
    mux2     #(32) pcmux (.d0(pcplus4), .d1(pctarget), .s(pcsrc), .y(pcnext));

    regfile #(FALLING(1'b0)) rf (.clk, .we(regwrite & ~rst), .a1(instr[19:15]), .a2(instr[24:20]),
        .a3(instr[11:7]), .wd(result), .rd1(srca), .rd2(writedata));
    extend   ext (.instr(instr[31:7]), .immsrc, .immext);
    mux2     #(32) srcbmx(.d0(writedata), .d1(immext), .s(alusrc), .y(srcb));
    alu      #(32) alu0 (.a(srca), .b(srcb), .ctrl(alucontrol), .y(aluresult), .zero);
    mux3     #(32) resmux(.d0(aluresult), .d1(readdata), .d2(pcplus4), .s(resultsrc), .y(result));
endmodule
```

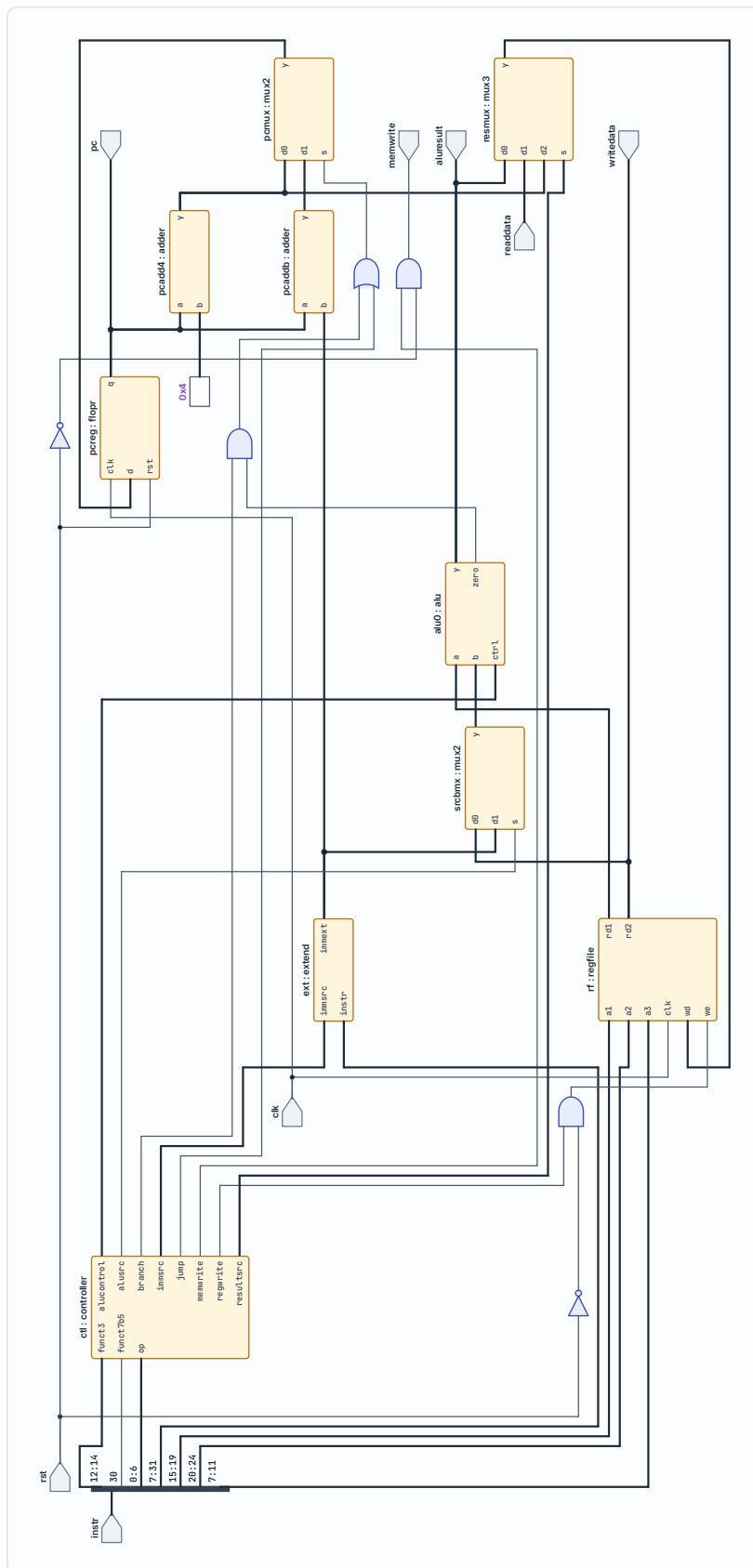


Figure 71. Synthesized single-cycle core with its building blocks kept as black boxes. The instruction bus enters at the left and splits into its fields (7:11 = rd, 15:19 = rs1, 20:24 = rs2, 0:6 = op, 12:14 = funct3, 30 = funct7[5]). The AND and OR gates at the top compute PCSrc. Rotated for legibility.

YOSYS → NETLISTSVG

7.2 Control

The controller is split in two, as in Harris and Harris [2]. A *main decoder* maps the opcode to datapath controls and a 2-bit `ALUOp` (Table 7.1). An *ALU decoder* then combines `ALUOp` with `funct3` and `funct7[5]` to produce the 3-bit ALU control (Table 7.2). Listing 7.2 implements both. Only R-type instructions distinguish `add` from `sub`, which is why the subtract condition is `funct75 · op5`. For `addi`, bit 30 belongs to the immediate and must be ignored: `addi x10, x0, -7` has bit 30 set, and the test program checks that it still adds. Unused controls default to 0, so an all-zero word decodes as a harmless no-op. The pipeline relies on this when it flushes a register to zero.

Table 7.1. Main decoder. Entries shown as 0 are don't-cares that the RTL sets to 0.

Instruction	op	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
R-type	0110011	1	00	0	0	00	0	10	0
I-type ALU	0010011	1	00	1	0	00	0	10	0
lw	0000011	1	00	1	0	01	0	00	0
sw	0100011	0	01	1	1	00	0	00	0
beq	1100011	0	10	0	0	00	1	01	0
jal	1101111	1	11	0	0	10	0	00	1

Table 7.2. ALU decoder.

ALUOp	funct3	funct7 ₅ · op ₅	ALUControl	Meaning
00	—	—	000	add (lw, sw)
01	—	—	001	subtract (beq)
10	000	0 / 1	000 / 001	add, addi / sub
10	010	—	101	slt, slti
10	110	—	011	or, ori
10	111	—	010	and, andi

Listing 7.2. Main decoder and ALU decoder.

rv_blocks.sv

```
// Main decoder + ALU decoder
module controller (
    input  logic [6:0] op,
    input  logic [2:0] funct3,
    input  logic      funct7b5,
    output logic      regwrite, alusrc, memwrite, branch, jump,
    output logic [1:0] resultsrc, immsrc,
    output logic [2:0] alucontrol);
    logic [1:0] aluop;
    logic      rtypesub;
    always_comb begin
        // defaults
        {regwrite, immsrc, alusrc, memwrite, resultsrc, branch, aluop, jump} = '0;
        case (op)
            7'b0110011: begin regwrite = 1; aluop = 2'b10; end           // R-type
            7'b0010011: begin regwrite = 1; alusrc = 1; aluop = 2'b10; end // I-type ALU
            7'b0000011: begin regwrite = 1; alusrc = 1; resultsrc = 2'b01; end // lw
            7'b0100011: begin immsrc = 2'b01; alusrc = 1; memwrite = 1; end // sw
            7'b1100011: begin immsrc = 2'b10; branch = 1; aluop = 2'b01; end // beq
            7'b1101111: begin regwrite = 1; immsrc = 2'b11; resultsrc = 2'b10; jump = 1; end // jal
            default: ;
        endcase
    end
    assign rtypesub = funct7b5 & op[5]; // only R-type distinguishes add/sub
    always_comb
        case (aluop)
            2'b00: alucontrol = 3'b000;
            2'b01: alucontrol = 3'b001;
            default:
                case (funct3)
                    3'b000: alucontrol = rtypesub ? 3'b001 : 3'b000;
                    3'b010: alucontrol = 3'b101;
                    3'b110: alucontrol = 3'b011;
                    3'b111: alucontrol = 3'b010;
                    default: alucontrol = 3'b000;
                endcase
            endcase
        endcase
    endmodule
```

Listing 7.3. Register file: x0 reads as zero; the write edge is a parameter (falling for the pipeline, rising for the single-cycle and multicycle cores).

rv_blocks.sv

```
// Register file. With FALLING = 1 (the pipeline's choice) it is written on the falling
// clock edge, so a value written in the first half of a cycle can be read in the second
// half (removes one WB→ID hazard). The single-cycle and multicycle cores write on the
// rising edge (FALLING = 0), which gives their write-back paths a full clock cycle.
module regfile #(parameter bit FALLING = 1'b1) (
    input  logic      clk, we,
    input  logic [4:0] a1, a2, a3,
    input  logic [31:0] wd,
    output logic [31:0] rd1, rd2);
    logic [31:0] rf [31:0];
    initial for (int i = 0; i < 32; i++) rf[i] = 32'd0; // defined power-up state (simulation)
    if (FALLING) begin : g_fall
        always_ff @(negedge clk) if (we) rf[a3] ≤ wd;
    end else begin : g_rise
        always_ff @(posedge clk) if (we) rf[a3] ≤ wd;
    end
    assign rd1 = (a1 ≠ 5'd0) ? rf[a1] : 32'd0;
    assign rd2 = (a2 ≠ 5'd0) ? rf[a2] : 32'd0;
endmodule
```

7.3 Critical path and performance

The clock period of a single-cycle machine is set by its slowest instruction, and that is `lw`, the only instruction that uses every stage. Its path runs from the PC register through instruction memory, the register-file read (or, in parallel, the decoder, extender and ALUSrc multiplexer), the ALU, a data-memory read and the result multiplexer, and ends at the register-file setup:

$$T_c \geq t_{pcq,PC} + t_{mem} + \max[t_{RFread}, t_{dec} + t_{ext} + t_{mux}] + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}.$$

Performance follows the "iron law" of processor performance [6]:

$$\text{execution time} = N_{\text{instr}} \times \text{CPI} \times T_c.$$

Here CPI is the average number of clock cycles per instruction. A single-cycle machine has $\text{CPI} = 1$ by construction. It executed the test program's 60 instructions in exactly 60 cycles. Every instruction, including the trivial `addi`, pays the `lw` period. Pipelining attacks T_c instead of CPI. Section 8.11 measures T_c for this design: 105 NAND levels with a Kogge–Stone adder, set by the branch/jump PC update and, one level behind it, the write-back of `lw`.

CHAPTER 8

The Pipelined Processor

8.1 Throughput versus latency

Suppose the combinational logic of the single-cycle datapath has total delay t_{logic} . Dividing it into k stages separated by registers, each with sequencing overhead $t_{seq} = t_{pcq} + t_{setup}$, gives at best

$$T_c^{(k)} = \frac{t_{logic}}{k} + t_{seq}, \quad \text{speedup} \leq \frac{t_{logic} + t_{seq}}{t_{logic}/k + t_{seq}} \xrightarrow{k \rightarrow \infty} \frac{t_{logic} + t_{seq}}{t_{seq}}.$$

Three effects erode that ideal. The stages are never perfectly balanced, and the slowest one sets T_c . Register overhead does not shrink as k grows. And, most importantly for architecture, instructions depend on one another. When an instruction cannot proceed correctly in the next cycle — a *hazard* — the pipeline must either repair the data or waste cycles, and CPI rises above 1. Latency, the time for one instruction to pass through the pipeline, never improves; only throughput does.

8.2 Five stages

The datapath is cut at its natural boundaries: Fetch, Decode, Execute, Memory and Writeback (Figure 8.1). Pipeline registers F/D, D/E, E/M and M/W carry each instruction's data and its already-decoded control signals, so each stage acts on the instruction it currently holds. The names of these signals carry a stage suffix (pcF, rdE, aluresultM) — a convention borrowed from Harris and Harris that makes hazard logic readable. Listing 8.1 shows how the stages, registers and hazard unit are wired, and Figure 8.2 shows the fetch stage. Every pipeline register is an instance of one parameterized `pipereg` with an enable (for stalling) and a synchronous clear (for flushing). Clearing a register turns its instruction into a *bubble*: the all-zero control word writes nothing.

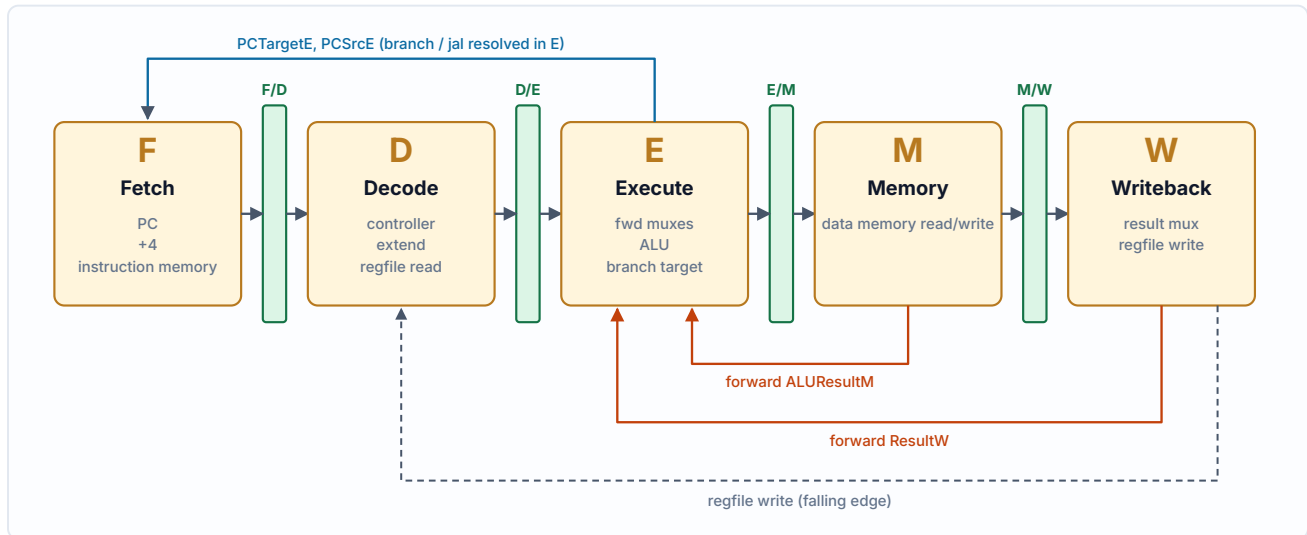


Figure 8.1. The five-stage pipeline. Green bars are pipeline registers. Red paths are the two forwarding sources into Execute; the blue path is the redirect of the PC when a branch or jump is taken in Execute; the dashed path is the write-back into the register file in Decode.

Listing 8.1. Top level of the pipelined core: stage modules, pipeline registers, register file and hazard unit.

rv_pipe.sv

```

module rv_pipe (
    input logic clk, rst,
    output logic [31:0] pcF,
    input logic [31:0] instrF,
    output logic memwriteM,
    output logic [31:0] aluresultM, writedataM,
    input logic [31:0] readdataM);

    logic stallF, stallD, flushD, flushE, pcsrcE, regwriteW;
    logic [1:0] fwdAE, fwdBE;
    logic [2:0] ctrlW;
    logic [3:0] ctrlM;
    logic [9:0] ctrlD, ctrlE;
    logic [4:0] rs1D, rs2D, rdD, rs1E, rs2E, rdE, rdM, rdW;
    logic [31:0] pcplus4F, instrD, pcD, pcplus4D, immextD, rd1D, rd2D,
    rd1E, rd2E, pcE, immextE, pcplus4E, aluresultE, writedataE, pctargetE,
    pcplus4M, aluresultW, readdataW, pcplus4W, resultW;

    stage_F sF (.clk, .rst, .stallF, .pcsrcE, .pctargetE, .pcF, .pcplus4F);
    reg_FD rFD (.clk, .rst, .en(~stallD), .clr(flushD), .instrF, .pcF, .pcplus4F,
    .instrD, .pcD, .pcplus4D);
    stage_D sD (.instrD, .ctrlD, .immextD, .rs1D, .rs2D, .rdD);
    regfile rf (.clk, .we(regwriteW), .a1(rs1D), .a2(rs2D), .a3(rdW),
    .wd(resultW), .rd1(rd1D), .rd2(rd2D));
    reg_DE rDE (.clk, .rst, .clr(flushE), .ctrlD, .rd1D, .rd2D, .pcD, .immextD, .pcplus4D,
    .rs1D, .rs2D, .rdD, .ctrlE, .rd1E, .rd2E, .pcE, .immextE, .pcplus4E,
    .rs1E, .rs2E, .rdE);
    stage_E sE (.ctrlE, .rd1E, .rd2E, .pcE, .immextE, .resultW, .aluresultM, .fwdAE, .fwdBE,
    .aluresultE, .writedataE, .pctargetE, .pcsrcE);
    reg_EM rEM (.clk, .rst, .ctrlE(ctrlE[9:6]), .aluresultE, .writedataE, .pcplus4E, .rdE,
    .ctrlM, .aluresultM, .writedataM, .pcplus4M, .rdM);
    assign memwriteM = ctrlM[0];
    reg_MW rMW (.clk, .rst, .ctrlM(ctrlM[3:1]), .aluresultM, .readdataM, .pcplus4M, .rdM,
    .ctrlW, .aluresultW, .readdataW, .pcplus4W, .rdW);
    stage_W sW (.ctrlW, .aluresultW, .readdataW, .pcplus4W, .resultW, .regwriteW);
    hazard hu (.rs1D, .rs2D, .rs1E, .rs2E, .rdE, .rdM, .rdW,
    .pcsrcE, .resultsrcE0(ctrlE[7]), .regwriteM(ctrlM[3]), .regwriteW,
    .fwdAE, .fwdBE, .stallF, .stallD, .flushD, .flushE);
endmodule

```

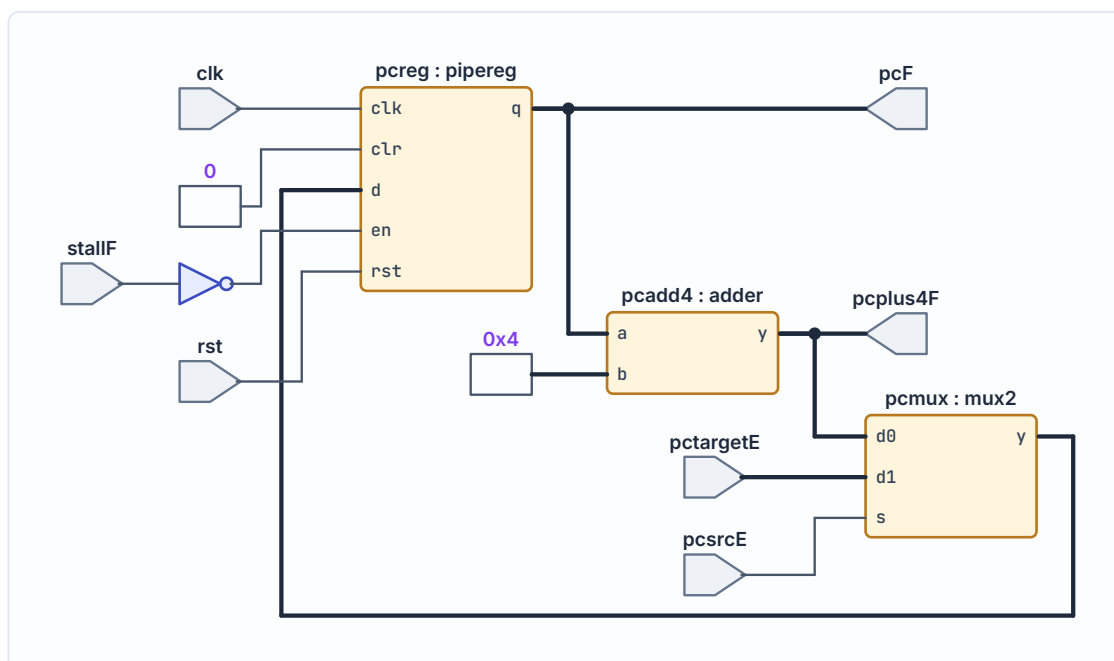


Figure 8.2. Fetch stage: next-PC multiplexer, PC register with stall enable, $PC + 4$ adder. YOSYS → NETLISTSVG

8.3 Watching the pipeline run

Figure 8.3 is drawn directly from the simulation trace. Each column is a clock cycle, each row a stage, and each cell names the instruction occupying that stage during the cycle. After four cycles of filling, the pipeline holds five instructions at once. In cycle 8, the backward `jal` reaches Execute, where it is resolved as taken. The two instructions fetched behind it — `sw` in Decode and `lw` in Fetch — were fetched on the assumption that the jump would not be taken, so they are squashed. In cycle 9, `add` at the loop target enters Fetch, and bubbles move down the pipeline in the vacated slots.



Figure 8.3. Simulated pipeline occupancy, cycles 0–9. The jump in Execute at cycle 8 squashes the two instructions behind it.

8.4 Data hazards and forwarding

An instruction in Execute may need a register value that an older instruction, still in Memory or Writeback, has computed but not yet written. In cycle 7 of the trace, `beq x2, x3` is in Execute while `addi x2, x2, 1`, which updates `x2`, is in Memory. The value in the D/E register is stale. The fix is to *forward* the newer value into the ALU inputs. The selection for the first operand is

$$\text{ForwardAE} = \begin{cases} 10 & \text{if } rs1_E = rd_M \wedge \text{RegWrite}_M \wedge rs1_E \neq 0 \quad (\text{from Memory}) \\ 01 & \text{else if } rs1_E = rd_W \wedge \text{RegWrite}_W \wedge rs1_E \neq 0 \quad (\text{from Writeback}) \\ 00 & \text{otherwise (register-file value)} \end{cases}$$

ForwardBE is identical with $rs2_E$. The priority matters: if both later stages write the same register, the Memory-stage instruction is the more recent, so its value wins. The $\neq 0$ guard keeps `x0` reading as zero even when an instruction "writes" it, as `jal x0, loop` does. Figure 8.4 shows the two forwarding multiplexers ahead of the ALU. The trace shows both sources at work: in cycle 5, `add x1, x1, x2` takes `x2` from Writeback; in cycle 7, `beq` takes it from Memory.

A third case needs no forwarding at all. When the reading instruction is in Decode and the writer is in Writeback, the value arrives through the register file itself, because the file is written on the falling edge and read in the second half of the same cycle.

HARMLESS FALSE POSITIVES

The hazard unit compares $rs2_E$ even for I-type instructions, where bits 24–20 belong to the immediate. The trace shows such "forwards" in cycles 3 and 6. They are harmless: the forwarded value lands on `WriteDataE`, which an I-type instruction never uses, because its ALU operand comes from the immediate. The processor was accepted on its architectural results, not on the absence of such events.

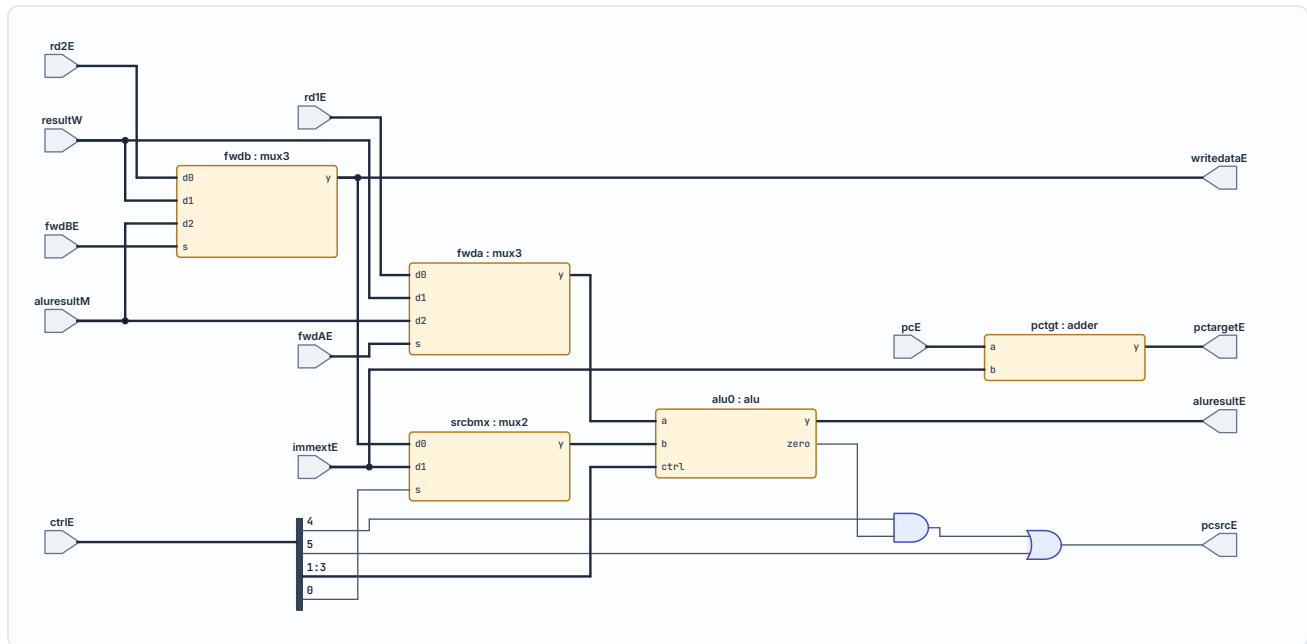


Figure 8.4. Execute stage: the two forwarding multiplexers (*fwdA*, *fwdB*), the ALUSrc multiplexer, the ALU and the branch-target adder. The control bundle *ctrlE* is split into branch (bit 4), jump (5), ALU control (3:1) and ALUSrc (0). YOSYS → NETLISTSVG

8.5 The load-use stall

Forwarding cannot help `lw`. Its result exists only at the end of the Memory stage, one cycle too late for an instruction in Execute immediately behind it. The only remedy is to delay that instruction by one cycle. When an `lw` in Execute writes a register that the instruction in Decode reads, the hazard unit asserts

$$lwStall = ResultSrc_{E,0} \cdot [(rs1_D = rd_E) + (rs2_D = rd_E)],$$

with $StallF = StallD = lwStall$ and $FlushE = lwStall$. Fetch and Decode hold their contents for one cycle, and a bubble enters Execute. $ResultSrc_{E,0}$ is 1 only for `lw`, as Table 7.1 shows. Figure 8.5 shows the sequence: in cycle 65, `lw x4` is in Execute and `add x5, x4, x4` is in Decode; the stall holds `add` and `sub`. In cycle 67, `add` reaches Execute with `lw` in Writeback, and both of its operands are forwarded from Writeback.

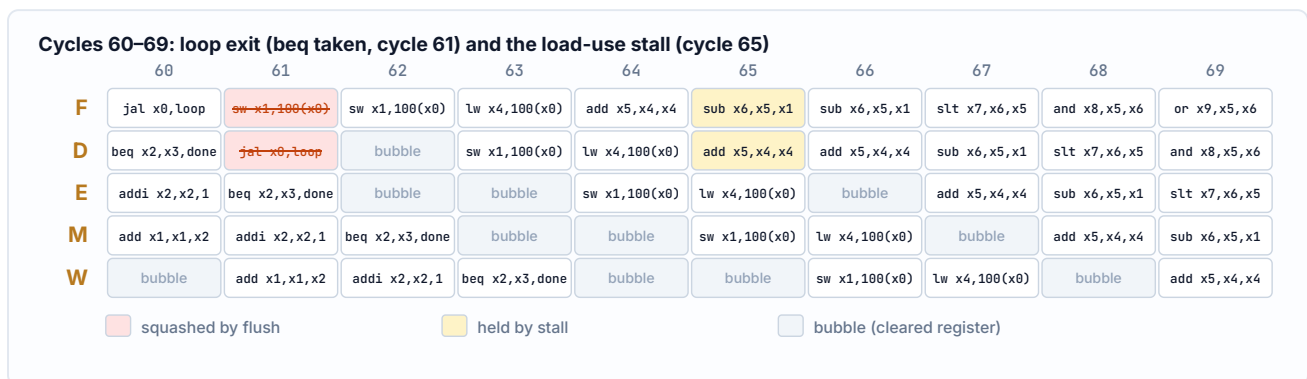


Figure 8.5. Simulated pipeline occupancy, cycles 60–69: the loop-exit branch is taken in cycle 61, and the load-use stall occurs in cycle 65.

8.6 Control hazards

The pipeline fetches sequentially without waiting for a branch to be decided. In effect it always predicts "not taken". Branches and jumps are resolved in Execute. When $PCSrc_E = Branch_E \cdot Zero_E + Jump_E$ is true, the PC is redirected to the target, and the two instructions fetched in the meantime are discarded:

$$\text{FlushD} = \text{PCSrc}_E, \quad \text{FlushE} = \text{lwStall} + \text{PCSrc}_E.$$

Every taken control transfer therefore costs two cycles. The trace also shows a detail of this policy. When `beq` exits the loop in cycle 61, its target is `sw` at address `0x1c` — the very instruction already in Fetch, two slots behind it. That copy is still squashed and fetched again, because the hardware never compares the target with what it has already fetched. The same simple policy also explains why the instruction after `jal` `x11` never executes, which the test program checks.

8.7 The hazard unit

The hazard unit (Listing 8.2, Figure 8.6) is pure combinational logic over register indices and a few control bits. The upper half of the figure is the stall and flush logic; the lower half is the two priority multiplexers of the forwarding equations, with the " $\neq 0$ " guards drawn as reduction-OR gates.

Listing 8.2. Hazard unit: forwarding, load-use stall and flush.

`rv_pipe.sv`

```
module hazard (
  input logic [4:0] rs1D, rs2D, rs1E, rs2E, rdE, rdM, rdW,
  input logic      pcsrcE, resultsrcE0, regwriteM, regwriteW,
  output logic [1:0] fwdAE, fwdBE,
  output logic      stallF, stallD, flushD, flushE);
  logic lwstall;
  // Forwarding: prefer the most recent writer (M over W); never forward x0.
  always_comb begin
    fwdAE = 2'b00;
    if (rs1E != 5'd0) begin
      if (rs1E == rdM && regwriteM) fwdAE = 2'b10;
      else if (rs1E == rdW && regwriteW) fwdAE = 2'b01;
    end
    fwdBE = 2'b00;
    if (rs2E != 5'd0) begin
      if (rs2E == rdM && regwriteM) fwdBE = 2'b10;
      else if (rs2E == rdW && regwriteW) fwdBE = 2'b01;
    end
  end
  // Load-use hazard: data from lw isn't available until the end of M → stall one cycle.
  assign lwstall = resultsrcE0 & ((rs1D == rdE) | (rs2D == rdE));
  assign stallF  = lwstall;
  assign stallD  = lwstall;
  // Control hazard: branch/jump taken in E → squash the two younger instructions.
  assign flushD  = pcsrcE;
  assign flushE  = lwstall | pcsrcE;
endmodule
```

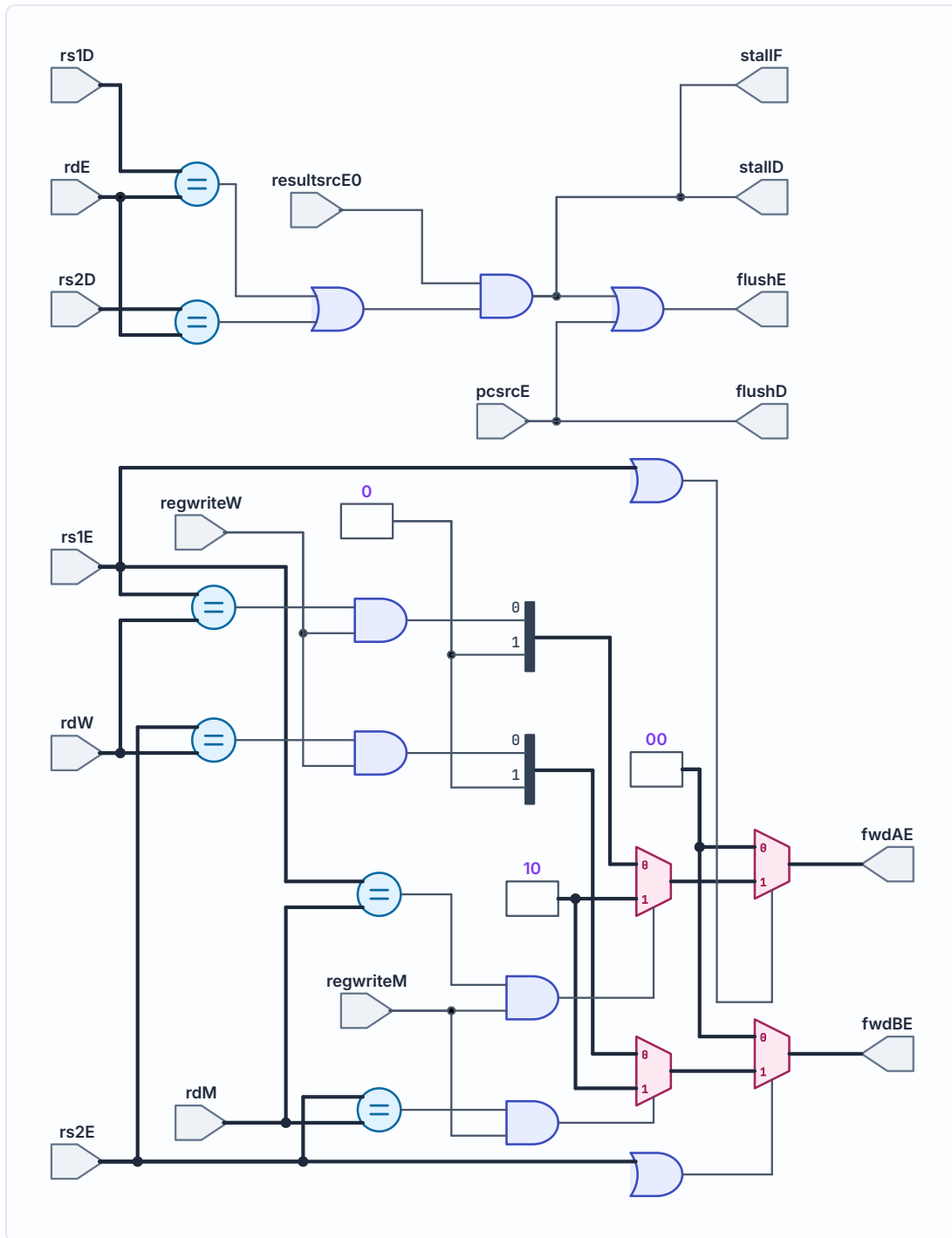


Figure 8.6. Synthesized hazard unit.

YOSYS → NETLISTSVG

8.8 Measured behaviour

Both processors produced the reference architectural state (Chapter 10). The trace allows an exact accounting of where the pipeline's cycles went (Table 8.1). Numbering cycles from 0, the first instruction completes Writeback in cycle 4 and the halt instruction in cycle 86, so the run takes 87 cycles against the single-cycle machine's 60. An ideal pipeline would advance the remaining 59 instructions in 59 cycles. The difference, 23 cycles, is exactly one load-use stall plus two squashed slots for each of the 11 taken control transfers: nine backward jumps, the loop-exit branch and the forward call. The loop body in this program is branch-dense, so its steady-state CPI, $82/59 \approx 1.39$, is higher than typical workloads would produce.

Over the whole run, the pipeline is faster whenever its clock period is below $60/87 \approx 69\%$ of the single-cycle period. Five perfectly balanced stages could approach one fifth of that period, less the register overhead of Section 8.1.

Table 8.1. Cycle accounting from the simulation trace.

Quantity	Value	Source
Instructions executed (including first halt)	60	reference simulator
Cycle (0-based) in which first instruction completes W	4	trace
Cycle (0-based) in which halt completes W	86	trace
Load-use stalls	1	trace (<code>stallD</code>)
Taken branches/jumps before halt	11	trace (<code>flushD</code>)
Accounting: $59 + 1 + 2 \times 11$	$82 = 86 - 4$	exact
Steady-state CPI	$82 / 59 \approx 1.39$	derived
Total cycles, pipeline vs. single-cycle	87 vs. 60	traces

8.9 Where real processors go next

Each improvement below targets one of the costs just measured.

- **Branch prediction.** A dynamic predictor, with a branch target buffer supplying the target at fetch time, removes most of the two-cycle penalty (Section 8.10 builds one). Modern predictors are right for the large majority of branches [6].
- **Earlier branch resolution.** Comparing for equality in Decode cuts the penalty to one cycle, at the cost of new forwarding paths into Decode.
- **Deeper pipelines.** These shorten T_c further but raise the misprediction penalty, so they are only worthwhile when prediction is good.
- **Superscalar and out-of-order execution.** These raise instructions per cycle above one by running several instructions at once and scheduling them dynamically. Their bookkeeping — register renaming and a reorder buffer — generalizes the forwarding logic of this chapter.

8.10 Adding dynamic branch prediction

Section 8.8 attributed 22 of the 23 lost cycles to taken control transfers, all flushed because the pipeline assumed "not taken". A dynamic predictor removes most of that cost by guessing in Fetch, before the instruction has even been decoded [6, 30, 32]. The predictor added here (Listing 8.3) is a 16-entry *branch-target buffer* (BTB) indexed by `PC[5:2]`. Each entry holds:

- a valid bit and a tag (the remaining PC bits);
- the branch target;
- a jump flag;
- a 2-bit saturating counter: values 00 and 01 predict not taken, 10 and 11 predict taken, and each outcome moves the counter one step.

When the fetched PC hits an entry that predicts taken (always for jumps), the next PC is the stored target. The branch is still resolved in Execute, exactly as before. Only a *misprediction* — the wrong direction, or the right direction with the wrong target — flushes the two younger instructions and redirects the PC to the correct address: the target if taken, $PC + 4$ if not.

The hazard unit is unchanged: its flush input is simply driven by the misprediction signal instead of every taken branch.

Listing 8.3. The branch-target buffer with 2-bit counters.

rv_pipe_bp.sv

```

module bpred (
    input logic clk, rst,
    input logic [31:0] pcF,
    output logic predTakenF,
    output logic [31:0] predTargetF,
    input logic updE, // a branch or jump is in Execute
    input logic [31:0] pcE, targetE,
    input logic takenE, isJumpE);
    logic valid [16];
    logic [25:0] tag [16];
    logic [31:0] tgt [16];
    logic [1:0] ctr [16]; // 00/01 predict not taken, 10/11 predict taken
    logic jmp [16];
    logic [3:0] iF, iE;
    logic hitF, hitE;
    assign iF = pcF[5:2];
    assign iE = pcE[5:2];
    assign hitF = valid[iF] && (tag[iF] == pcF[31:6]);
    assign hitE = valid[iE] && (tag[iE] == pcE[31:6]);
    assign predTakenF = hitF && (jmp[iF] || ctr[iF][1]);
    assign predTargetF = tgt[iF];
    always_ff @(posedge clk) begin
        if (rst) begin
            for (int k = 0; k < 16; k++) valid[k] ≤ 1'b0;
        end else if (updE) begin
            valid[iE] ≤ 1'b1;
            tag[iE] ≤ pcE[31:6];
            tgt[iE] ≤ targetE;
            jmp[iE] ≤ isJumpE;
            if (!hitE) ctr[iE] ≤ takenE ? 2'b10 : 2'b01; // new entry: weak
            else if (takenE) ctr[iE] ≤ (ctr[iE] == 2'b11) ? 2'b11 : ctr[iE] + 2'b01;
            else ctr[iE] ≤ (ctr[iE] == 2'b00) ? 2'b00 : ctr[iE] - 2'b01;
        end
    end
endmodule

```

On the test program, the predicting pipeline mispredicts only three control transfers before the halt: the first backward `jal` (the BTB is still empty), the loop exit and the forward call. The halt therefore completes in cycle 70 instead of 86, and the lost cycles fall from 23 to $3 \times 2 + 1 = 7$ (Table 8.2). Figure 8.7 shows the effect: the first backward jump is flushed, and nine cycles later the same jump is predicted and the loop continues without a bubble. On the 2 000 random programs of Section 10.1, prediction cut flush events from 40 359 to 18 019, a 55 % reduction.

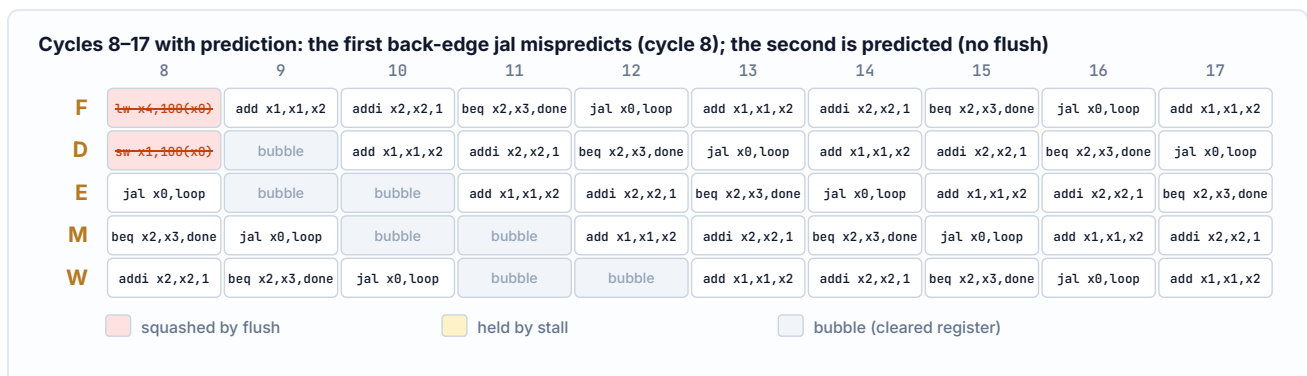


Figure 8.7. Simulated occupancy of the predicting pipeline, cycles 8–17. The first back-edge `jal` is mispredicted (flush in cycle 8); when it is fetched again in cycle 12, the BTB supplies the target and the loop continues without a bubble.

Table 8.2. Effect of prediction on the test program (0-based cycle numbers; both pipelines verified against the reference simulator).

	No prediction	2-bit BTB
Flushing control transfers before the halt	11	3
Load-use stalls	1	1
Cycle in which the halt completes W	86	70
Total cycles	87	71
Steady-state CPI (cycles after the first completion ÷ 59)	1.39	1.12

A PREDICTOR CANNOT BREAK CORRECTNESS — ONLY SPEED

Every prediction is checked in Execute before the instruction can change architectural state, so even a terrible predictor produces correct results; it only wastes cycles. Mutation testing made this concrete (Section 10.2). Deleting the target comparison from the misprediction check changed nothing at all, because each BTB tag stores all the remaining PC bits: an entry can only match its own branch, whose target never changes. By contrast, a recovery path that always jumps to the target *was* caught, because it corrupts execution whenever a branch predicted taken actually falls through.

8.11 Measured clock periods: does pipelining pay?

Sections 8.8 and 8.10 counted clock cycles. The iron law also needs the clock period, and this section measures it for all four processors with one consistent model. The single-cycle and multicycle cores appear in Chapter 7 and Appendix D.6.

Method. Each processor is synthesized by Yosys to simple gates, flattened, and timed as a longest-path problem.

- **Gate costs.** Each gate costs its depth in the NAND realizations of Table 2.1: NOT 1, AND and OR 2, XOR 3, and a 2:1 multiplexer 2 from data or 3 from select.
- **Flip-flops.** The flip-flop parameters were measured by simulating the NAND master-slave flip-flop of Section 5.3 with unit gate delays: clock-to-Q is 3 levels, setup 1 and hold 0.
- **Memories.** The instruction and data memories are modelled by the synthesized 64-word × 32-bit random-access memory (RAM): a read takes 13 levels from address to data, and a write needs 12 levels from its inputs to the storage flip-flops.
- **Two clock edges.** The analysis tracks rising- and falling-edge flip-flops separately. A path from one edge to the other gets only half a cycle, which matters for the pipeline's falling-edge register file.

Two design corrections came out of building this analysis:

- The single-cycle and multicycle cores had inherited the pipeline's falling-edge register file. That forced their write-back paths into half a cycle, and in a first, ABC-based analysis it doubled the single-cycle period (270 levels against 135). The register file now takes its write edge as a parameter: the pipeline keeps the falling edge, the other two write on the rising edge, as textbook designs do.
- The ALU's expression `a + b + ctrl[0]` synthesized as two chained adders. It is now written as one adder whose extra low-order bit supplies the carry-in (Listing 4.2), which cut the Brent-Kung ALU from 72 to 60 levels.

After both changes, the full random co-simulation (Section 10.1), the mutation suite and the formal ALU proof were re-run with identical results.

Why not ABC? Yosys normally hands the logic to the ABC optimizer, which restructures it for area. Table 8.3 shows the effect on a 32-bit adder: with ABC, a Brent–Kung carry network ends up 67 levels deep and a Kogge–Stone one 50. Mapped structurally, the same networks are 38 and 24 levels, close to the 16 levels of the hand-built adder of Section 4.6. Delay-driven mapping would need a timing library that this flow does not have. The processor figures below therefore use structural mapping, so that they reflect the architecture rather than the optimizer's area heuristics.

Table 8.3. Depth (NAND levels) and size of 32-bit arithmetic under two mapping flows.

Block	Carry network	ABC (area-oriented)	Structural
adder	Brent–Kung	67 levels, 387 gates	38 levels, 220 gates
adder	Kogge–Stone	50 levels, 469 gates	24 levels, 436 gates
ALU	Brent–Kung	77 levels	60 levels
ALU	Kogge–Stone	76 levels	56 levels

Table 8.4. Measured clock period (critical path + setup, NAND levels) and run time of the 60-instruction test program. Gates/flip-flops are for the Brent–Kung build; run time uses the Kogge–Stone period.

Processor	Gates / flip-flops	Period (BK)	Period (KS)	Critical path	Cycles	Run time
single-cycle	4 358 / 1 056	112	105	PC update (branch/jump target) then write-back of <code>lw</code>	60	6 300
multicycle	4 322 / 1 274	97	92	ALU → Result → PC (Fetch computes PC + 4)	230	21 160
pipeline	5 108 / 1 531	101	97	Execute: forwarding → ALU → zero → branch decision → flush of D/E	87	8 439
pipeline + prediction	7 901 / 2 593	108	104	as the pipeline plus the misprediction comparison	71	7 384

Table 8.4 contains the central result of this chapter. With a memory as fast as the synthesized RAM, the pipeline's clock period is only 8 % shorter than the single-cycle machine's (97 against 105 levels). Its extra cycles therefore make it *slower* overall, and even the predicting pipeline trails the single-cycle machine. The reason is imbalance, not overhead:

- **Execute is overloaded.** It contains the forwarding multiplexers, the whole ALU, the 32-bit zero detector and the branch decision, and that decision then fans out to clear the 185-bit D/E register. It is nearly the entire single-cycle critical path.
- **The other stages are short.** Fetch, Decode, Memory and Writeback finish in a fraction of the period.
- **Half-cycle paths are not a problem.** The falling-edge register-file paths need only 32 levels — 16 levels in each half cycle.
- **Prediction costs 7 levels.** The misprediction comparison sits on that same critical path.

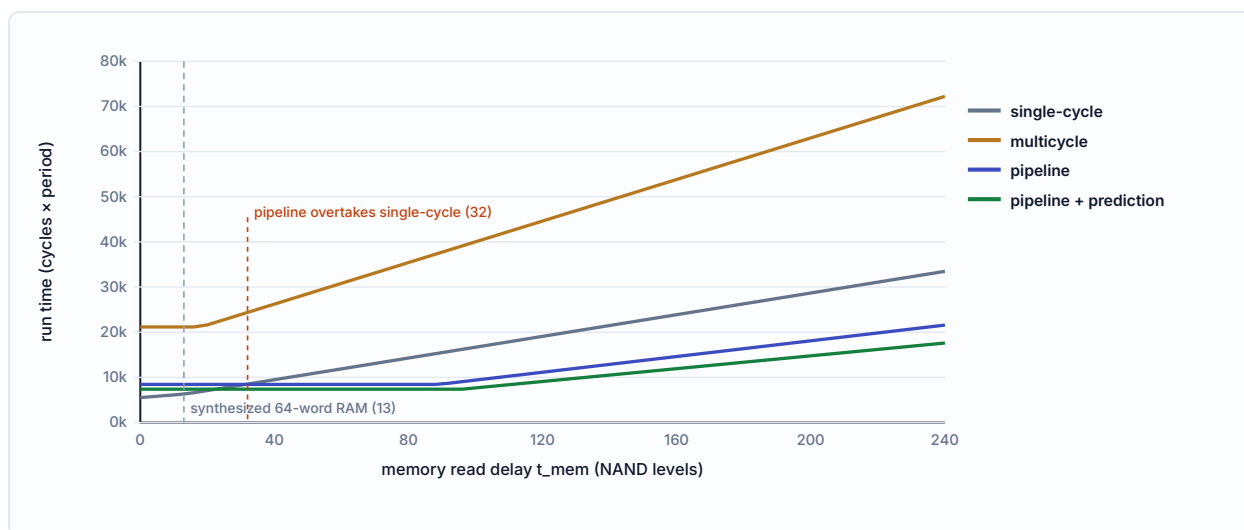


Figure 8.8. Run time of the test program versus memory access delay (Kogge–Stone builds; everything else measured). Only paths through a memory lengthen, so the single-cycle and multicycle machines, which read memory twice per instruction path, degrade fastest.

Real memories are much slower relative to logic than a flip-flop RAM, and Figure 8.8 shows what that does. Every processor's period depends on the memory delay, and the pipeline's period is the least sensitive, because each memory access sits alone in its own stage. The plain pipeline overtakes the single-cycle machine once a memory read costs 32 NAND levels, and the predicting pipeline at 24. The multicycle machine never wins on this program: its period falls far more slowly than its cycle count rises.

Two remedies follow from the measurement, and both are standard:

- **Rebalance Execute.** Resolve branches in Decode with a dedicated equality comparator, which also halves the misprediction penalty, and keep the flush off the ALU path.
- **Split the long stage.** Deeper pipelines do this once the per-register overhead (here 4 levels) is small compared with the logic being divided.

LIMITS OF THE MODEL

Unit gate delays ignore fan-out and wire load. The flush signal and the register-file read multiplexers drive hundreds of inputs, so real silicon would penalize the pipeline and the register file more than shown here. The memories are flip-flop arrays, not SRAM. The multicycle analysis is structural: it also reports paths that the controller never exercises. The longest of those (ALU → Result → memory, 87 levels) is shorter than the true critical path, so it does not change the result.

CHAPTER 9

Floating Point and an FPU

Physical quantities span many orders of magnitude, and fixed-point numbers (Section 3.8) cannot cover them in one format. Floating-point numbers store a significand and an exponent separately, like scientific notation in base 2. This chapter covers:

- the IEEE 754 standard that every modern processor follows;
- how floating-point addition and multiplication work and why they are hard to get exactly right;
- a synthesizable, verified floating-point unit (FPU) for 32-bit IEEE numbers;
- two ways to give the processors of this primer floating-point capability: as a memory-mapped coprocessor, built and verified here, and as the RISC-V F extension, whose integration into the pipeline is explained.

9.1 The IEEE 754 formats

IEEE 754 [33] defines the formats, results and rounding of floating-point arithmetic. Its 32-bit *binary32* format (C's `float`) packs three fields into a word (Figure 9.1): a sign s , an 8-bit biased exponent e and a 23-bit fraction f .

- **Normal numbers.** For $1 \leq e \leq 254$, the value is $(-1)^s \times 1.f \times 2^{e-127}$. The leading 1 is implicit, and the *bias* 127 lets the exponent field be compared as an unsigned number.
- **Zeros and subnormals.** With $e = 0$, $f = 0$ encodes ± 0 , and any other f encodes a *subnormal* number $(-1)^s \times 0.f \times 2^{-126}$. Subnormals fill the gap between the smallest normal number and zero evenly (*gradual underflow*).
- **Infinities and NaN.** With $e = 255$, $f = 0$ encodes $\pm\infty$, and any other f encodes NaN ("not a number"), the result of invalid operations.

The 64-bit *binary64* format (`double`) has the same structure with an 11-bit exponent (bias 1023) and a 52-bit fraction.

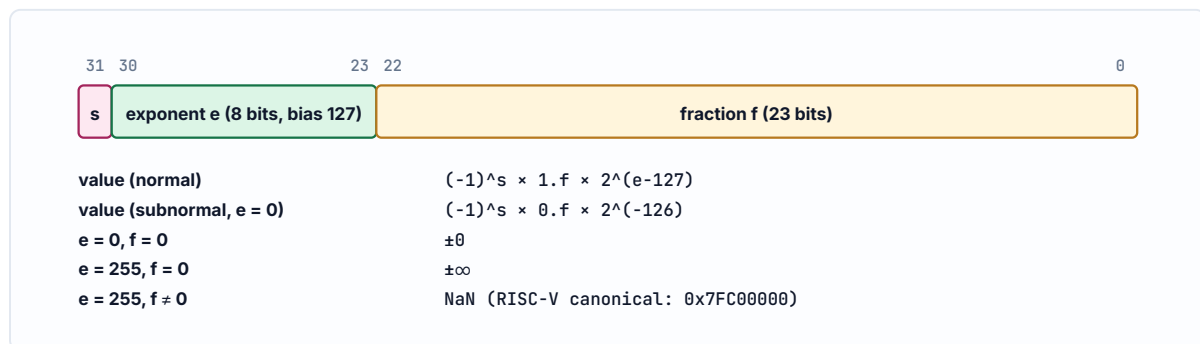


Figure 9.1. IEEE 754 binary32 layout and the meaning of its field combinations.

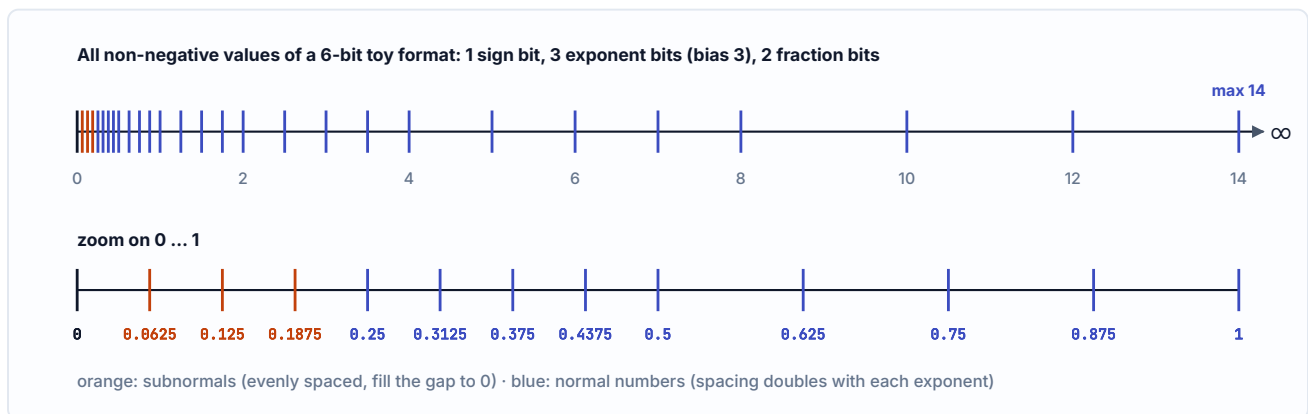
Table 9.1 shows real encodings, all checked with NumPy. The value 0.1 has no finite binary expansion, so the stored number is the nearest representable one, slightly larger than 0.1. And $16\,777\,217 = 2^{24} + 1$ cannot be represented at all: above 2^{24} the spacing of binary32 numbers exceeds 1. Binary32 holds about 7 significant decimal digits, binary64 about 16.

Table 9.1. Selected binary32 encodings (s, e and f are the field values; the last column is the exact stored value).

number	bits	s	e	f	stored value
1.0	3F800000	0	127	000000	1
-2.5	C0200000	1	128	200000	-2.5
0.1	3DCCCCCD	0	123	4CCCCD	0.100000001490116119384765625
1/3	3EAAAAAB	0	125	2AAAAAB	0.3333333432674407958984375
$16\,777\,216 = 2^{24}$	4B800000	0	151	000000	16 777 216
16 777 217	4B800000	0	151	000000	16 777 216 (nearest representable)
largest finite	7F7FFFFF	0	254	7FFFFFF	$\approx 3.402\,823\,5 \times 10^{38}$
smallest normal	00800000	0	1	000000	$2^{-126} \approx 1.175\,494\,4 \times 10^{-38}$
smallest subnormal	00000001	0	0	000001	$2^{-149} \approx 1.401\,298\,5 \times 10^{-45}$

9.2 Precision, rounding and special values

Floating-point numbers are not evenly spaced. Figure 9.2 shows every non-negative value of a toy 6-bit format with the same structure as binary32 (1 sign bit, 3 exponent bits with bias 3, 2 fraction bits). The spacing doubles with every exponent step, and the subnormals continue the smallest spacing down to zero. The gap between a number and its successor is called one *ulp* (unit in the last place); for binary32 it is 2^{-23} at 1.0.

**Figure 9.2.** Every non-negative value of a 6-bit toy floating-point format, with a zoom on the interval from 0 to 1.

Since most real results fall between two representable numbers, IEEE 754 requires *correct rounding*: the result must be exactly what rounding the true result would give. The default mode, which this chapter implements, is round to nearest, ties to even. RISC-V also offers round toward zero, down, up, and nearest with ties away from zero, selected by the 3-bit `frm` field of its floating-point control and status register `fcsr`.

Two consequences surprise programmers. First, decimal fractions are approximations: $0.1 + 0.2 \neq 0.3$ in double precision, though the rounding happens to agree in binary32 (both give `0x3E99999A`). Second, large integers lose precision: in binary32, $2^{24} + 1 = 2^{24}$.

Operations with no sensible numerical answer produce special values rather than stopping the program:

- **Invalid operations** ($\infty - \infty$, $0 \times \infty$, $0/0$, $\sqrt{-1}$) give NaN. RISC-V always returns the *canonical* NaN, `0x7FC00000`, rather than propagating an input NaN's payload [13].
- **Overflow** rounds to $\pm\infty$, and a finite number divided by zero gives $\pm\infty$.
- **Signed zero** is kept: $(-0) + (-0) = -0$, but $x - x = +0$.

- **Comparisons** treat $+0 = -0$ as equal and report every comparison with NaN as false, even $\text{NaN} = \text{NaN}$.

Each such event also sets a sticky flag in `fcsr`: NV (invalid), DZ (divide by zero), OF (overflow), UF (underflow) or NX (inexact). RISC-V never traps on them; software reads the flags. Goldberg's survey [34] remains the classic explanation of these rules.

9.3 The algorithms

Multiplication is the simpler operation.

1. The result sign is $s_a \oplus s_b$ and the exponents add.
2. The significands are multiplied exactly, $24 \times 24 \rightarrow 48$ bits (Appendix A).
3. The product is *normalized* by shifting its leading 1 to the top.
4. The product is *rounded* to 24 significant bits.
5. If the exponent has fallen below the normal range, the significand is first shifted right into the subnormal range.

Addition takes more steps.

1. Order the operands by magnitude.
2. *Align* the smaller one by shifting it right by the exponent difference.
3. Add or subtract the significands, depending on the signs.
4. Normalize, which after a subtraction can require a large left shift, "massive cancellation".
5. Round, and renormalize if rounding carried out of the top bit.

Correct rounding needs to know three things about the bits below the result: the first discarded bit (*guard*), the next (*round*), and whether *any* further bit is 1 (*sticky*). Round-to-nearest-even then increments exactly when

$$\text{inc} = g \cdot (r + st + \text{lsb}).$$

Here the round and sticky bits are merged into one sticky term. The sticky bit must include every bit shifted out during alignment, or rounding decisions near a tie go wrong; one of the mutations in Section 9.4 confirms this.

9.4 A synthesizable FPU

`hdl/fpu.sv` implements binary32 addition, subtraction, multiplication and the three comparisons, as combinational logic. Listings 9.1–9.2 show the multiplier and adder. Two details are specific to this design:

- **Uniform internal form.** Operands are unpacked into a 24-bit significand with an explicit leading bit and a biased exponent in which subnormals use 1. Normal and subnormal inputs then follow the same arithmetic.
- **Structure.** The adder/subtractor, multiplier and comparator work in parallel, and the operation code selects the result (Figure 9.4).
- **Logarithmic normalization.** Leading zeros are counted by `lzc64` (Figure 9.3), which tests and removes 32, 16, 8, 4, 2 and 1 leading zeros in turn. A first version counted with a `for` loop, which synthesizes to a linear priority chain.

Listing 9.1. Floating-point multiplier (binary32, round to nearest, ties to even).

`fpu.sv`

```
module fp_mul32 (input logic [31:0] a, b, output logic [31:0] y);
  localparam logic [31:0] QNaN = 32'h7FC0_0000; // RISC-V canonical NaN
  logic      sa, sb, sy;
  logic [7:0] ea, eb;
```

```

logic [22:0] fa, fb;
logic      nana, nanb, infa, infb, za, zb;
logic [23:0] ma, mb;
logic [9:0]  Ea, Eb;
logic [47:0] P, Pn, Ps;
logic [5:0]  lz;
logic signed [11:0] Er;
logic [11:0] sh;
logic      stk, g, st, inc;
logic [24:0] rnd;
logic [11:0] Ef;
logic [31:0] fin;
assign {sa, ea, fa} = a;
assign {sb, eb, fb} = b;
assign sy  = sa ^ sb;
assign nana = (ea == 8'hFF) && (fa != 0);
assign nanb = (eb == 8'hFF) && (fb != 0);
assign infa = (ea == 8'hFF) && (fa == 0);
assign infb = (eb == 8'hFF) && (fb == 0);
assign za  = (ea == 8'h00) && (fa == 0);
assign zb  = (eb == 8'h00) && (fb == 0);
assign ma  = {ea != 0, fa};
assign mb  = {eb != 0, fb};
assign Ea  = (ea == 0) ? 10'd1 : {2'b00, ea};
assign Eb  = (eb == 0) ? 10'd1 : {2'b00, eb};
assign P   = ma * mb;                                // exact 48-bit product
lzc64 clz (.x({P, 16'hFFFF}), .lz(lz));              // padding bounds the count at 48
assign Pn = P << lz;                                // leading one at bit 47
assign Er = $signed({2'b00, Ea}) + $signed({2'b00, Eb}) - 12'sd126 - $signed({6'b0, lz});
always_comb begin                                    // gradual underflow
    if (Er < 12'sd1) begin
        sh = 12'(12'sd1 - Er);
        if (sh ≥ 12'd48) begin Ps = '0; stk = |Pn; end
        else begin Ps = Pn >> sh; stk = |(Pn & ((48'd1 << sh) - 48'd1)); end
    end else begin
        sh = '0; Ps = Pn; stk = 1'b0;
    end
end
assign g  = Ps[23];                                // guard bit
assign st = (|Ps[22:0]) | stk;                      // sticky bit
assign inc = g & (st | Ps[24]);                     // round to nearest, ties to even
assign rnd = {1'b0, Ps[47:24]} + {24'd0, inc};
always_comb begin
    if (Er ≥ 12'sd1) begin                            // normal result
        Ef = 12'(Er) + {11'd0, rnd[24]};
        fin = (Ef ≥ 12'd255) ? {sy, 8'hFF, 23'd0}    // overflow → infinity
                : {sy, Ef[7:0], rnd[24] ? rnd[23:1] : rnd[22:0]};
    end else begin                                    // subnormal (or rounds up to normal)
        Ef = {11'd0, rnd[23]};
        fin = {sy, Ef[7:0], rnd[22:0]};
    end
end
always_comb
    if (nana || nanb || (infa && zb) || (infb && za)) y = QNAN;
    else if (infa || infb) y = {sy, 8'hFF, 23'd0};
    else if (za || zb) y = {sy, 31'd0};
    else y = fin;
endmodule

```

Listing 9.2. Floating-point adder/subtractor.

fpu.sv

```

module fp_add32 (input logic [31:0] a, b, input logic sub, output logic [31:0] y);
    localparam logic [31:0] QNAN = 32'h7FC0_0000;
    logic      sa, sb, sB, sS, eff_sub;
    logic [7:0] ea, eb;
    logic [22:0] fa, fb;
    logic      nana, nanb, infa, infb, za, zb, a_big;
    logic [23:0] ma, mb, mB, mS;
    logic [9:0]  Ea, Eb, EB, ES, d;
    logic [26:0] bg, sm, t;
    logic [27:0] S, Sn;

```

```

logic [4:0]  lz;
logic [9:0]  shl, E;
logic       g, st, inc;
logic [24:0] rnd;
logic [10:0] Ef;
logic [31:0] fin;
assign {sa, ea, fa} = a;
assign eb  = b[30:23];
assign fb  = b[22:0];
assign sb  = b[31] ^ sub;                                // a - b = a + (-b)
assign nana = (ea == 8'hFF) && (fa != 0);
assign nanb = (eb == 8'hFF) && (fb != 0);
assign infa = (ea == 8'hFF) && (fa == 0);
assign infb = (eb == 8'hFF) && (fb == 0);
assign za   = (ea == 8'h00) && (fa == 0);
assign zb   = (eb == 8'h00) && (fb == 0);
assign ma   = {ea != 0, fa};
assign mb   = {eb != 0, fb};
assign Ea   = (ea == 0) ? 10'd1 : {2'b00, ea};
assign Eb   = (eb == 0) ? 10'd1 : {2'b00, eb};
// order the operands by magnitude
assign a_big = {Ea, ma} >= {Eb, mb};
assign {sB, EB, mB} = a_big ? {sa, Ea, ma} : {sb, Eb, mb};
assign {sS, ES, mS} = a_big ? {sb, Eb, mb} : {sa, Ea, ma};
assign d  = EB - ES;
assign bg = {mB, 3'b000};                                // guard, round, sticky positions
always_comb begin                                         // align the smaller operand
    t = '0;
    if (d >= 10'd27) sm = {26'd0, mS};
    else begin
        t = {mS, 3'b000} >> d;
        sm = t | {26'd0, |({mS, 3'b000} & ((27'd1 << d) - 27'd1))}; // jam lost bits into sticky
    end
end
assign eff_sub = sB ^ sS;
assign S = eff_sub ? ({1'b0, bg} - {1'b0, sm}) : ({1'b0, bg} + {1'b0, sm});
logic [5:0] lz6;
lzc64 clz (.x({S[26:0], 37'h1F_FFFF_FFFF}), .lz(lz6)); // leading zeros of S[26:0], at most 27
assign lz = lz6[4:0];
always_comb begin                                         // normalize
    if (S[27]) begin
        Sn = {1'b0, S[27:1]} | {27'd0, S[0]};
        E  = EB + 10'd1;
        shl = '0;
    end else begin
        shl = ({5'd0, lz} < (EB - 10'd1)) ? {5'd0, lz} : (EB - 10'd1); // stop at the subnormal range
        Sn  = S << shl;
        E   = EB - shl;
    end
end
assign g  = Sn[2];
assign st = Sn[1] | Sn[0];
assign inc = g & (st | Sn[3]);
assign rnd = {1'b0, Sn[26:3]} + {24'd0, inc};
always_comb begin
    if (rnd[24]) Ef = {1'b0, E} + 11'd1; // rounding overflowed the significand
    else if (rnd[23]) Ef = {1'b0, E};
    else Ef = 11'd0; // subnormal result (E is 1 here)
    if (Ef >= 11'd255) fin = {sB, 8'hFF, 23'd0};
    else fin = {sB, Ef[7:0], rnd[24] ? rnd[23:1] : rnd[22:0]};
end
always_comb
    if (nana || nanb || (infa && infb && (sa != sb))) y = QNAN;
    else if (infa) y = {sa, 8'hFF, 23'd0};
    else if (infb) y = {sb, 8'hFF, 23'd0};
    else if (za && zb) y = {sa & sb, 31'd0}; // +0 unless both are -0
    else if (za) y = {sb, eb, fb};
    else if (zb) y = a;
    else if (S == 28'd0) y = 32'd0; // exact cancellation gives +0
    else y = fin;
endmodule

```

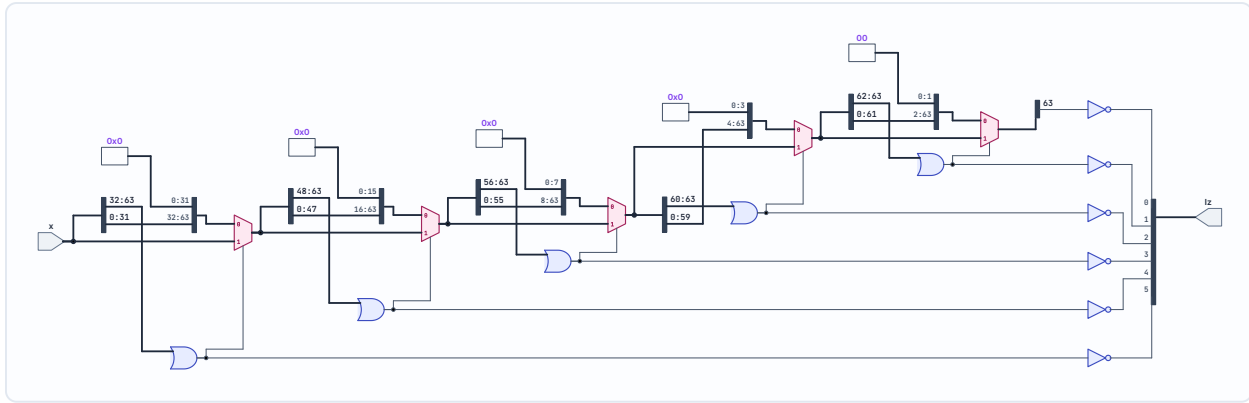


Figure 9.3. Logarithmic-depth leading-zero counter: six stages each test the top 2^k bits with a reduction OR and shift them out. YOSYS → NETLISTSVG

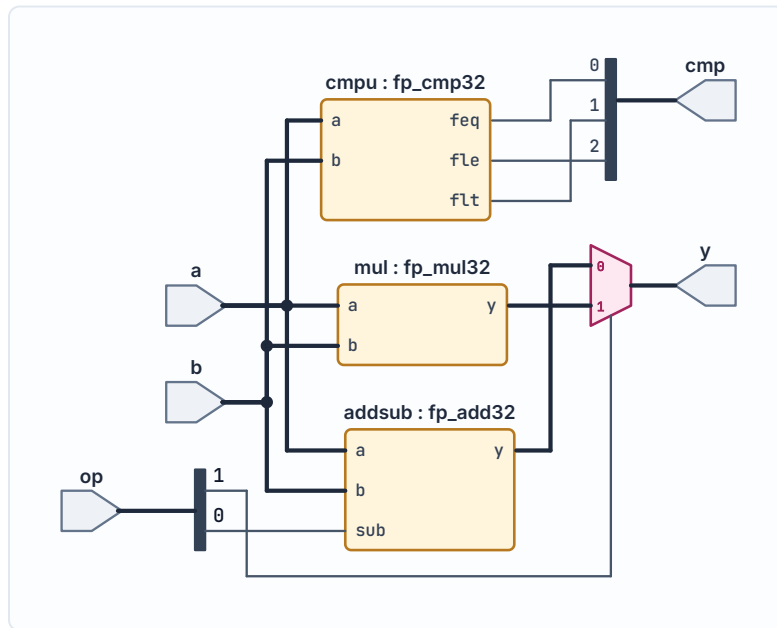


Figure 9.4. The FPU core: adder/subtractor, multiplier and comparator in parallel; *op* selects the arithmetic result. YOSYS → NETLISTSVG

Verification. A NumPy reference model (`sim/fpvec.py`) generated 300 015 operand pairs, each applied as an addition, a subtraction and a multiplication: 900 045 vectors in all. Many operands are arbitrary 32-bit patterns. The rest are biased toward the hard cases:

- ordinary, tiny, huge and subnormal magnitudes, and edge fractions;
- zeros, infinities and NaNs;
- exact cancellation ($a - a$) and nearly equal operands;
- products aimed at the overflow and underflow thresholds.

Among the expected results are 50 931 NaNs, 94 480 infinities, 134 228 zeros (43 392 of them -0) and 57 261 subnormals. The FPU matched every result bit for bit, and every comparison as well; NaN results were required to be the canonical NaN. To show that the test can fail, five realistic bugs were injected one at a time and run against a 60 045-vector subset. Each was detected (Table 9.2).

Table 9.2. FPU mutation testing (60 045 vectors each; mismatch counts per operation).

injected bug	mismatching results	by operation
multiplier rounds ties away from zero (no ties-to-even)	38 of 60 045	add 0, sub 0, mul 38
adder rounds ties away from zero	1 040 of 60 045	add 827, sub 213, mul 0
multiplier ignores gradual underflow	4 968 of 60 045	add 0, sub 0, mul 4968
adder drops the sticky bit when aligning	120 of 60 045	add 60, sub 60, mul 0
adder returns +0 for $(-0) + (-0)$	113 of 60 045	add 31, sub 82, mul 0

9.5 What floating point costs in hardware

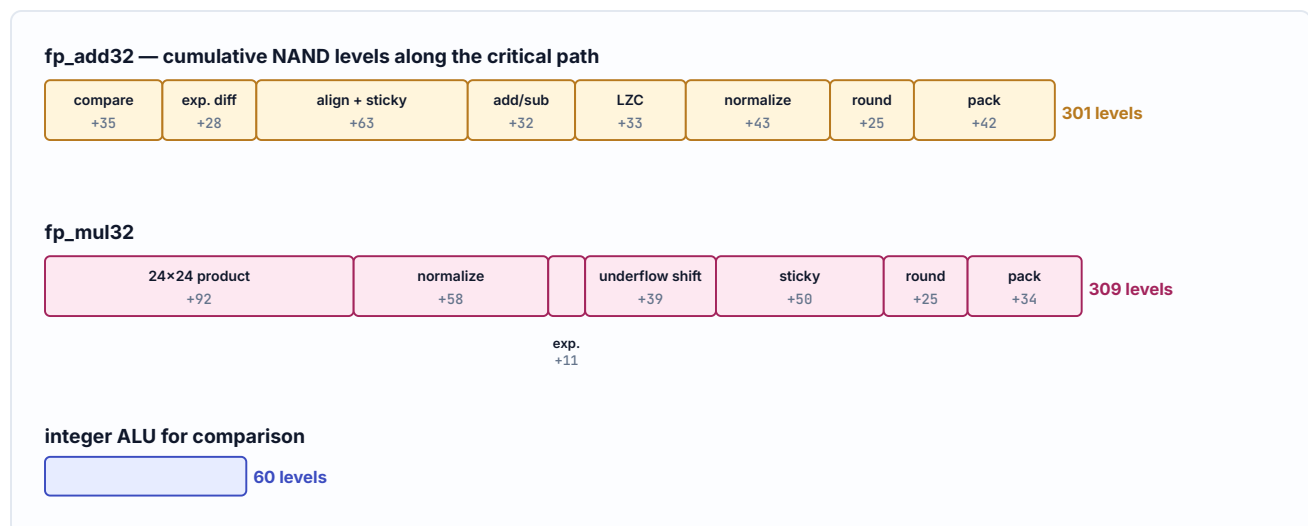
Table 9.3 measures the FPU with the structural timing model of Section 8.11: gates costed at their NAND depth, and Brent–Kung carry networks.

Table 9.3. Size and depth of the FPU blocks (combinational), with the integer ALU for comparison.

block	gates	NAND-equivalent	depth (NAND levels)
integer ALU (Section 4.5, for reference)	614	—	60
comparison (fp_cmp32)	356	940	50
add / subtract (fp_add32)	2 296	7 236	301
multiply (fp_mul32)	5 704	17 233	309

The floating-point adder and multiplier are each about five times deeper than the entire integer ALU, and have about 4 and 9 times as many gates. Figure 9.5 shows where the depth goes, as cumulative arrival times at the named stage boundaries:

- adder: compare magnitudes 35 → exponent difference 63 → align + sticky 126 → add / subtract 158 → leading-zero count 191 → normalize 234 → round 259 → pack + specials 301;
- multiplier: significand product 92 → normalize 150 → result exponent 161 → underflow shift 200 → sticky bit 250 → round 275 → pack + specials 309.

**Figure 9.5.** Where the depth goes: cumulative NAND levels at each stage boundary of the floating-point adder and multiplier, against the integer ALU.

Three conclusions follow.

- **Clock period.** The pipeline of Chapter 8 has a 97-level period, of which the ALU takes 60. Putting a combinational FPU in its place would stretch the Execute path to roughly $97 - 60 + 309 \approx 346$ levels (an estimate, not a synthesis result), more than tripling the clock period for *every* instruction. Real processors therefore split floating-point operations over several pipeline stages or clock cycles, and hold the rest of the processor while the result is not ready (Section 9.7).
- **Clear optimization targets.** Alignment with its sticky bit costs 63 levels, because the design forms the sticky mask by subtraction. Production FPUs instead derive the sticky bit from a trailing-zero count, predict the normalization shift in parallel with the addition (leading-zero anticipation), and use tree multipliers (Appendix A).
- **Area.** The FPU is larger than the rest of the integer core. That is why small microcontrollers often omit it and emulate floating point in software.

9.6 Using the FPU as a memory-mapped coprocessor

The least invasive way to give the existing processors floating-point capability is to attach the FPU to the data bus, like any peripheral (Appendix C.8). The wrapper `fpu_mmio` exposes five registers at 0x400–0x413:

- 0x400 and 0x404: operands A and B;
- 0x408: the operation;
- 0x40C: the result (read-only);
- 0x410: the comparison flags (read-only).

A decoder selects them whenever `a[31:5] = 32`, and every other address goes to the data memory, whose write enable is gated off for this range (Figures 9.6 and 9.7). No processor RTL changes at all. The same bus module was attached to all four processors, and software uses the FPU with ordinary `sw` and `lw` instructions.

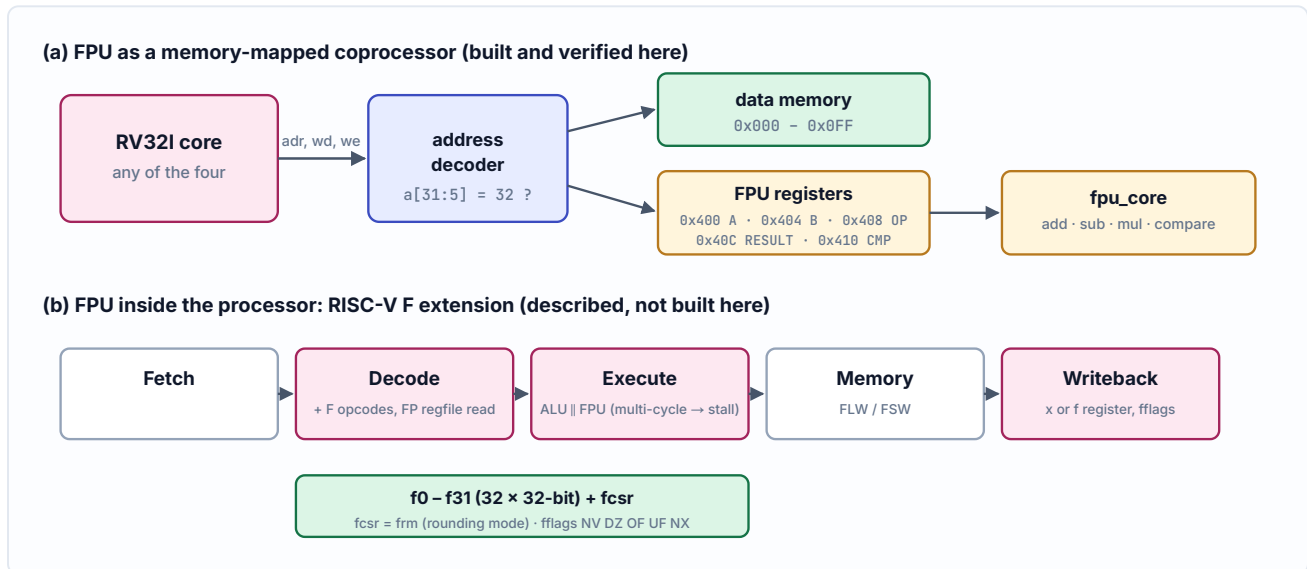


Figure 9.6. (a) The memory-mapped FPU built here; (b) the RISC-V F extension, which moves floating point into the processor.

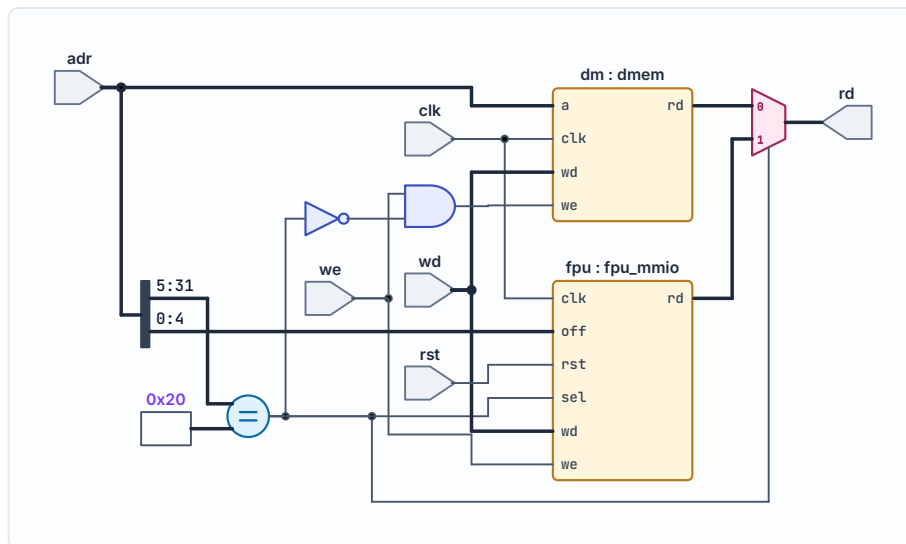


Figure 9.7. The bus: the address decoder ($a[31:5] = 32$, the constant $0x20$), the data memory with gated write enable, the FPU, and the read-back multiplexer.

YOSYS → NETLISTSVG

The program `sim/fpudemo.py` walks through ten operand pairs stored in data memory. For each pair it writes A and B, selects addition and reads the result, selects multiplication and reads the result, then reads the comparison flags, storing all three back into memory. On all four processors, every stored result matched the NumPy reference (Table 9.4). The table includes the cases that make floating point subtle:

- the tie $2^{24} + 1$ rounds back to 2^{24} ;
- a subnormal times 0.5 rounds to even;
- $1e30 - 1e30$ gives +0;
- $\infty + (-\infty)$ gives the canonical NaN;
- overflow gives ∞ ;
- every comparison involving NaN or $\pm\infty$ of opposite signs behaves as the rules of Section 9.2 require.

Table 9.4. Results of the FPU demonstration program (identical on the single-cycle, pipelined, predicting and multicycle processors). `cmp = {a≤b, a<b, a=b}`.

operands	a	b	a + b	a × b	cmp
1.5 and 2.0	3FC00000	40000000	40600000	40400000	110
0.1 and 0.2 (inexact)	3DCCCCCD	3E4CCCCD	3E99999A	3CA3D70B	110
16 777 216 and 1 (tie)	4B800000	3F800000	4B800000	4B800000	000
3.0 and -0.5	40400000	BF000000	40200000	BFC00000	000
largest finite, twice	7F7FFFFF	7F7FFFFF	7F800000	7F800000	101
subnormal and 0.5	00000003	3F000000	3F000000	00000002	110
-0 and -0	80000000	80000000	80000000	00000000	101
$+\infty$ and $-\infty$	7F800000	FF800000	7FC00000	FF800000	000
NaN and 1.0	7FC00000	3F800000	7FC00000	7FC00000	000
$1e30$ and $-1e30$	7149F2CA	F149F2CA	00000000	FF800000	000

The price of this approach is speed and convenience. Every operation costs several load and store instructions, the operands occupy the FPU's registers rather than dedicated floating-point registers, and the compiler cannot use it without a custom library. Early personal computers used separate coprocessor chips, with a tighter interface than this but the same basic idea.

9.7 Extending the processor: the RISC-V F extension

The F extension [13] makes floating point part of the instruction set. The following subsections describe how it would be added to the pipeline of Chapter 8; that integration is not built here.

New state

- 32 floating-point registers `f0 – f31`, 32 bits each for single precision. Unlike `x0`, `f0` is an ordinary register.
- The `fcsr` register, holding the rounding mode `frm` and the five sticky exception flags `fflags`.

New instructions

- `FLW` and `FSW` load and store floating-point values. They use two new major opcodes (LOAD-FP 0000111 and STORE-FP 0100111) with the same formats as `lw` and `sw`.
- All other operations share the OP-FP opcode 1010011 and are distinguished by `funct7`:
 - arithmetic: `FADD.S` 0000000, `FSUB.S` 0000100, `FMUL.S` 0001000, `FDIV.S` 0001100 and `FSQRT.S` 0101100;
 - sign injection `FSGNJ` and `FMIN / FMAX`;
 - comparisons `FEQ / FLT / FLE` (`funct7` 1010000), which write 0 or 1 into an *integer* register;
 - conversions `FCVT` between integers and floats;
 - bit moves `FMV` between `x` and `f` registers, and `FCLASS`.
- For arithmetic operations the `funct3` field carries a rounding mode; 111 means "use `frm`".

Datapath changes

Figure 9.6(b) summarizes them; the list follows the stages.

1. **Decode.** The main decoder recognizes the new opcodes and produces extra control signals. These say which register file each of `rs1`, `rs2` and `rd` refers to (integer or floating point), and whether the result comes from the ALU or the FPU. A second register file holds `f0 – f31`.
2. **Execute.** The FPU sits beside the ALU. Because it is far slower (Section 9.5), it is built as a multicycle or internally pipelined unit with a *valid/ready* handshake. While it works, the hazard unit stalls Fetch and Decode and inserts bubbles — the load-use stall of Section 8.5, held for several cycles.
3. **Forwarding.** Forwarding is duplicated for the floating-point registers. Its comparisons must also check that source and destination are the same *kind* of register, since `x5` and `f5` are unrelated.
4. **Writeback.** The result is written to the integer or floating-point register file. The FPU's exception flags are ORed into `fflags` only when the instruction commits, so a squashed instruction never sets a flag.
5. **Operating-system support.** The privileged architecture adds a status field (FS in `mstatus`) that tracks whether floating-point state is in use, so an operating system saves the 32 extra registers only when necessary.

Software view

RISC-V software is compiled for an ABI. With `ilp32f`, floating-point arguments travel in `f` registers and the compiler emits F instructions directly. With `ilp32`, the compiler calls library routines ("soft-float") that perform the same IEEE arithmetic with integer instructions. That library route is how the processors of this primer would run floating-point C code today, and it is exactly the kind of software the FPU of Section 9.6 would accelerate.

CHAPTER 10

Verification

A design is only as trustworthy as its checks. This chapter records what was verified and how. All checks run with Icarus Verilog 12.0; Table 10.1 lists them. The unit testbench compiles every module with the `SIM` macro, so NAND gates have unit delay and the feedback circuits settle as real ones would.

Table 10.1. Unit verification. "Exhaustive" means every input combination was applied.

Device under test	Stimulus	Checks
NOT, AND, OR, XOR, MUX, half adder, full adder (NAND-structural)	exhaustive over 4 inputs	48
4-bit ripple adder, 4-bit CLA, 4×4 array multiplier	exhaustive, 512 vectors	1 536
4-bit ALU, all eight control codes (value and zero)	exhaustive, 2 048 vectors	4 096
D latch (transparency, hold), master-slave flip-flop (edge sampling, hold)	directed	8
8-bit shift-add and Booth multipliers; restoring and non-restoring dividers	exhaustive, 65 536 operand pairs each	261 888
Subtotal, unit testbench		267 576 — 0 failures
Generated 8×8 Dadda multiplier (adder and ripple-CPA variants) and 8×8 array multiplier	exhaustive, 65 536 pairs each	0 failures (separate testbenches)
16 × 8-bit register memory (Appendix C): unique fill, walking ones, write disable, random traffic	directed + 20 000 random operations	22 320 — 0 failures
Hardwired vs. microprogrammed controller (Appendix D)	128 opcodes × 12 cycles + 200 000 random cycles; cores 11 × 128	201 536 + 1 408 — 0 mismatches
Number-system program (Chapter 3) on all four processors (five configurations)	directed, compared with the reference simulator	all registers and memory match
IEEE 754 FPU (Chapter 9): add, subtract, multiply, compare	900 045 vectors from a NumPy reference	0 mismatches; 5 injected bugs all detected
FPU attached to all four processors	10 operand pairs, 30 results each	all results match
<i>Negative control:</i> one microcode bit flipped	same stimulus	716 mismatches (expected)
<i>Negative control:</i> Booth multiplier with the guard bit removed	exhaustive, 65 536 pairs	255 failures, all with $a = -128$ (expected)

The processors were first verified by *co-simulation against a reference model* on the directed program. A Python instruction-set simulator, written independently of the RTL, executes the program and records the final registers and memory. Each processor then runs for 200 cycles, and its register file and data memory are compared with that record. A separate trace confirms that the single-cycle core reaches the halt instruction on its 60th cycle. Both the single-cycle and the pipelined core matched in all 31 writable registers and in every data word. The test program was built to exercise each hazard class and the sign-sensitive corner cases (Section 6.3). Section 10.1 then goes beyond the directed program with randomly generated ones.

10.1 Random-program co-simulation

A directed program tests what its author thought of. To go beyond that, a generator (`sim/cosim.py`) produces random but always-terminating programs, and each one runs on the reference simulator and on five processor configurations: single-cycle, pipeline, pipeline with prediction, and the multicycle processor with each of its two controllers. Table 10.2 summarizes the run. The comparison is stricter than a final-state check. Every register write (register and value) and every memory write (address and value) is logged in order, and the five streams must equal the reference streams exactly.

Programs are assembled from blocks, and each block type targets a known risk:

- **Random operations.** Random R- and I-type operations over a small pool of registers (x1–x8), so dependencies between neighbouring instructions are frequent. Immediates are biased toward the edges of their range.
- **Memory access.** Stores and loads relative to a base register, with negative offsets included, followed by immediate use of the loaded value (load-use).
- **Counted loops.** Loops with backward branches and jumps; register x31 is reserved as the counter.
- **Forward transfers.** Forward conditional branches and forward `jal`, including links into x0.
- **Write-write-read chains.** Two consecutive writes to one register followed by a read, which tests forwarding priority.
- **Overflow corners.** A positive and a negative value doubled repeatedly toward $\pm 2^{31}$, then compared, which reaches signed-overflow corners.
- **Mixed branch outcomes.** A loop containing a parity-dependent branch (taken, not taken, taken, ...), which forces predicted-taken branches to fall through.

Table 10.2. Random co-simulation totals (seeds 1000–2999).

Quantity	Value
Programs	2 000
Instructions executed (reference simulator)	249 472
Register writes / memory writes compared	163 538 / 10 326
Load-use stalls exercised (both pipelines)	1 783
Forwards into Execute: from M / from W (pipeline, used operands only)	46 131 / 36 066
Flush events: without / with prediction	40 359 / 18 019
Mismatches (single, pipeline, pipeline+BP, multicycle ×2)	0, 0, 0, 0, 0

Static instruction mix of the generated programs: `addi` 23 %, `beq` 11 %, `sw` 8 %, `jal` 8 %, `lw` 7 %, `slt` 6 %, `add` 6 %, `andi` 6 %, `sub` 6 %, `ori` 5 %, `slti` 5 %, `or` 5 %, and 5 %.

Building this harness exposed a real flaw in the single-cycle core. At the time, its register file wrote on the falling clock edge, so an instruction's write could land while reset was still asserted, before logging began. Architectural state should not change during reset, so the core now gates its register and memory write enables with reset (Listing 7.1); Section 8.11 later moved its register file to the rising edge. The directed results of Chapters 7 and 8 were re-run and are unchanged.

10.2 Mutation testing: does the test bench have teeth?

A test that never fails proves little unless it is known to be able to fail. *Mutation testing* [31] injects one plausible bug at a time and counts how many test programs expose it (Table 10.3). The first version of the generator caught the forwarding-priority bug in only 10 of 200 programs and the SLT-overflow bug in

none. After chain and doubling blocks were added, the SLT bug was still caught only 6 times, and a newly added prediction-recovery bug was never caught. Those gaps are what led to the final two-sign doubling and parity blocks of Section 10.1. With them, every behaviour-changing mutant is caught.

One mutant is never caught, and cannot be: it is *equivalent*. As Section 8.10 explains, the target check it removes can never matter in this predictor.

Table 10.3. Mutation testing (200 fresh random programs per mutant, seeds 5000–5199).

Injected bug	Processor	Programs exposing it	Note
forwarding priority reversed (W before M)	pipeline	56 / 200	
load-use stall removed	pipeline	79 / 200	
predictor target not checked	pipeline + prediction	0 / 200	equivalent mutant — cannot change behaviour
misprediction recovery always jumps to the target	pipeline + prediction	56 / 200	
SLT overflow correction removed	single-cycle	21 / 200	
microcode: ExecuteI uses register, not immediate	multicycle (microcoded)	200 / 200	

10.3 Formal verification

Simulation samples behaviour; a formal proof covers all of it. Each property below, labelled F1–F10, is written as SystemVerilog assertions in a wrapper module with unconstrained inputs (`hdl/formal.sv`) and discharged by Yosys' built-in SAT solver [16]. Combinational properties use `sat -prove-asserts`. The sequential equivalence of the two controllers (F7) uses `sat -tempinduct`: the property holds in the reset state, and if the two machines agree in one cycle they agree in the next, so they agree forever. Every job completed (Table 10.4). 8 of the 10 proofs took under a second; the slowest was the 8×8 multiplier equivalence (40 s), the hardest SAT problem here.

Table 10.4. Formal proofs. Each covers every input value (F7: every input sequence from reset).

Property	Result	Solver time
F1 adders (32-bit KS = RCA = +)	proved	9.8 s
F2 multipliers (8×8 array = Dadda = *)	proved	40.2 s
F3 ALU (32-bit) meets specification	proved	0.3 s
F4 hazard-unit safety properties	proved	0.3 s
F5 write decoder one-hot	proved	0.3 s
F6 control cores equal (16 states × 128 ops)	proved	0.3 s
F7 controllers sequentially equivalent	proved	0.4 s
F8 K-map forms of segment e (valid BCD)	proved	0.3 s
F9 majority SoP = PoS = full-adder carry	proved	0.3 s
F10 NAND-level operators equal +, −, ==, < (signed and unsigned)	proved	0.4 s

The proofs add what no amount of simulation can:

- **F1** covers all 2^{65} input combinations of the 32-bit adders.

- **F6** includes the five state codes that never occur in simulation, where the two controllers could in principle have differed.
- **F4** states the hazard unit's safety rules explicitly: legal selects, no forwarding of x0, forwarding only from a real writer, the newest value wins, and every stall and redirect inserts the required bubbles.

To confirm the proofs are not vacuous, four of the bugs were injected again and each property re-run (Table 10.5); every one produced a counterexample. The SLT counterexample is instructive. With $a = 0x80040000$ and $b = 0x00080000$, $a - b$ overflows, and the uncorrected sign bit says "not less than". Random co-simulation exposed this bug in only 21 of 200 programs; the solver found it in a fraction of a second.

Table 10.5. Negative controls: each injected bug must make its proof fail.

Mutated design	Outcome
F1 with a Kogge–Stone combine cell wired to the wrong position	counterexample
F3 with the SLT overflow correction removed	counterexample
F4 with forwarding priority reversed	counterexample
F6 with one flipped microcode bit (JAL PCUpdate)	counterexample

WHAT THESE CHECKS DO NOT COVER

The processors implement only the instruction subset of Chapter 6, with no exceptions, interrupts or control and status registers, so neither random nor formal checks say anything about those. The random programs are bounded to 64 instruction words and a 64-word data memory. The multipliers and dividers of the appendices were exhausted at 8 bits, and the 32-bit adders and ALU were proved. Gate depth is a logical measure, not an electrical one: it ignores fan-out, wire load and the different speeds of pull-up and pull-down networks.

CHAPTER 11

Summary

Starting from one gate, this primer built a working, verified computer:

- NAND alone suffices for all logic: a NOT costs one gate, a full adder nine.
- Two's complement lets one adder serve signed and unsigned numbers. Only the reading of the result and of overflow differs, and RISC-V leaves overflow checks to software instead of keeping flags.
- Linear carry propagation makes the ripple adder's depth exactly $2N + 3$, which motivates lookahead and prefix carry computation.
- Cross-coupled NANDs store a bit, and two latches in master-slave form store it on a clock edge. The setup and hold inequalities then define what "synchronous" means.
- On that foundation a single-cycle RV32I-subset processor follows almost mechanically from the instruction formats.
- Pipelining that processor buys clock period, and three mechanisms buy back correctness: forwarding for data, stalling for loads and flushing for branches. Measured on a branch-heavy program, they cost 23 cycles over 59 instructions; a 2-bit branch-target buffer cuts that to 7. Measured clock periods show why balance matters: with a fast memory the pipeline's overloaded Execute stage leaves its period within 8 % of the single-cycle machine's, and pipelining pays only once memory access is slow (Section 8.11).
- Canonical SoP/PoS forms and Karnaugh maps turn truth tables into minimal two-level logic, and a Kogge–Stone prefix adder cuts 64-bit carry depth from 132 NAND levels to 18.
- Four processors — single-cycle, multicycle, pipelined and predicting — agree with the reference model on 2 000 random programs, and ten properties hold by formal proof. Mutation testing shows that both methods catch real bugs, and that random testing needs deliberately targeted patterns to do so.

Floating point needs far more hardware than integer arithmetic. A correctly rounded binary32 adder and multiplier are each about five times deeper than the whole integer ALU. Attached as a memory-mapped coprocessor, the verified FPU gives all four processors IEEE 754 arithmetic without any change to their RTL.

The appendices complete the picture. Multiplication trades area for speed along the axis from the shift-add iteration to Booth recoding and tree reduction. Division is inherently sequential and is accelerated mainly by redundant digit sets. Memory addressing reduces to one idea used twice: decode the address to enable a write, and multiplex on it to read. Microprogramming stores control sequences instead of wiring them. With the contents fixed, the tools reduce both forms to the same gates, so the choice between them is about flexibility, not size.

APPENDIX A

Multiplication

A.1 Partial products

Binary long multiplication forms one partial product per multiplier bit, $pp_i = a \cdot b_i \cdot 2^i$, and adds them:

$$p = \sum_{i=0}^{n-1} (a b_i) 2^i, \quad pp_{i,j} = a_j \wedge b_i.$$

Each partial product is a row of AND gates. An $n \times n$ unsigned product fits in $2n$ bits. Hardware multipliers differ only in how, and how fast, they add these n rows.

A.2 The array multiplier

The direct approach adds the rows with $n - 1$ ripple adders, each absorbing one partial product (Listing A.1, Figure A.1). The structure is regular and easy to lay out, but the worst-case path crosses every row and, within each row, the full carry chain, so delay grows as $O(n)$ with a large constant. The 4×4 instance was checked against every one of its 256 input pairs (plus the unused carry-in dimension of the testbench).

Listing A.1. 4×4 array multiplier: AND-plane partial products, three ripple rows.

arith.sv

```
// 4x4 unsigned array multiplier: 16 AND partial products, three 4-bit ripple rows.
module mul_array4 (input logic [3:0] a, b, output logic [7:0] p);
  logic [3:0] pp0, pp1, pp2, pp3, s1, s2, s3;
  logic      c1, c2, c3;
  assign pp0 = a & {4{b[0]}};
  assign pp1 = a & {4{b[1]}};
  assign pp2 = a & {4{b[2]}};
  assign pp3 = a & {4{b[3]}};
  rca4 row1 (.a({1'b0, pp0[3:1]}), .b(pp1), .cin(1'b0), .s(s1), .cout(c1));
  rca4 row2 (.a({c1, s1[3:1]}), .b(pp2), .cin(1'b0), .s(s2), .cout(c2));
  rca4 row3 (.a({c2, s2[3:1]}), .b(pp3), .cin(1'b0), .s(s3), .cout(c3));
  assign p = {c3, s3, s2[0], s1[0], pp0[0]};
endmodule
```

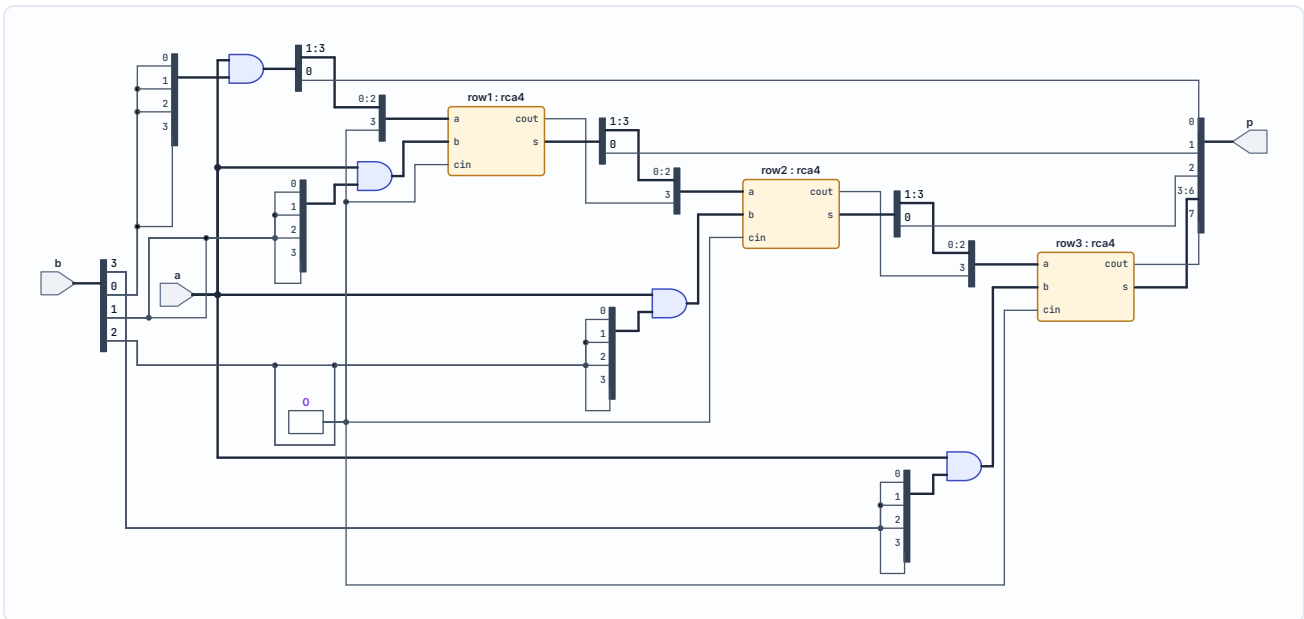


Figure A.1. Synthesized array multiplier. The four AND cells form the partial-product rows; each *rca4* instance adds one row to the shifted running sum.

YOSYS → NETLISTSVG

A.3 Sequential shift-and-add

When area matters more than speed, a single adder can be reused n times. A $(2n+1)$ -bit register $\{hi, lo\}$ starts with $hi = 0$ and $lo = b$. Each iteration adds a to hi if $lo_0 = 1$, then shifts the whole register right by one. The multiplier's bits are consumed from the bottom while the product fills in from the top, so after n iterations $\{hi, lo\}$ holds $a \times b$. The combinational core of one iteration is the `mul_step` module (Figure A.2), which the sequential wrapper instantiates (Listing A.2).

Listing A.2. One shift-add iteration and its sequential wrapper.

arith.sv

```

// One iteration of shift-and-add: conditionally add, then shift {hi,lo} right.
module mul_step #(parameter int W = 4) (
  input  logic [W:0] hi,
  input  logic [W-1:0] lo, m,
  output logic [W:0] hi_n,
  output logic [W-1:0] lo_n);
  logic [W-1:0] addend;
  logic [W:0] sum;
  assign addend = lo[0] ? m : '0;           // multiplier LSB selects the addend
  assign sum = {1'b0, hi[W-1:0]} + {1'b0, addend};
  assign hi_n = {1'b0, sum[W:1]};
  assign lo_n = {sum[0], lo[W-1:1]};
endmodule

// Sequential shift-and-add multiplier (unsigned). W iterations, one adder.
module mul_seq #(parameter int W = 8) (
  input  logic clk, rst, start,
  input  logic [W-1:0] a, b,
  output logic [2*W-1:0] p,
  output logic done);
  logic [W:0] hi;           // accumulator with carry bit
  logic [W-1:0] lo, m;      // lo starts as multiplier, fills with product
  logic [W:0] hi_n;
  logic [W-1:0] lo_n;
  logic [$clog2(W+1)-1:0] cnt;
  mul_step #(W) step (.hi, .lo, .m, .hi_n, .lo_n);
  always_ff @(posedge clk)
    if (rst) begin cnt ≤ '0; done ≤ 1'b0; end
    else if (start) begin hi ≤ '0; lo ≤ b; m ≤ a; cnt ≤ W; done ≤ 1'b0; end
    else if (cnt ≠ 0) begin
      hi ≤ hi_n; lo ≤ lo_n;           // add-then-shift in one step
      cnt ≤ cnt - 1'b1;
      done ≤ (cnt == 1);
    end
  end
  assign p = {hi[W-1:0], lo};
endmodule

```

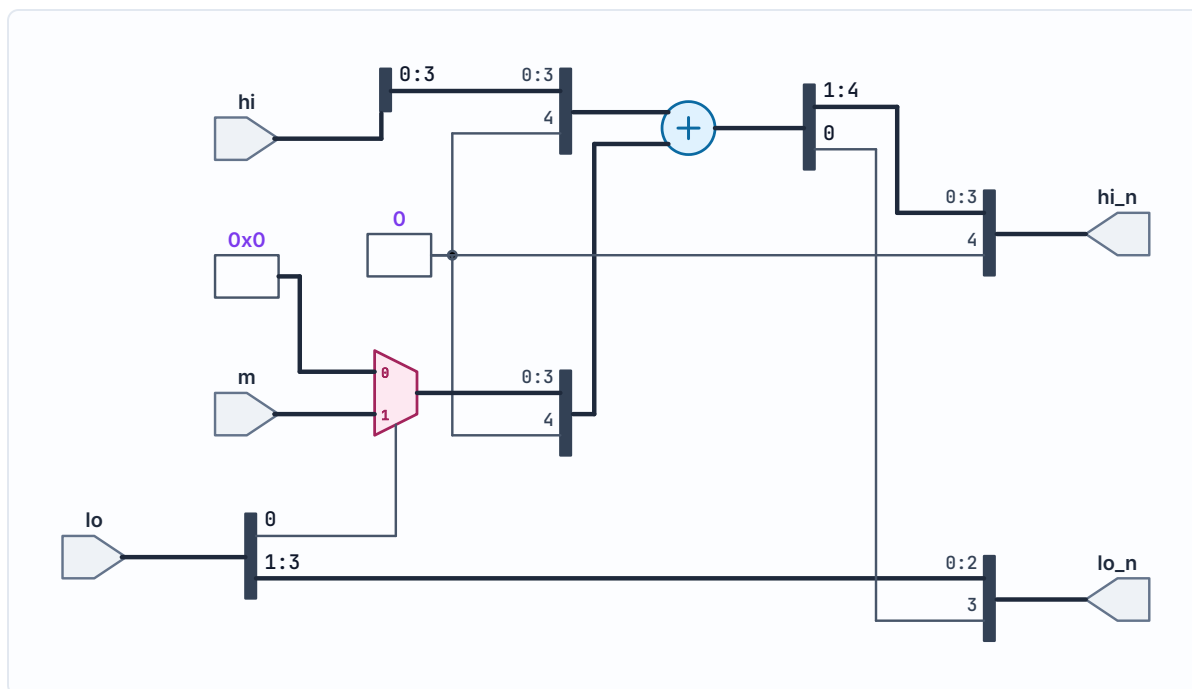


Figure A.2. One iteration at $W = 4$: the multiplexer selects m or 0 according to lo_0 , the adder adds it to hi , and the split/join bars implement the one-bit right shift of the concatenation.

YOSYS → NETLISTSVG

A.4 Booth recoding

Andrew Booth's 1951 algorithm [7] multiplies signed numbers directly and skips runs of ones. It relies on a telescoping identity that rewrites a two's-complement multiplier as a sum of signed digits:

$$\langle b \rangle = \sum_{i=0}^{n-1} (b_{i-1} - b_i) 2^i, \quad b_{-1} = 0,$$

which holds because the sum telescopes to $-b_{n-1}2^{n-1} + \sum_{i < n-1} b_i 2^i$. Each digit $d_i = b_{i-1} - b_i$ lies in $\{-1, 0, +1\}$. Scanning the pair (b_i, b_{i-1}) therefore tells the multiplier to subtract a at the start of a run of ones, add it at the end of the run, and do nothing inside a run (Table A.1).

Table A.1. Radix-2 Booth recoding.

b _i	b _(i-1)	digit d_i	action
0	0	0	shift only
0	1	+1	add a , shift (end of a run of 1s)
1	0	-1	subtract a , shift (start of a run)
1	1	0	shift only (inside a run)

The shift must be arithmetic, so that the sign is preserved. The accumulator also needs one guard bit. For $n = 8$ and $a = -128$, the step "subtract a " from zero yields $+128$, which does not fit in 8 bits. A negative control confirms this: with the guard bit removed, the exhaustive test fails on 255 of the 65 536 pairs, every one of them with $a = -128$. Listing A.3 keeps the guard bit and passes. Radix-2 Booth does not reduce the worst-case number of additions; alternating bit patterns are its worst case. Its lasting value is the *modified* (radix-4) recoding attributed to MacSorley [15]. It examines overlapping triples $(b_{2i+1}, b_{2i}, b_{2i-1})$ and produces digits in $\{-2, \dots, +2\}$, which halves the number of partial products. Every such digit is a shift and an optional negation of a , so no hard multiples are needed. Most fast multipliers use it.

Listing A.3. Sequential radix-2 Booth multiplier (signed) with a guard bit.

arith.sv

```
// Sequential Booth (radix-2) multiplier, signed.
module mul_booth #(parameter int W = 8) (
  input logic clk, rst, start,
  input logic signed [W-1:0] a, b,
  output logic signed [2*W-1:0] p,
  output logic done);
  logic signed [W:0] acc, m, nxt; // one guard bit avoids overflow at M = -2^(W-1)
  logic [W-1:0] q;
  logic q_1;
  logic [$clog2(W+1)-1:0] cnt;
  always_comb
    case ({q[0], q_1})
      2'b01: nxt = acc + m; // end of a run of 1s
      2'b10: nxt = acc - m; // start of a run of 1s
      default: nxt = acc; // inside a run: no operation
    endcase
  always_ff @(posedge clk)
    if (rst) begin cnt ≤ '0; done ≤ 1'b0; end
    else if (start) begin acc ≤ '0; m ≤ {a[W-1], a}; q ≤ b; q_1 ≤ 1'b0;
      cnt ≤ W; done ≤ 1'b0; end
    else if (cnt ≠ 0) begin
      {acc, q, q_1} ≤ $signed({nxt, q, q_1}) >> 1; // arithmetic shift right
      cnt ≤ cnt - 1'b1;
      done ≤ (cnt = 1);
    end
  assign p = {acc[W-1:0], q};
endmodule
```

A.5 Tree multipliers

Adding partial products one row at a time is slow because every addition waits for a carry. Wallace [12] observed that carries need not be propagated until the very end. A full adder used as a 3:2 *counter* takes three bits of equal weight and emits a sum of the same weight plus a carry of the next weight. Layers of such counters can therefore reduce the partial-product matrix in parallel until only two rows remain, and a single fast carry-propagate adder (CPA) then finishes the job. Each layer reduces the height by roughly a factor of 3/2, so the number of layers is $O(\log n)$. Dadda [14] refined the schedule to reduce each column only as much as each layer requires, which minimizes the number of counters in the tree. The maximum heights after successive layers follow the sequence

$$d_1 = 2, \quad d_{j+1} = \left\lfloor \frac{3}{2} d_j \right\rfloor \Rightarrow 2, 3, 4, 6, 9, 13, 19, 28, \dots$$

An 8×8 multiplier, with matrix height 8, needs four layers ($8 \rightarrow 6 \rightarrow 4 \rightarrow 3 \rightarrow 2$), against seven ripple rows for the array multiplier. Applying Dadda's rule column by column — reduce a column only as far as the next target height, using a full adder when two or more bits must go and a half adder when one must — gives the exact schedule of Figure A.3: 35 full adders and 7 half adders, leaving two rows that span bit weights 1 to 14. That schedule was generated programmatically, turned into a structural multiplier (`dadda8.sv`), and tested against all 65 536 operand pairs. Combined with radix-4 Booth recoding, which halves the initial height, and a parallel-prefix final adder, this is the canonical high-performance multiplier [9, 23].

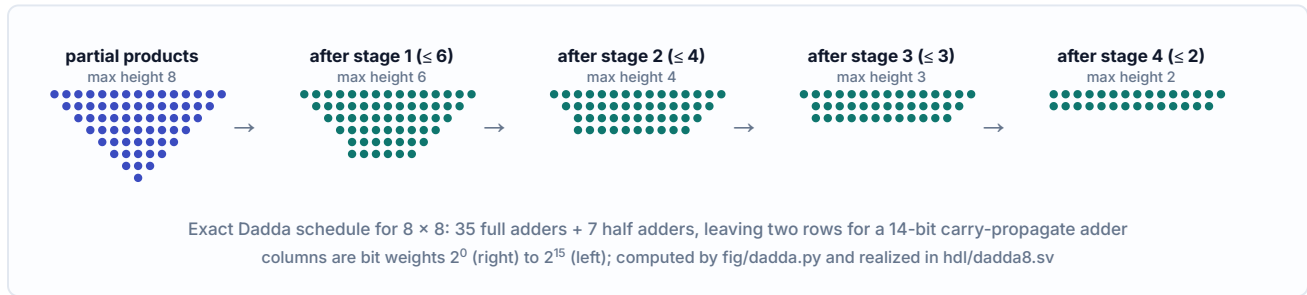


Figure A.3. Exact Dadda reduction for 8×8 unsigned multiplication. Each dot is a bit; each column is one weight. Blue: initial partial products; teal: bits remaining after each counter layer.

Table A.2 measures what the tree buys. Both multipliers were built from the same nine-NAND full adders, flattened, and timed as in Section 4.3 (a partial-product AND counts as two NAND levels). To isolate the effect of the tree, the Dadda version finishes with a plain ripple adder. Even so, its critical path is 40 % shorter than the array's, with slightly fewer gates. The same cell-level delay model, which reproduces both static-timing results exactly, puts the end of the tree itself at 24 levels, so the ripple adder adds 18 more. Replacing it with the 16-bit Kogge–Stone adder of Section 4.6 (`dadda8_ks.sv`, also verified exhaustively and formally) shortens the path only from 42 to 38 levels, at a cost of 230 extra gates. The reason is the tree's arrival profile. Its low- and high-order columns are shallow and finish early, so a ripple carry travels through them while the middle columns are still being reduced, and a uniform prefix adder cannot exploit that. Production multipliers therefore use final adders shaped to the arrival times of their inputs [9].

Table A.2. Measured NAND-level critical path of 8×8 multipliers (flattened netlists, all verified exhaustively and proved equal to a·b). FA = full adder, HA = half adder, KS16 = 16-bit Kogge–Stone adder.

Design	Counters	NAND gates	AND2 cells	Critical path
Array (mul_array8.sv): 7 ripple rows	56 FA	504	64	70 levels
Dadda tree + ripple CPA (dadda8_rca.sv)	35 FA + 7 HA + 16 FA	493	64	42 levels
Dadda tree + Kogge–Stone CPA (dadda8_ks.sv)	35 FA + 7 HA + KS16	723	64	38 levels
— of which: Dadda tree alone	35 FA + 7 HA	—	—	24 levels (model)

A.6 RISC-V M-extension semantics

The M extension adds `MUL`, which returns the low XLEN bits of the product (XLEN is the register width, 32 for RV32), and `MULH`, `MULHSU` and `MULHU`, which return the high XLEN bits for signed×signed, signed×unsigned and unsigned×unsigned operands [13]. The low half is identical for signed and unsigned operands, which is why one `MUL` suffices. The specification recommends that software needing both halves request `MULH[[S]U]` immediately before `MUL` with the same operands, so that an implementation can fuse the two into one multiplication. In the datapath of Chapter 7, a combinational multiplier would enter as another ALU operation. A sequential one requires the processor to stall Execute until its done signal rises, which is the same stall mechanism as in Section 8.5, driven by a different condition.

APPENDIX B

Division

B.1 Restoring division

Division has no parallel-prefix shortcut, because each quotient bit depends on the remainder left by the previous one. The restoring algorithm is pencil-and-paper division in base 2. With the partial remainder r initially 0, each of the n steps does the following:

1. shift the next dividend bit into r , giving $r' = 2r + a_k$;
2. trial-subtract, $t = r' - d$;
3. if $t \geq 0$, keep it as the new remainder and set the quotient bit to 1; otherwise *restore* r' and set the quotient bit to 0.

The invariant after step k is

$$\lfloor a/2^{n-k} \rfloor = d \cdot q_{(k)} + r_{(k)}, \quad 0 \leq r_{(k)} < d,$$

so after n steps q and r are the true quotient and remainder. Because $r < d \leq 2^n - 1$, the shifted remainder needs $n + 1$ bits, and the trial difference one more for its sign. Listing B.1 and Figure B.1 show the implementation. The dividend's register doubles as the quotient register: bits leave at the top as quotient bits enter at the bottom.

Listing B.1. One restoring iteration and the sequential divider.

arith.sv

```

// One restoring-division iteration: shift in a dividend bit, trial-subtract,
// keep the difference only if it is non-negative.
module div_step #(parameter int W = 4) (
    input  logic [W:0]  rem,
    input  logic [W-1:0] q, dv,
    output logic [W:0]  rem_n,
    output logic [W-1:0] q_n);
    logic [W:0]  sh;
    logic [W+1:0] trial;
    logic        neg;
    assign sh    = {rem[W-1:0], q[W-1]};
    assign trial = {1'b0, sh} - {2'b00, dv};
    assign neg   = trial[W+1];
    assign rem_n = neg ? sh : trial[W:0];          // restore on negative
    assign q_n   = {q[W-2:0], ~neg};
endmodule

// Restoring division (unsigned). Divide-by-zero yields q = all ones, r = a,
// which is exactly the RISC-V M-extension convention for DIVU/REMU.
module div_restoring #(parameter int W = 8) (
    input  logic        clk, rst, start,
    input  logic [W-1:0] a, d,
    output logic [W-1:0] q, r,
    output logic        done);
    logic [W:0]  rem, rem_n;
    logic [W-1:0] dv, q_n;
    logic [$clog2(W+1)-1:0] cnt;
    div_step #(W) step (.rem, .q, .dv, .rem_n, .q_n);
    always_ff @(posedge clk)
        if (rst) begin cnt ≤ '0; done ≤ 1'b0; end
        else if (start) begin rem ≤ '0; q ≤ a; dv ≤ d; cnt ≤ W; done ≤ 1'b0; end
        else if (cnt ≠ 0) begin
            rem ≤ rem_n;
            q    ≤ q_n;
            cnt ≤ cnt - 1'b1;
            done ≤ (cnt = 1);
        end
    end
    assign r = rem[W-1:0];
endmodule

```

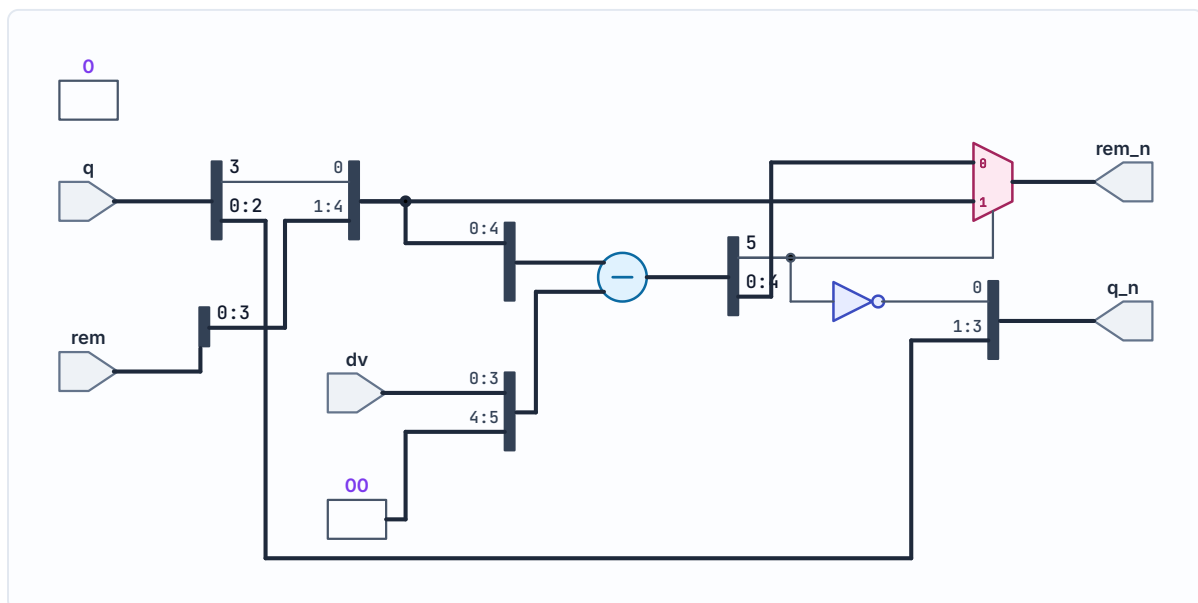


Figure B.1. One restoring iteration at $W = 4$: the shift join, the trial subtractor, and a multiplexer that restores on a negative result. The complement of the sign bit becomes the new quotient bit.

YOSYS → NETLISTSVG

DIVISION BY ZERO FOR FREE

With $d = 0$ every trial subtraction succeeds, so every quotient bit is 1 and the remainder accumulates the whole dividend. The unsigned divider therefore returns $q = 2^n - 1$ and $r = a$ without any special-case logic. This is exactly what the RISC-V M extension specifies for DIVU and REMU [13], and the testbench checks it for all 256 dividends on both dividers.

B.2 Non-restoring division

Restoring wastes work: after a failed trial it adds d back, and the next step then shifts and subtracts again. Merging the two operations gives

$$2(r') - d \quad \text{with} \quad r' = t + d \Rightarrow 2t + d.$$

The non-restoring rule is therefore: keep the possibly negative remainder, and in the next step *add* d if the remainder is negative and *subtract* it otherwise. The quotient bit is 1 whenever the new remainder is non-negative. Every step now costs exactly one addition or subtraction, with no multiplexer on the remainder. After the last step, a single correction adds d back if the remainder is negative. Listing B.2 implements this with one extra cycle for the correction, and it was checked exhaustively alongside the restoring divider.

Listing B.2. Non-restoring divider with final remainder correction.

arith.sv

```
// Non-restoring division (unsigned): add or subtract every step, never restore;
// one final correction of the remainder.
module div_nonrestoring #(parameter int W = 8) (
  input logic clk, rst, start,
  input logic [W-1:0] a, d,
  output logic [W-1:0] q, r,
  output logic done);
  logic signed [W+1:0] rem, sh, nxt, dv;
  logic [W-1:0] qq;
  logic [$clog2(W+1)-1:0] cnt;
  logic fix;
  assign sh = {rem[W:0], qq[W-1]};
  assign nxt = rem[W+1] ? sh + dv : sh - dv; // sign of previous remainder decides
  always_ff @(posedge clk)
    if (rst) begin cnt ≤ '0; done ≤ 1'b0; fix ≤ 1'b0; end
    else if (start) begin rem ≤ '0; qq ≤ a; dv ≤ {2'b00, d}; cnt ≤ W;
      done ≤ 1'b0; fix ≤ 1'b0; end
    else if (cnt ≠ 0) begin
      rem ≤ nxt;
      qq ≤ {qq[W-2:0], ~nxt[W+1]};
      cnt ≤ cnt - 1'b1;
      fix ≤ (cnt = 1);
    end
    else if (fix) begin
      if (rem[W+1]) rem ≤ rem + dv; // final remainder correction
      fix ≤ 1'b0; done ≤ 1'b1;
    end
  assign q = qq;
  assign r = rem[W-1:0];
endmodule
```

B.3 SRT division

Both algorithms above spend a full-width carry-propagate addition on every quotient bit, just to learn the remainder's sign. SRT division, named for Sweeney, Robertson [20] and Tocher [21], removes that cost with a *redundant* quotient digit set, for example $\{-1, 0, +1\}$ or $\{-2, \dots, +2\}$ in radix 4. Redundancy means an approximate choice can be corrected in later steps. The next digit can then be chosen from a few leading bits of the remainder and divisor, and the remainder itself can be held in carry-save form, so no carry

propagates until the end. The digit-selection function is a small lookup table. Its correctness is subtle, and history records the cost of getting it wrong: the 1994 Intel Pentium FDIV flaw came from missing entries in the lookup table of a radix-4 SRT divider [22]. Ercegovac and Lang give the full theory [23].

B.4 RISC-V division semantics

The M extension defines `DIV`, `DIVU`, `REM` and `REMU`. Quotients round toward zero, and the remainder takes the sign of the dividend. Neither of the two exceptional cases in Table B.1 traps, so the hardware needs no exception path [13]:

Table B.1. RISC-V M-extension exceptional cases (XLEN = L).

Condition	DIVU	REMU	DIV	REM
divide by zero	$2^L - 1$	dividend	-1	dividend
overflow: $-2^{L-1} \div -1$	—	—	-2^{L-1}	0

A signed divider is usually built around an unsigned core. It divides the magnitudes, negates the quotient when the operand signs differ, and gives the remainder the dividend's sign. The overflow row then comes out right automatically. As an unsigned number, $|-2^{L-1}|$ is 2^{L-1} ; dividing it by 1 leaves it unchanged; both operands are negative, so no negation is applied; and the bit pattern 2^{L-1} reads as -2^{L-1} , with remainder 0. Division by zero does not come out right. For a negative dividend the operand signs differ, so the unsigned result $2^L - 1$ would be negated to +1 instead of the required -1. A compliant signed divider must therefore suppress the quotient negation when the divisor is zero. This signed wrapper is described but was not implemented or tested here.

APPENDIX C

Memory Addressing

Chapter 5 built a register; Chapter 7 used a register file and two memories without examining how a particular word is found. This appendix builds a complete addressable memory — sixteen 8-bit registers selected by a 4-bit address — and follows a value through its write path and read path. Every block is synthesized and drawn from its netlist, and the memory was verified against a reference model.

C.1 Addresses and capacity

A memory is an array of 2^n storage words addressed by an n -bit number. The memory here has $n = 4$ and word width $w = 8$:

$$N = 2^n = 16 \text{ words}, \quad \text{capacity} = N \cdot w = 128 \text{ bits}, \quad n = \lceil \log_2 N \rceil.$$

An address is only a name. The hardware turns it into a physical selection in two different ways, which is the central idea of this appendix. For writing, the address is *decoded* into sixteen one-hot enable lines, so that exactly one register loads. For reading, the address *steers a multiplexer* that routes one register's contents to the output. Listing C.1 is the complete memory; Figure C.1 shows its structure as a 4×4 grid.

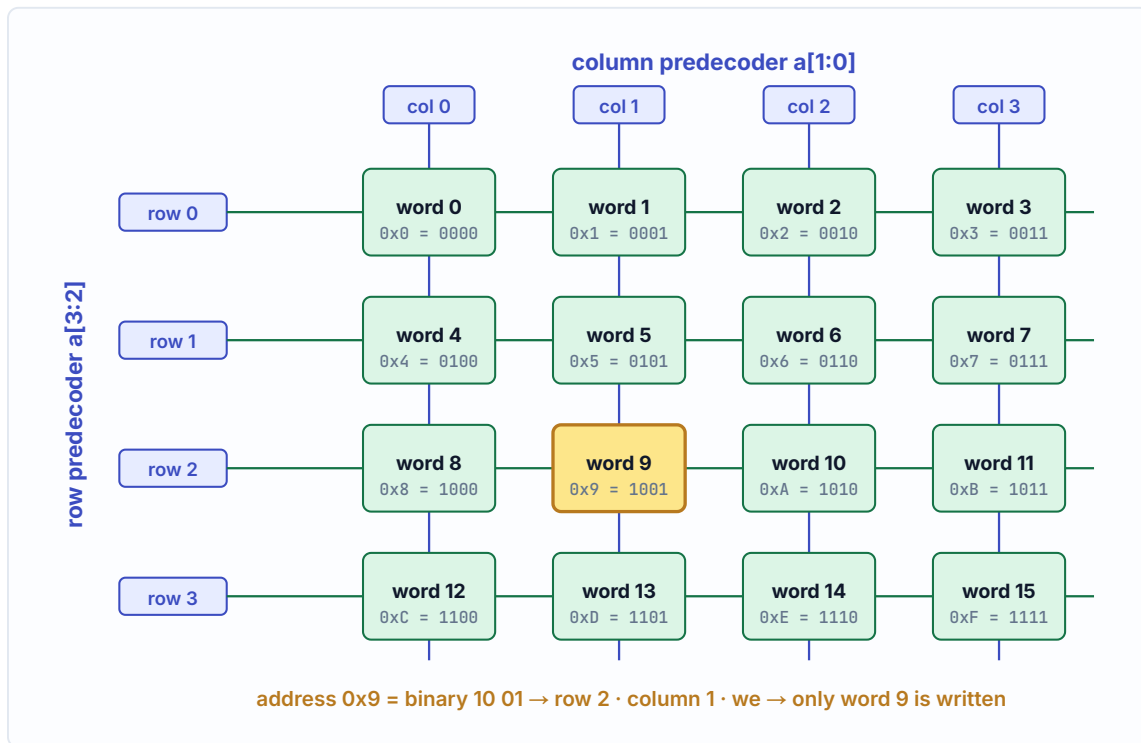


Figure C.1. Coincident selection. The 4-bit address is split into a row index $a[3:2]$ and a column index $a[1:0]$; a word is written only where its row line, its column line and the write enable are all active. The highlighted word is the one selected by address 0x9.

C.2 The write path: decoding

A 2-to-4 decoder produces four outputs, exactly one of which is high when enabled. Each output is the AND of the enable with the address bits or their complements (Figure C.2):

$$y_0 = en \overline{a_1} \overline{a_0}, \quad y_1 = en \overline{a_1} a_0, \quad y_2 = en a_1 \overline{a_0}, \quad y_3 = en a_1 a_0.$$

A 4-to-16 decoder could be built the same way, with sixteen 5-input ANDs. Instead, it is built from two 2-to-4 *predecoders*, one for the row bits and one for the column bits, followed by sixteen 2-input ANDs, $y_{4r+c} = row_r \cdot col_c$ (Figure C.4). The write enable is folded into the row predecoder, so an idle memory activates no word at all. Predecoding keeps every gate's fan-in small and gives the regular row/column wiring of Figure C.1; real memories use the same split, with rows becoming word lines. As synthesized, the whole decoder is 26 two-input AND cells and 4 inverters.

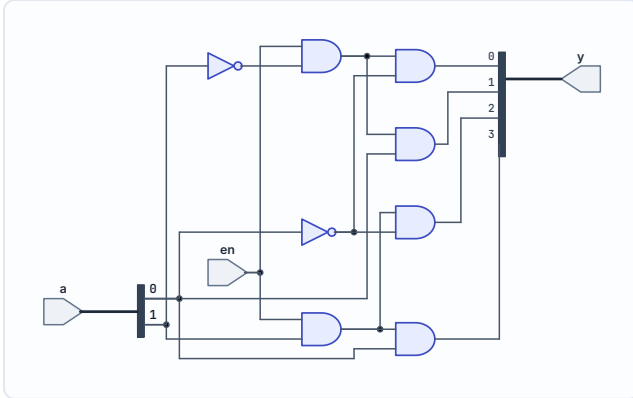


Figure C.2. 2-to-4 decoder with enable: two inverters and a shared AND term per half. YOSYS → NETLISTSVG

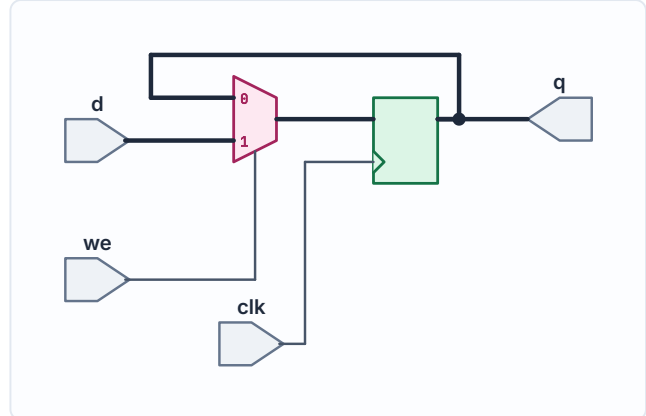


Figure C.3. One storage word: an 8-bit D flip-flop with a load-enable multiplexer (Section 5.5). YOSYS → NETLISTSVG

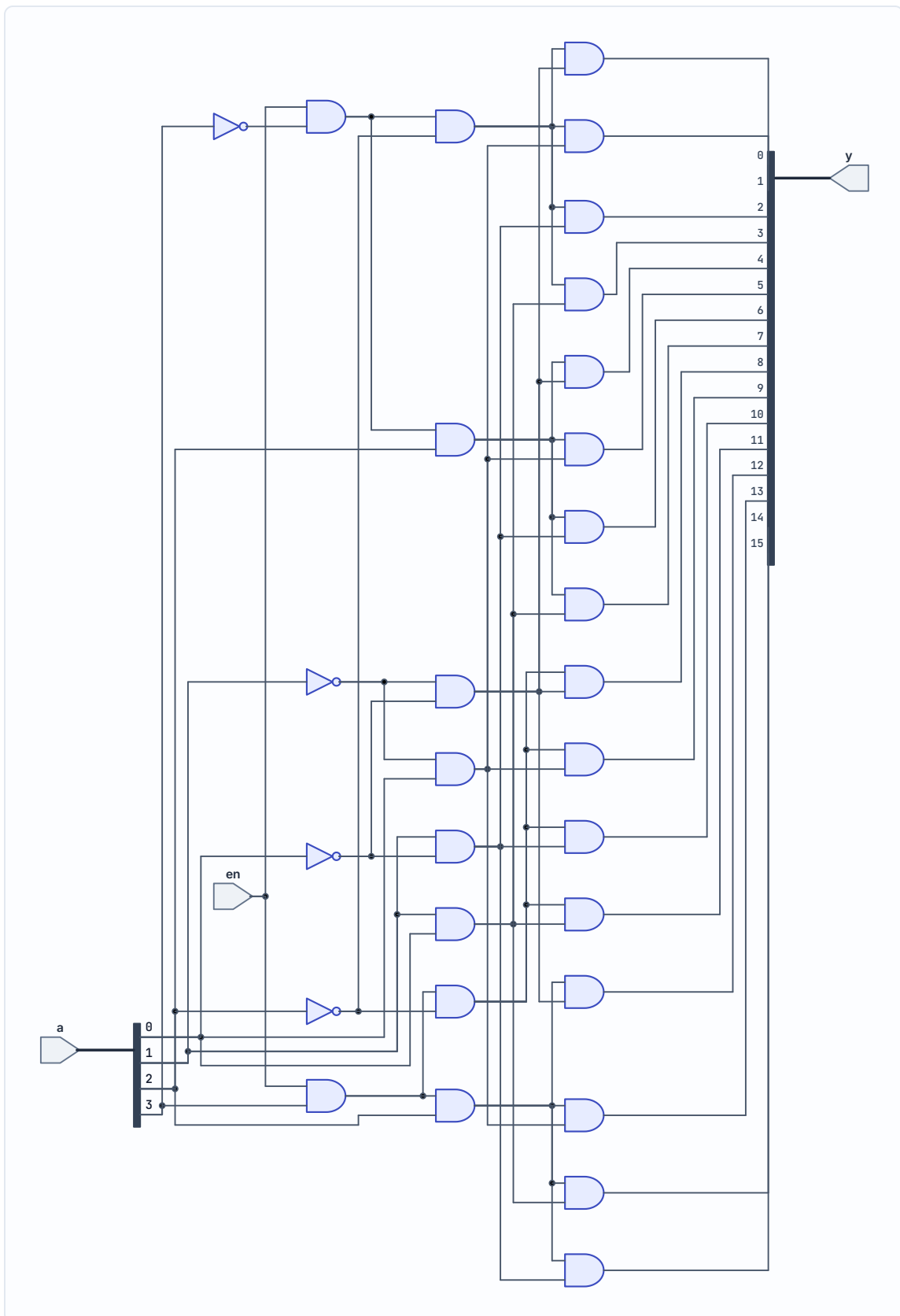


Figure C.4. 4-to-16 decoder after flattening: the row predecoder (with enable) and column predecoder on the left, sixteen row-column ANDs on the right feeding the one-hot output bus.

YOSYS → NETLISTSVG

C.3 The storage words

Each word is the enabled register of Section 5.5, eight flip-flops sharing one clock and one load enable (Figure C.3). All sixteen words receive the same write data `wd`; only the decoder decides which one captures it on the rising clock edge. That shared data bus is why decoding must be one-hot: two enables active at once would copy the value into two words.

C.4 The read path: multiplexing

Reading needs no clock. A 16:1 multiplexer selects one of the sixteen 8-bit outputs, and it is built as a balanced tree of fifteen 2:1 multiplexers in four levels. Each level consumes one address bit, starting with the least significant (Figure C.5). The tree's depth is $\log_2 16 = 4$ multiplexer levels. The alternatives are the one-hot AND-OR form, which uses a second decoder on the read address and a wide OR, and the tri-state bus of Section 2.5. Both trade the tree's depth for fan-in or electrical loading.

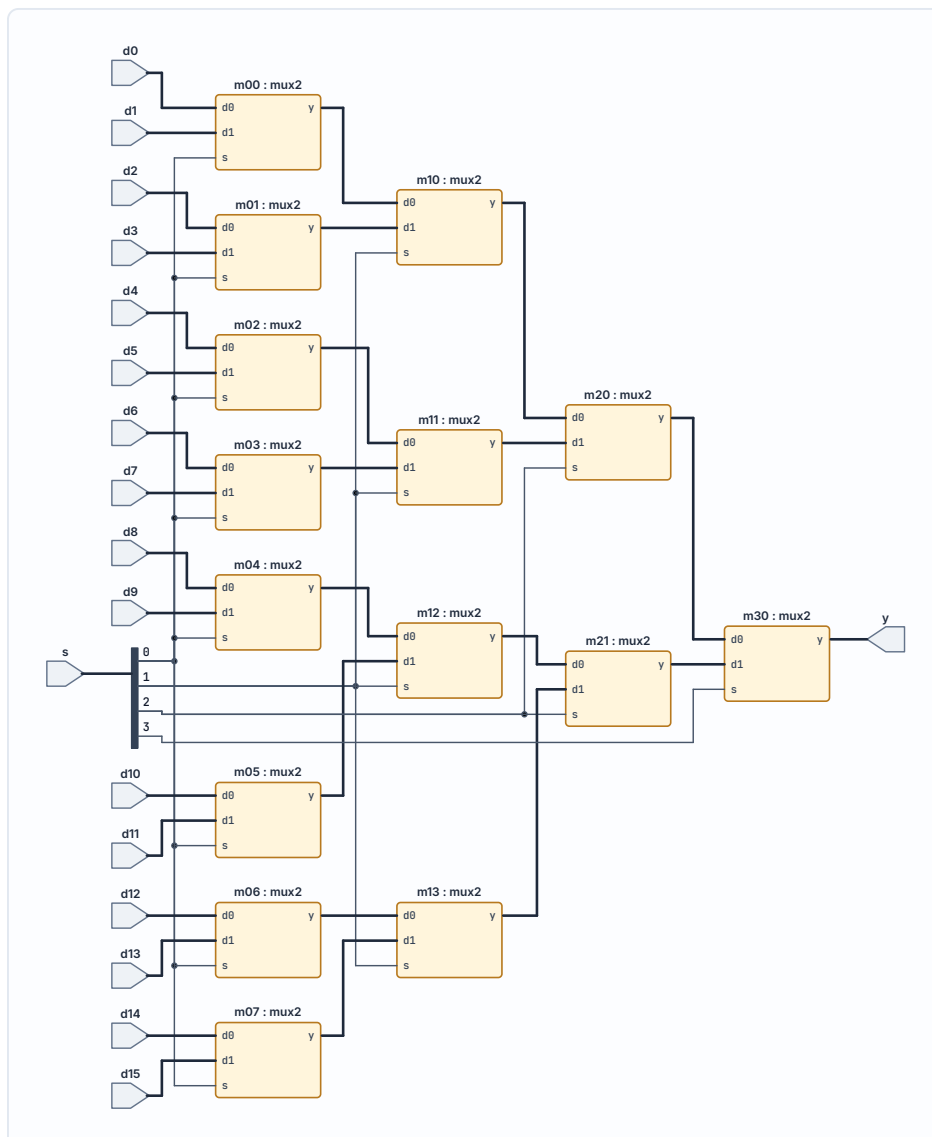


Figure C.5. The 16:1 read multiplexer as a four-level tree of 8-bit 2:1 multiplexers (*m00–m07* use *ra[0]*, *m10–m13* *ra[1]*, *m20–m21* *ra[2]*, *m30* *ra[3]*).

YOSYS → NETLISTSVG

C.5 The complete memory

Figure C.6 is the synthesized top level: decoder, sixteen registers and the read tree, with the decoder's 16-bit enable bus fanned out to the registers' `we` inputs and every register output routed to the multiplexer. Writing is synchronous and reading is asynchronous. The write address `wa` and the read address `ra` are independent, so a word can be read while another is written.

Listing C.1. The 4-to-16 decoder and the 16×8 -bit memory (`dec2to4`, `reg8` and `mux16x8` are in the same file and appear in Figures C.2, C.3 and C.5).

mem16.sv

```
// 4-to-16 decoder built from two predecoders: row = a[3:2], column = a[1:0].
// y[4r+c] = row[r] & col[c] - the same "coincident selection" as a 4x4 grid.
module dec4to16 (input logic [3:0] a, input logic en, output logic [15:0] y);
    logic [3:0] row, col;
    dec2to4 prow (.a(a[3:2]), .en(en), .y(row));
    dec2to4 pcol (.a(a[1:0]), .en(1'b1), .y(col));
    for (genvar r = 0; r < 4; r++) begin : g_r
        for (genvar c = 0; c < 4; c++) begin : g_c
            assign y[4*r+c] = row[r] & col[c];
        end
    end
endmodule

// The memory: synchronous write, asynchronous (combinational) read
module mem16x8 (
    input logic clk, we,
    input logic [3:0] wa, ra,
    input logic [7:0] wd,
    output logic [7:0] rd);
    logic [15:0] wen;
    logic [7:0] q0, q1, q2, q3, q4, q5, q6, q7, q8, q9, q10, q11, q12, q13, q14, q15;
    dec4to16 dec (.a(wa), .en(we), .y(wen));
    reg8 r00 (.clk, .we(wen[0]), .d(wd), .q(q0));
    reg8 r01 (.clk, .we(wen[1]), .d(wd), .q(q1));
    reg8 r02 (.clk, .we(wen[2]), .d(wd), .q(q2));
    reg8 r03 (.clk, .we(wen[3]), .d(wd), .q(q3));
    reg8 r04 (.clk, .we(wen[4]), .d(wd), .q(q4));
    reg8 r05 (.clk, .we(wen[5]), .d(wd), .q(q5));
    reg8 r06 (.clk, .we(wen[6]), .d(wd), .q(q6));
    reg8 r07 (.clk, .we(wen[7]), .d(wd), .q(q7));
    reg8 r08 (.clk, .we(wen[8]), .d(wd), .q(q8));
    reg8 r09 (.clk, .we(wen[9]), .d(wd), .q(q9));
    reg8 r10 (.clk, .we(wen[10]), .d(wd), .q(q10));
    reg8 r11 (.clk, .we(wen[11]), .d(wd), .q(q11));
    reg8 r12 (.clk, .we(wen[12]), .d(wd), .q(q12));
    reg8 r13 (.clk, .we(wen[13]), .d(wd), .q(q13));
    reg8 r14 (.clk, .we(wen[14]), .d(wd), .q(q14));
    reg8 r15 (.clk, .we(wen[15]), .d(wd), .q(q15));
    mux16x8 rmux (.d0(q0), .d1(q1), .d2(q2), .d3(q3), .d4(q4), .d5(q5), .d6(q6), .d7(q7),
        .d8(q8), .d9(q9), .d10(q10), .d11(q11), .d12(q12), .d13(q13), .d14(q14),
        .d15(q15), .s(ra), .y(rd));
endmodule
```

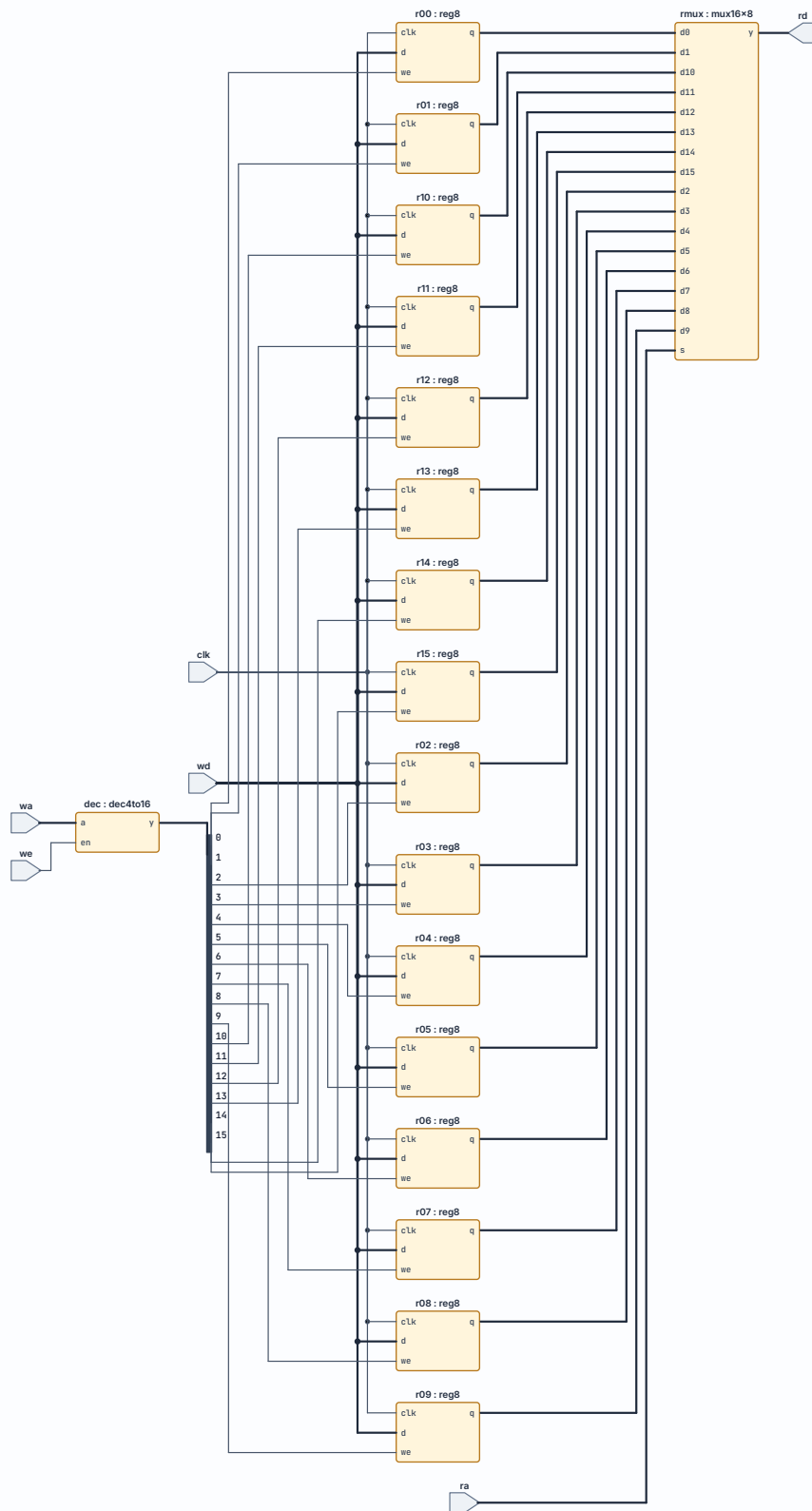


Figure C.6. The complete 16-word $\times$ 8-bit memory. Left: the 4-to-16 decoder, whose one-hot bus fans out to the *we* pins of registers r00–r15; right: the read multiplexer tree. Instances are kept as black boxes.

YOSYS $\rightarrow$ NETLISTSVG

C.6 Timing of a write and a read

Figure C.7 is taken directly from simulation. In the first highlighted cycle, `we` is high with `wa = 3` and `wd = 3C`. At the rising edge word 3 captures `3C`, and because `ra = 3`, the output `rd` changes to the new value just after the edge. A later cycle writes `96` into word 9, which appears on `rd` once `ra` is switched to 9. The last write attempt presents `69` with `we = 0`, and word 9 keeps `96`. The single-cycle and multicycle register files behave the same way. The pipeline's register file writes on the falling edge instead, so its new value appears half a cycle earlier (Section 8.4).

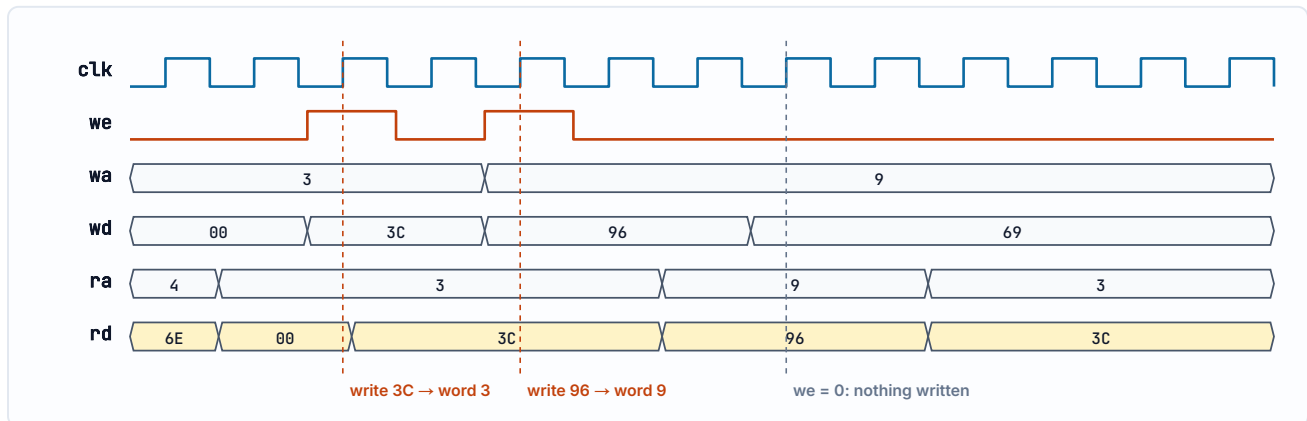


Figure C.7. Simulated write and read sequence of the 16×8 memory (Icarus Verilog trace, 1 ns sampling, 10 ns clock). Values are hexadecimal.

C.7 Verification

The memory was checked against a 16-entry reference array in four phases, totalling 22 320 read comparisons with no failures:

1. **Unique fill.** Every word receives a distinct value and all sixteen are read back (16 checks).
2. **Walking ones.** Each of the 128 bit positions is written in turn, and the whole memory is read after every write (2 048 checks). A decoder that enabled two words, or a stuck multiplexer select, would corrupt a second word and be caught.
3. **Write disable.** A write is attempted at every address with `we = 0` (256 checks).
4. **Random traffic.** 20 000 iterations of random writes, some with `we = 0`, interleaved with random reads.

C.8 From registers to real memory

The flip-flop memory above costs roughly two dozen transistors per bit. Dense memories keep the same addressing structure but replace the storage and read circuits:

- **SRAM.** Each bit is a six-transistor cross-coupled cell. The decoder's row outputs become *word lines* that connect a whole row of cells to vertical *bit lines*. A column multiplexer and a sense amplifier then detect the small voltage difference on the selected bit lines [8].
- **DRAM.** Each bit is one transistor and one capacitor. It is denser still, but reading is destructive and the charge leaks, so the memory must be periodically refreshed.
- **Memory maps.** In a computer, address decoding also happens one level up. The address space is partitioned between memories and peripherals, and a decoder on the upper address bits selects which device responds. Peripherals mapped this way are *memory-mapped I/O*.
- **Byte addressing.** RISC-V addresses bytes, so 32-bit words sit at multiples of four. The data memory of Chapter 7 therefore indexes its array with `a[31:2]`: the test program's address 100 is word 25.

C.9 Caches

A processor fetches from memory on every instruction, so memory speed bounds processor speed. Large memories are slow and fast memories are small. A *cache* is a small, fast memory that holds copies of recently used words, and it works because programs exhibit *locality*: they tend to reuse the same addresses (temporal locality) and their neighbours (spatial locality) [1, 6].

The simplest organization is *direct-mapped* (Figure C.8). The address is split into an *index*, which selects exactly one cache line, and a *tag*, which is stored in that line and compared on every access. A read is a *hit* when the line is valid and its tag matches. Otherwise it is a *miss*: the word is read from the backing memory and copied into the line, evicting whatever was there. Listing C.2 puts a 4-line cache in front of the 16-word memory of this appendix, using the two low address bits as the index and the two high bits as the tag. Writes go to the backing memory and are copied into the cache.

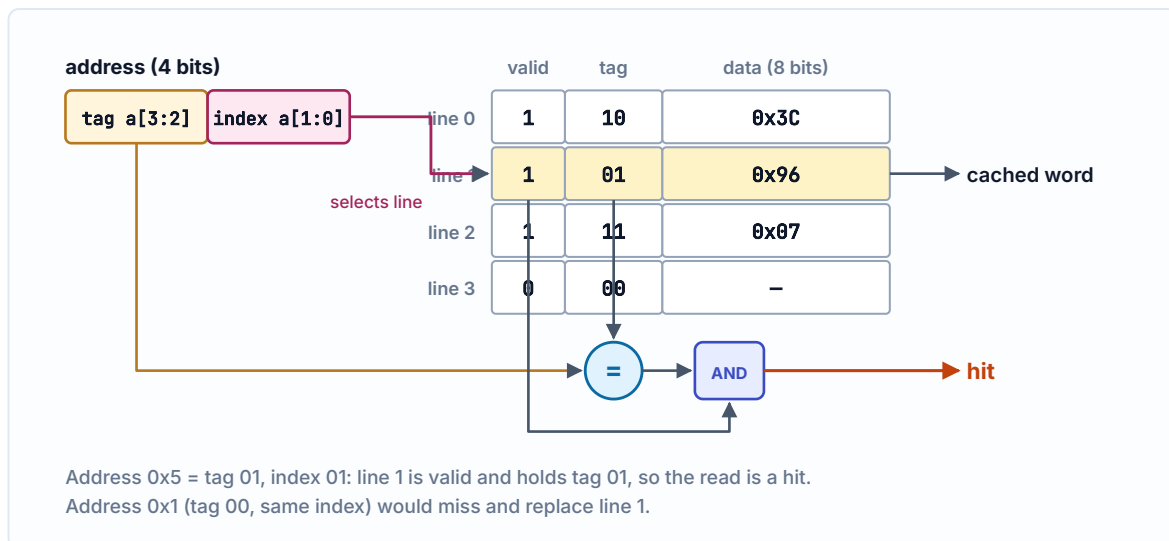


Figure C.8. Direct-mapped cache organization: the index selects one line; a hit requires the valid bit and a tag match.

Listing C.2. Four-line direct-mapped cache with hit/miss counters in front of the 16×8 memory.

cache.sv

```

module dm_cache (
    input logic      clk, rst,
    input logic      re,           // read request (one per clock cycle)
    input logic [3:0] ra,
    output logic [7:0] rdata,
    output logic      hit,
    input logic      we,
    input logic [3:0] wa,
    input logic [7:0] wd,
    output logic [15:0] hits, misses);
    logic [3:0] valid;
    logic [1:0] tag [4];
    logic [7:0] data [4];
    logic [7:0] mem_rd;
    logic [1:0] ri, wi;
    assign ri = ra[1:0];
    assign wi = wa[1:0];
    mem16x8 backing (.clk, .we, .wa, .ra, .wd, .rd(mem_rd));
    assign hit = valid[ri] && (tag[ri] == ra[3:2]);
    assign rdata = hit ? data[ri] : mem_rd;
    always_ff @(posedge clk) begin
        if (rst) begin
            valid <= '0; hits <= '0; misses <= '0;
        end else begin
            if (re) begin
                if (hit) hits <= hits + 1'b1;
                else begin
                    misses <= misses + 1'b1;
                    valid[ri] <= 1'b1; tag[ri] <= ra[3:2]; data[ri] <= mem_rd; // load on miss
                end
            end
            if (we) begin // write-through
                valid[wi] <= 1'b1; tag[wi] <= wa[3:2]; data[wi] <= wd;
            end
        end
    end
endmodule

```

The cache RTL was driven with five address traces, and its hit and miss counters were compared with an independent Python model of the same policy (`fig/cachesim.py`). The counts agreed exactly on every trace, and all 497 data reads returned the correct value (Table C.1). The traces were chosen to isolate the mechanisms:

- **Sequential sweep.** A sweep over all sixteen addresses never hits: each address evicts the previous occupant of its line, the next access to that line wants a different tag, and so the whole trace consists of *conflict misses*.
- **Ping-pong.** Alternating between two addresses that share an index (1 and 5) thrashes completely. Two addresses on different lines (1 and 6) miss only on their first touch.
- **Tight loop.** A loop over three words misses three times and then always hits.

Table C.1. Measured cache behaviour (RTL counters, identical to the Python model).

Trace	Hits	Misses	Hit rate
sequential sweep $\times 4$	0	64	0 %
loop over 3 words	57	3	95 %
ping-pong 1 $\leftrightarrow$ 5 (same index)	0	40	0 %
ping-pong 1 $\leftrightarrow$ 6 (different index)	38	2	95 %
uniform random (25% writes)	72	221	25 %

A real workload shows the same mechanisms. The test program's 60 instruction fetches touch 25 distinct words, so at least 25 fetches must miss in any cache that starts empty. Table C.2 replays that fetch trace, taken from the reference simulator, through direct-mapped caches of increasing size. With one or two lines, every fetch misses, because the four-word loop body keeps evicting itself. From four lines upward, every repeated fetch hits and only the 25 unavoidable first-touch (*cold*) misses remain.

Table C.2. Instruction-fetch trace of the test program through direct-mapped caches.

Cache lines	Hits	Misses	Miss rate
1	0	60	100 %
2	0	60	100 %
4	35	25	42 %
8	35	25	42 %
16	35	25	42 %
32	35	25	42 %
64	35	25	42 %

The payoff is summarized by the average memory access time,

$$\text{AMAT} = t_{\text{cache}} + m \cdot t_{\text{miss penalty}},$$

where m is the miss rate. Misses fall into three classes: *cold* (first touch), *conflict* (two addresses competing for one line) and *capacity* (the cache is simply too small). Set-associative caches let each address use any of several lines, which removes most conflict misses at the price of several parallel tag comparators. Multi-level caches stack progressively larger and slower memories [6].

APPENDIX D

Microcode and Hardwired Control

A processor's control unit turns an instruction into the control signals that steer the datapath. There are two classical ways to build one. In *hardwired* control, the signals are produced by logic gates — combinational logic or a finite-state machine — synthesized from the specification. In *microprogrammed* control, the signals are stored as words in a read-only memory (ROM), the *microcode*, and read out in sequence by a small internal computer. This appendix builds the same control unit both ways (Figure D.1) and compares them.

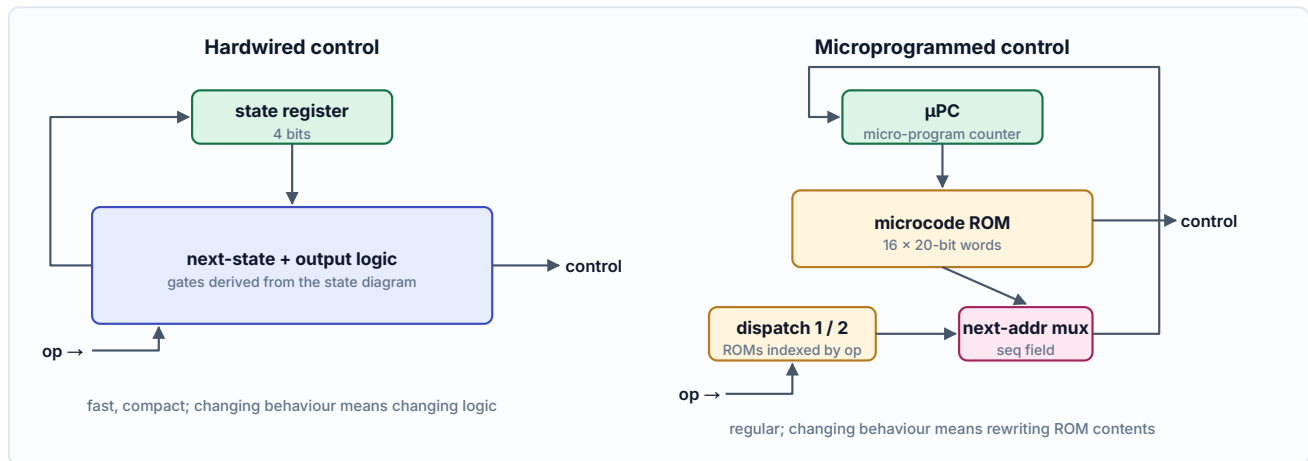


Figure D.1. The two ways to generate control signals. Left: a state register and gates. Right: a micro-program counter (μ PC) addresses a microcode ROM whose words hold both the control signals and the sequencing information; dispatch ROMs map the opcode to the first microinstruction of each instruction's routine.

D.1 Why sequencing is needed

The single-cycle controller of Chapter 7 is hardwired and purely combinational: one instruction, one clock cycle, one control word. A *multicycle* processor instead reuses one adder and one memory across several shorter cycles, so each instruction needs a *sequence* of control words. Figure D.2 shows the eleven-state sequence used here. It follows the structure of the Harris & Harris RISC-V multicycle controller [2]. Every instruction shares **Fetch** (read the instruction and compute $PC + 4$) and **Decode** (read registers and compute the branch target). Decode then branches on the opcode, and the memory instructions branch once more on whether they read or write.

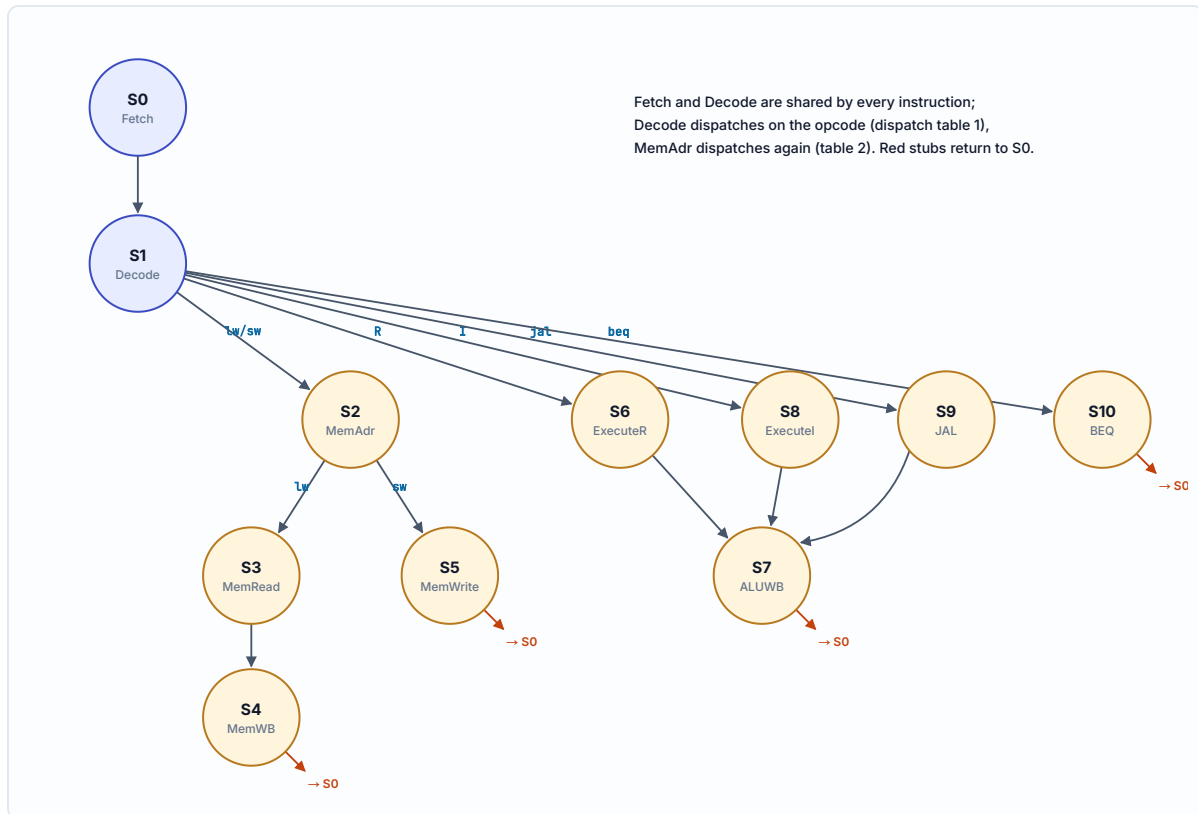


Figure D.2. Control sequence of the multicycle controller: 11 states, two opcode-dependent branch points (dispatches).

Sections D.2–D.5 treat the control unit on its own, comparing two implementations that must produce identical signal sequences. Section D.6 then completes the multicycle datapath it drives.

D.2 The hardwired implementation

The hardwired controller (Listing D.1) is a textbook finite-state machine: a 4-bit state register, next-state logic written as a `case` on the state and opcode, and output logic written as a `case` on the state alone, which makes it a Moore machine. Synthesis turns both `case` statements into gates. Changing the behaviour, for example adding an instruction, means editing and re-synthesizing that logic.

Listing D.1. Hardwired multicycle controller: combinational core and state register.

microcode.sv

```
// ----- (a) hardwired: next-state and output logic as gates
module ctl_hw_core (
  input  logic [3:0] state,
  input  logic [6:0] op,
  output logic [3:0] nxt,
  output logic [13:0] ctrl);
always_comb begin
  case (state)
    4'd0: nxt = 4'd1;
    4'd1: case (op)
      7'b0000011, 7'b0100011: nxt = 4'd2; // lw, sw
      7'b0110011:           nxt = 4'd6; // R-type
      7'b0010011:           nxt = 4'd8; // I-type ALU
      7'b1101111:           nxt = 4'd9; // jal
      7'b1100011:           nxt = 4'd10; // beq
      default:               nxt = 4'd0; // unimplemented: refetch
    endcase
    4'd2: nxt = (op == 7'b0000011) ? 4'd3 : (op == 7'b0100011) ? 4'd5 : 4'd0;
    4'd3: nxt = 4'd4;
    4'd6, 4'd8, 4'd9: nxt = 4'd7;
    default: nxt = 4'd0; // 4, 5, 7, 10 and unused codes
  endcase
//                               Adr IR SrcA SrcB ALUOp Res PCU RW MW Br
```

```

    case (state)
      4'd0: ctrl = {1'b0,1'b1,2'b00,2'b10,2'b00,2'b10,1'b1,1'b0,1'b0,1'b0};
      4'd1: ctrl = {1'b0,1'b0,2'b01,2'b01,2'b00,2'b00,1'b0,1'b0,1'b0,1'b0};
      4'd2: ctrl = {1'b0,1'b0,2'b10,2'b01,2'b00,2'b00,1'b0,1'b0,1'b0,1'b0};
      4'd3: ctrl = {1'b1,1'b0,2'b00,2'b00,2'b00,2'b00,1'b0,1'b0,1'b0,1'b0};
      4'd4: ctrl = {1'b0,1'b0,2'b00,2'b00,2'b00,2'b01,1'b0,1'b1,1'b0,1'b0};
      4'd5: ctrl = {1'b1,1'b0,2'b00,2'b00,2'b00,2'b00,1'b0,1'b0,1'b1,1'b0};
      4'd6: ctrl = {1'b0,1'b0,2'b10,2'b00,2'b10,2'b00,1'b0,1'b0,1'b0,1'b0};
      4'd7: ctrl = {1'b0,1'b0,2'b00,2'b00,2'b00,2'b00,1'b0,1'b1,1'b0,1'b0};
      4'd8: ctrl = {1'b0,1'b0,2'b10,2'b01,2'b10,2'b00,1'b0,1'b0,1'b0,1'b0};
      4'd9: ctrl = {1'b0,1'b0,2'b01,2'b10,2'b00,2'b00,1'b1,1'b0,1'b0,1'b0};
      4'd10: ctrl = {1'b0,1'b0,2'b10,2'b00,2'b01,2'b00,1'b0,1'b0,1'b0,1'b1};
      default: ctrl = '0;
    endcase
  end
endmodule

module ctl_hardwired (
  input logic clk, rst,
  input logic [6:0] op,
  output logic [3:0] state,
  output logic [13:0] ctrl);
  logic [3:0] nxt;
  ctl_hw_core core (.state, .op, .nxt, .ctrl);
  always_ff @(posedge clk) state ≤ rst ? 4'd0 : nxt;
endmodule

```

D.3 The microprogrammed implementation

The microprogrammed controller replaces the logic with three memories and a multiplexer (Listing D.2, Figure D.3). Its *microinstruction* has three fields:

- **Control** (14 bits). One bit or bit-field per datapath signal, emitted unchanged. Encoding one bit per signal is called *horizontal* microcode; *vertical* microcode packs signals into encoded fields that need decoding, trading ROM width for extra logic.
- **Sequencing** (2 bits). This selects the next microinstruction: `00` jumps to the address in the next field, `01` dispatches through table 1, and `10` dispatches through table 2.
- **Next address** (4 bits). The target used for a jump.

The dispatch tables are 128-entry ROMs indexed by the opcode, implementing the two branch points of Figure D.2. Unimplemented opcodes dispatch to Fetch, matching the hardwired controller's `default` arm. The microcode itself is written symbolically and assembled into hexadecimal by a 40-line Python *micro-assembler*. Table D.1 is its output, and the ROMs are loaded with `$readmemh` at elaboration.

Table D.1. The microprogram as assembled by `fig/microasm.py`. Each row is one 20-bit microinstruction: 14 control bits, 2 sequencing bits and a 4-bit next address. Column abbreviations: Adr = AdrSrc, IRW = IRWrite, Res = ResultSrc, PCU = PCUpdate, RW = RegWrite, MW = MemWrite, Br = Branch.

μ PC	label	Adr	IRW	SrcA	SrcB	ALUOp	Res	PCU	RW	MW	Br	seq	next	hex
0	Fetch	0	1	00	10	00	10	1	0	0	0	jump	Decode	48A01
1	Decode	0	0	01	01	00	00	0	0	0	0	disp 1	—	14010
2	MemAdr	0	0	10	01	00	00	0	0	0	0	disp 2	—	24020
3	MemRead	1	0	00	00	00	00	0	0	0	0	jump	MemWB	80004
4	MemWB	0	0	00	00	00	01	0	1	0	0	jump	Fetch	00500
5	MemWrite	1	0	00	00	00	00	0	0	1	0	jump	Fetch	80080
6	ExecuteR	0	0	10	00	10	00	0	0	0	0	jump	ALUWB	22007
7	ALUWB	0	0	00	00	00	00	0	1	0	0	jump	Fetch	00100
8	ExecuteI	0	0	10	01	10	00	0	0	0	0	jump	ALUWB	26007
9	JAL	0	0	01	10	00	00	1	0	0	0	jump	ALUWB	18207
10	BEQ	0	0	10	00	01	00	0	0	0	1	jump	Fetch	21040

Listing D.2. Microprogrammed controller: microcode ROM, dispatch ROM, micro-sequencer.

microcode.sv

```
// ----- (b) microprogrammed: a micro-PC, a microcode ROM and two dispatch ROMs
// Microinstruction (20 bits) = { control word[13:0], seq[1:0], next[3:0] }
// seq 00: jump to next seq 01: dispatch via table 1 seq 10: dispatch via table 2
module urom (input logic [3:0] a, output logic [19:0] d);
    logic [19:0] mem [0:15];
    initial $readmemh("microcode.hex", mem);
    assign d = mem[a];
endmodule

module dispatch1 (input logic [6:0] op, output logic [3:0] addr);
    logic [3:0] mem [0:127];
    initial $readmemh("dispatch1.hex", mem);
    assign addr = mem[op];
endmodule

module ctl_mc_core (
    input logic [3:0] upc,
    input logic [6:0] op,
    output logic [3:0] unext,
    output logic [13:0] ctrl);
    logic [19:0] vinst;
    logic [3:0] d1, d2;
    urom rom (.a(upc), .d(vinst));
    dispatch1 dt1 (.op, .addr(d1));
    dispatch2 dt2 (.op, .addr(d2));
    assign ctrl = vinst[19:6];
    mux3 #(4) sel (.d0(vinst[3:0]), .d1(d1), .d2(d2), .s(vinst[5:4]), .y(unext));
endmodule

module ctl_microcoded (
    input logic clk, rst,
    input logic [6:0] op,
    output logic [3:0] upc,
    output logic [13:0] ctrl);
    logic [3:0] unext;
    ctl_mc_core core (.upc, .op, .unext, .ctrl);
    always_ff @(posedge clk) upc <= rst ? 4'd0 : unext;
endmodule
```

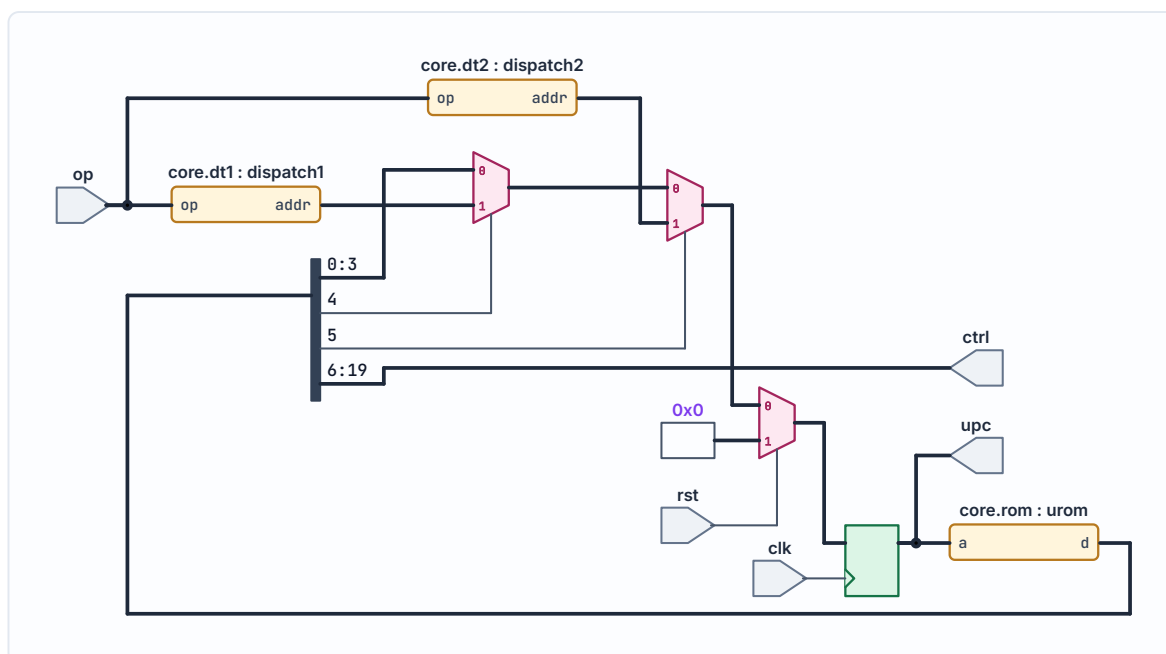


Figure D.3. The micro-sequencer after flattening, with the three ROMs as black boxes. The μ PC register (green) addresses the microcode ROM. Its 20-bit output is split into *ctrl* (6:19), the sequencing field (4:5) and the next address (0:3). Two 2:1 multiplexers implement the three-way next-address choice, and a third implements reset.

YOSYS → NETLISTSVG

D.4 Equivalence and cost

The two controllers were compared cycle by cycle. The first test holds each of the 128 possible opcodes for twelve cycles after reset. The second applies a random opcode on every one of 200 000 cycles, which drives both machines through every reachable state and transition. Across 201 536 comparisons of state and control word, there were no mismatches. The combinational cores alone were also compared exhaustively over all 11 states $\times$ 128 opcodes. As a negative control, flipping a single bit in one microinstruction (JAL's PCUpdate) produced 716 mismatches, so the test does detect microcode errors.

Table D.2. Synthesized size of the combinational cores (Yosys `synth -flatten`, then `abc -g NAND`; state register excluded).

Core	NAND	NOT	Total
Hardwired (ctl_hw_core)	64	53	117
Microprogrammed (ctl_mc_core)	64	49	113

Table D.2 makes the key point. When the microcode is fixed at synthesis time, a ROM is just a truth table, and the logic optimizer reduces both descriptions to essentially the same gates. At this scale, microprogramming saves no hardware. Its advantages are elsewhere:

- **Regularity.** Adding an instruction means writing a few microinstructions and a dispatch entry, not redesigning logic.
- **Changeability.** If the control store is a real, possibly writable, memory, behaviour can change after the hardware is built.
- **Scale.** The approach remains manageable for instruction sets with hundreds of complex, multi-step instructions.

Its costs are an extra memory access in every cycle and a control store that grows with the instruction set.

D.5 Hardcoded versus microcoded instructions in practice

Maurice Wilkes proposed microprogramming in 1951 as "the best way" to design a control unit, storing each instruction's elementary steps in a matrix rather than wiring them [24]. His EDSAC 2 used the idea, and by the 1960s it had become an established way to build control units. The IBM System/360 family is the best-known example: machines of very different speeds ran one instruction set, with the differences absorbed by microcode [25].

The RISC designs of the 1980s moved the other way. Their simple, fixed-format instructions — like the RV32I subset of this primer — decode easily in hardwired logic, and a hardwired decoder avoids the control-store access on every cycle [6]. Modern high-performance processors for complex instruction sets use both. Common instructions are decoded by hardwired logic into internal micro-operations, and the rare, complicated ones are expanded from a microcode ROM. Because that microcode can be patched by updates, errata can be corrected in the field. The division of labour is the one this appendix measured: hardwired where speed matters and the function is fixed, microcode where flexibility and complexity matter.

D.6 A complete multicycle processor

With the controller built and verified, the multicycle datapath it was designed for can be completed (Listing D.3, Figure D.4). The datapath follows the Harris & Harris multicycle design [2]. One ALU serves every arithmetic need: it computes $PC + 4$ in Fetch, the branch target in Decode, and the data result in Execute. Non-architectural registers hold values between cycles:

- **OldPC** and the instruction register, loaded together in Fetch;
- **A** and **WriteData**, the register-file outputs;
- **ALUOut**, the ALU result;
- **Data**, the memory read value.

The PC is written when **PCUpdate** is set, or when **Branch** is set and the ALU reports zero. One adaptation is needed: the instruction and data memories are kept separate, as in Chapters 7–8. The test program stores data at address 100, where its own halt instruction lives, so a shared memory would overwrite the code. **AdrSrc** therefore chooses which of the two memories the shared address addresses.

Listing D.3. The multicycle datapath; the controller is either implementation of Appendix D, selected by a parameter. `rv_multi.sv`

```
module rv_multi #(parameter bit MICRO = 1'b1) (
    input logic clk, rst,
    output logic [31:0] iadr, // instruction-memory address
    input logic [31:0] instr_rd,
    output logic [31:0] dadr, dwd, // data-memory address / write data
    output logic dwe,
    input logic [31:0] data_rd,
    output logic [3:0] state);

    logic [13:0] c;
    logic adrsrc, irwrite, pcupdate, regwrite, branch, zero, pcwrite;
    logic [1:0] alusrc, alusrcb, aluop, resultsrc, immsrc;
    logic [2:0] alucontrol;
    logic [31:0] pc, oldpc, instr, immext, rd1, rd2, a, wdata, srca, srcb,
        alureult, aluout, data, result;

    // ---- control unit (microprogrammed or hardwired, identical by Appendix D)
    if (MICRO) begin : g_mc
        ctl_microcoded cu (.clk, .rst, .op(instr[6:0]), .upc(state), .ctrl(c));
    end else begin : g_hw
        ctl_hardwired cu (.clk, .rst, .op(instr[6:0]), .state(state), .ctrl(c));
    end
    assign {adrsrc, irwrite, alusrc, alusrcb, aluop, resultsrc, pcupdate, regwrite, dwe, branch} = c;
    assign pcwrite = pcupdate | (branch & zero);
```

```

aludec ad (.aluop, .funct3(instr[14:12]), .funct7b5(instr[30]), .opb5(instr[5]), .alucontrol);
immdec id (.op(instr[6:0]), .immsrc);

// ---- architectural and non-architectural registers
always_ff @(posedge clk) begin
    if (rst) begin pc ≤ 32'd0; instr ≤ 32'h0000_0013; end // reset instr = nop (addi x0,x0,0)
    else begin
        if (pcwrite) pc ≤ result;
        if (irwrite) begin oldpc ≤ pc; instr ≤ instr_rd; end
    end
    a      ≤ rd1; // register-file outputs, captured every cycle
    wdata ≤ rd2;
    aluout ≤ aluresult;
    data  ≤ data_rd;
end

// ---- datapath
assign iadr = pc; // fetch address (AdrSrc = 0)
assign dadr = result; // data address (AdrSrc = 1): Result = ALUOut
assign dwd = wdata;
regfile #(.FALLING(1'b0)) rf (.clk, .we(regwrite), .a1(instr[19:15]), .a2(instr[24:20]), .a3(instr[11:7]),
    .wd(result), .rd1(rd1), .rd2(rd2));
extend ext (.instr(instr[31:7]), .immsrc, .immext);
mux3 #(32) msa (.d0(pc), .d1(oldpc), .d2(a), .s(alusrca), .y(srca));
mux3 #(32) msb (.d0(wdata), .d1(immext), .d2(32'd4), .s(alusrcb), .y(srcb));
alu #(32) alu0 (.a(srca), .b(srcb), .ctrl(alucontrol), .y(aluresult), .zero);
mux3 #(32) mres (.d0(aluout), .d1(data), .d2(aluresult), .s(resultsrc), .y(result));
endmodule

```

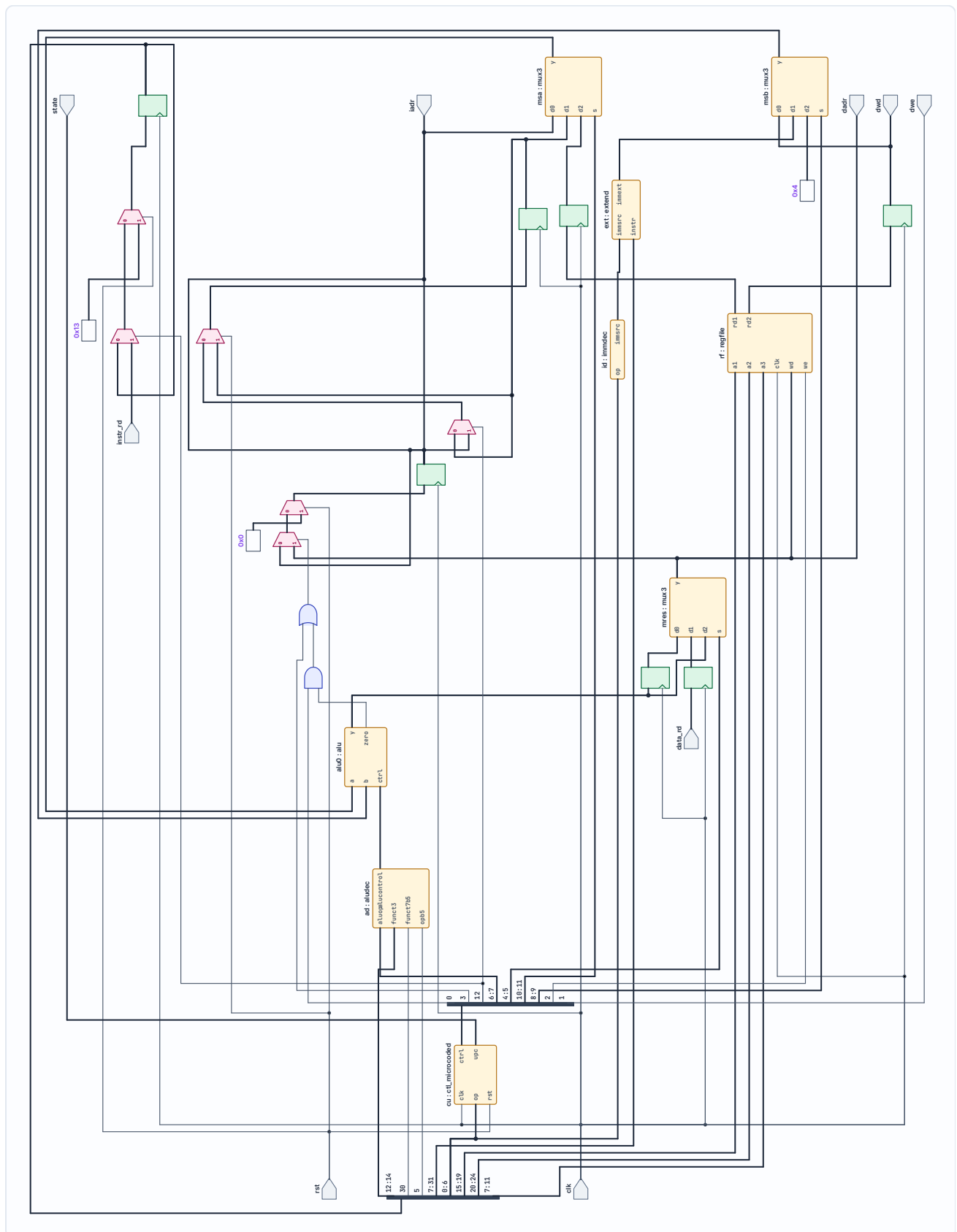


Figure D.4. Synthesized multicycle processor with the microcoded controller (*cu*) and datapath blocks as black boxes. Green cells are the non-architectural registers; the three *mux3* instances are the SrcA, SrcB and Result selectors. Rotated for legibility.

YOSYS → NETLISTSVG

Both controller variants run the test program to the reference state, and both pass all 2 000 random programs of Section 10.1 — further evidence, alongside the proofs of Section 10.3, that the two controllers are interchangeable. Counting state visits gives an exact cycle budget (Table D.3). The program's 59 instructions before the halt take 227 cycles. The halt itself (a branch, 3 cycles) completes in cycle 229, so the whole run takes 230 cycles, a CPI of 3.83.

Table D.3. Multicycle cycle accounting for the test program (state visits counted in simulation).

Instruction class	States	Cycles each	Count	Cycles
lw	S0 S1 S2 S3 S4	5	1	5
sw	S0 S1 S2 S5	4	4	16
R-type	S0 S1 S6 S7	4	17	68
I-type	S0 S1 S8 S7	4	17	68
jal	S0 S1 S9 S7	4	10	40
beq (excluding the halt)	S0 S1 S10	3	10	30
Total before the halt			59	227

Figure D.5 compares all four processors on the same program in clock cycles; Section 8.11 measures their clock periods and total run times. The cycle counts alone bound what each design must achieve to beat the single-cycle machine:

- the multicycle machine must run its clock more than $230/60 \approx 3.8$ times faster;
- the pipeline more than $87/60 \approx 1.45$ times faster;
- the predicting pipeline more than $71/60 \approx 1.18$ times faster.

An ideal five-way split would make the multicycle clock about five times faster, which would let it win narrowly. In the measured design it is only about 1.1× faster (Section 8.11), because the ALU → Result → PC path must still fit in one cycle. On speed the multicycle machine therefore loses clearly. Multicycle designs survive where hardware reuse matters more than speed.

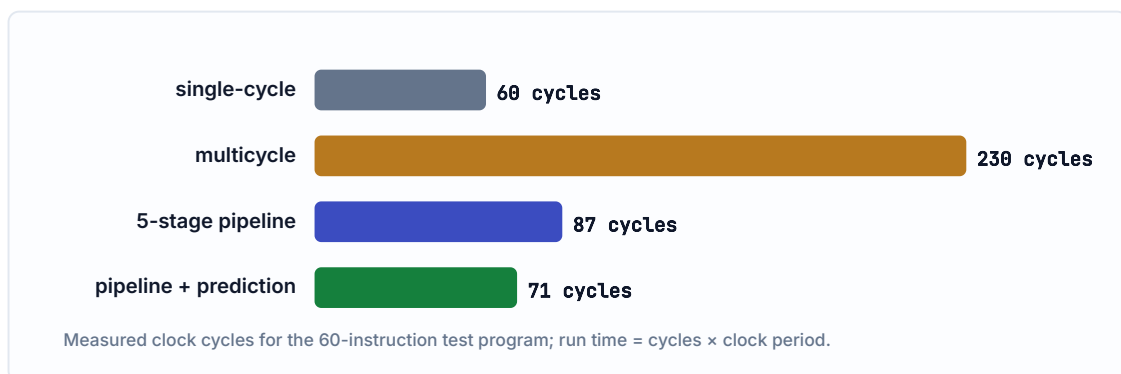


Figure D.5. Clock cycles to execute the same 60-instruction test program on the four verified processors.

APPENDIX E

Word-Level Operators at Gate Level

Most netlist figures in this primer mix ordinary gates with a few symbols that stand for a whole multi-bit operation. The circles marked +, −, = and < are the most visible of these (Figure 1.3), but several gate and multiplexer shapes are also drawn with a multi-bit input and mean more than a single gate. This appendix opens each of these symbols and shows exactly what it contains at gate level. Each is shown two ways:

- as Yosys itself lowers the cell to gates, rendered from the lowered netlist;
- as the NAND-only construction built from the cells of Chapters 2 and 4.

Both views are verified.

E.1 Why some operations are drawn as circles

When Yosys reads `assign y = a + b;`, it does not immediately produce gates. It records a single *word-level cell*, here `$add`, with multi-bit ports. Keeping arithmetic at that level lets later steps choose an architecture — ripple, Brent-Kung or Kogge-Stone for an adder (Sections 4.6 and 8.11). `netlistsvg` draws such cells as labelled circles. Only at the *technology-mapping* step (`techmap`) is each cell replaced by a network of single-bit gates, which is what the figures of this appendix show. Table E.1 summarizes the symbols.

Table E.1. Word-level symbols used in the netlist figures and their gate-level meaning.

Symbol	Yosys cell	Computes	Gate-level realization	Section
	<code>\$add</code>	$y = a + b \pmod{2^N}$	Generate/propagate per bit, carry network, sum XORs	E.2
	<code>\$sub</code>	$y = a - b \pmod{2^N}$	Inverters on b , adder with carry-in 1	E.3
	<code>\$eq</code>	$y = 1$ iff $a = b$	XOR per bit, OR tree, final NOT	E.4
	<code>\$lt</code>	$y = 1$ iff $a < b$	The borrow of $a - b$ (unsigned); plus a sign correction (signed)	E.5
  	<code>\$le</code> , <code>\$gt</code> , <code>\$ge</code>	other orderings	$\overline{b} < \overline{a}$, $b < a$, $\overline{a} < \overline{b}$ (in that order)	E.5
OR / NOT shape with a bus input	<code>\$reduce_or</code> , <code>\$reduce_bool</code> , <code>\$logic_not</code>	any bit set / no bit set	OR tree / OR tree + NOT	E.6
trapezoid	<code>\$mux</code> , <code>\$pmux</code>	select one word	One 2:1 multiplexer per bit / AND-OR with one-hot select	E.6
green box	<code>\$dff</code>	register	One D flip-flop per bit (Section 5.3)	5.5

E.2 The adder (+)

Figure E.1 is `$add` at 4 bits after Yosys' technology mapping. The left column of AND and XOR gates forms each bit's generate $g_i = a_i b_i$ and propagate $p_i = a_i \oplus b_i$ (Section 4.4). The AND-OR network in the middle is Yosys' default Brent-Kung carry-lookahead unit, which computes the carries c_i . The right-hand XORs form the sums $s_i = p_i \oplus c_i$. Bit 0 has no carry-in, so its sum is simply $a_0 \oplus b_0$. Because y has the same width as the operands, the carry out of the top bit is simply not built: the result is $a + b$ modulo 2^N .

The NAND-only equivalent is the ripple-carry adder of Figure 4.3 (nine NANDs per bit) or, when depth matters, the Kogge–Stone adder of Section 4.6.

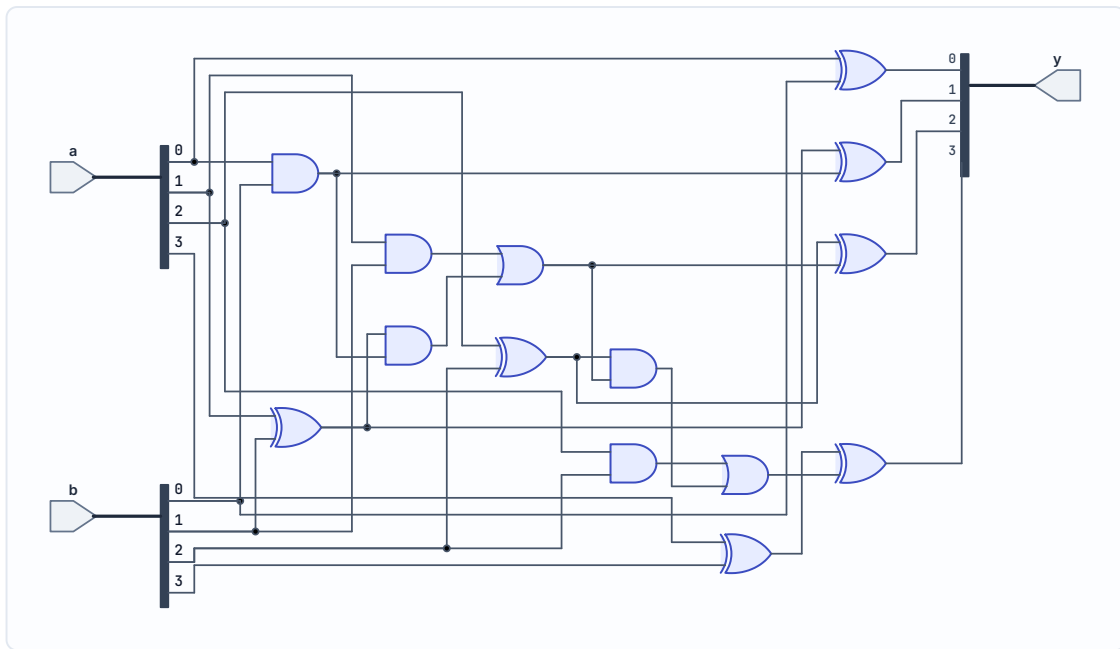


Figure E.1. The + circle at gate level: `assign y = a + b` (4 bits) after `techmap`. Generate/propagate gates, a Brent–Kung carry network and sum XORs. YOSYS → NETLISTSVG

E.3 The subtractor (–)

Two's-complement subtraction reuses the adder, because $a - b = a + \bar{b} + 1$ (Section 4.1). Yosys therefore lowers `$sub` to an inverter on every bit of `b` followed by the same adder structure, with the "1" entering as a carry-in of 1 (Figure E.2). Because that carry-in is a constant, the bit-0 sum XOR simplifies to an inverter: $s_0 = a_0 \oplus \bar{b}_0 = a_0 \oplus b_0$. The NAND-only version `sub_n` (Figure E.3, Listing E.1) does exactly this with `not_n` cells and the ripple adder of Chapter 4. Its carry-out is 1 precisely when no borrow occurs, that is, when $a \geq b$ as unsigned numbers. That signal is the key to the comparisons of Section E.5.

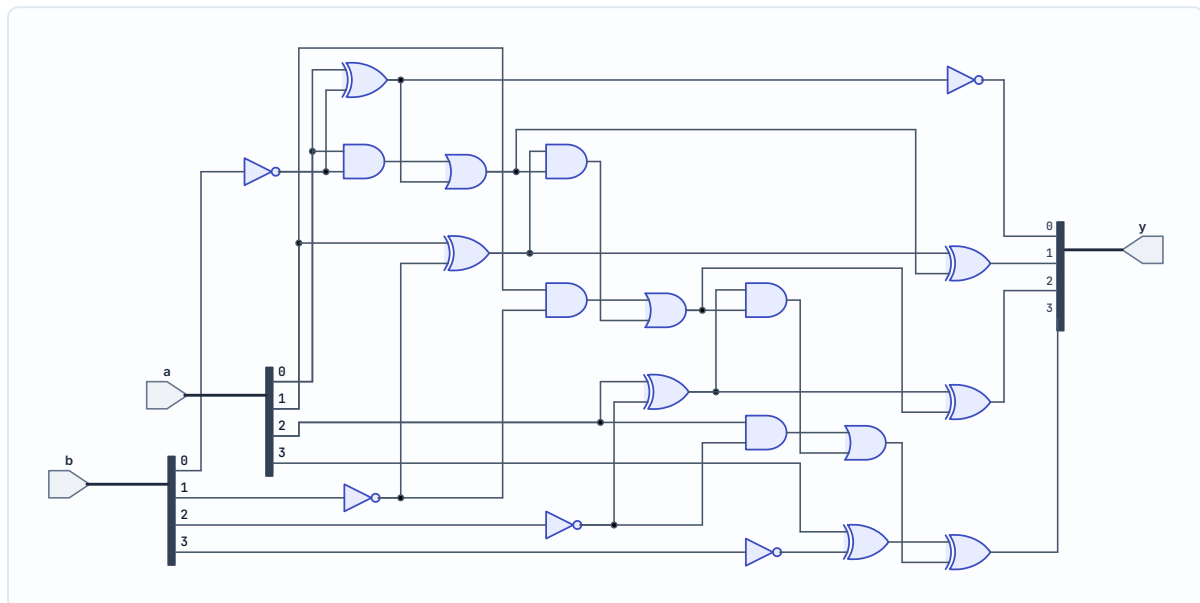


Figure E.2. The – circle at gate level (4 bits): inverters on `b` feeding the adder network with carry-in 1.

YOSYS → NETLISTSVG

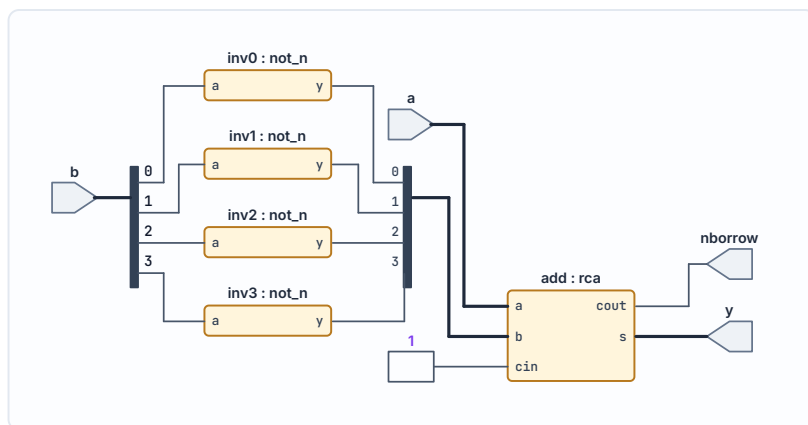


Figure E.3. NAND-structural subtractor: four NAND inverters and the ripple-carry adder (carry-in 1). *nborrow* = 1 exactly when $a \geq b$. `YOSYS → NETLISTSVG`

E.4 Equality (=)

Two words are equal when no bit position differs. One XOR per bit flags a difference, an OR of all the flags asks "does any bit differ?", and a final NOT turns that into "equal":

$$(a = b) = \overline{(a_0 \oplus b_0) + (a_1 \oplus b_1) + \dots + (a_{N-1} \oplus b_{N-1})}.$$

Yosys builds the OR as a balanced tree (Figure E.4 shows the 4-bit case), so the depth grows only logarithmically: 8 NAND levels at 4 bits and 14 at 32 bits. The NAND-only version `eq_n` (Figure E.5) uses the 4-NAND XOR and 3-NAND OR of Chapter 2 in the same tree shape. The `≠` operator (`$ne`) is the same circuit without the final NOT. The hazard unit's tests `rs1E = rdM` are exactly these cells, and its "`≠ 0`" guards are the OR tree alone (`$reduce_bool`, Section E.6).

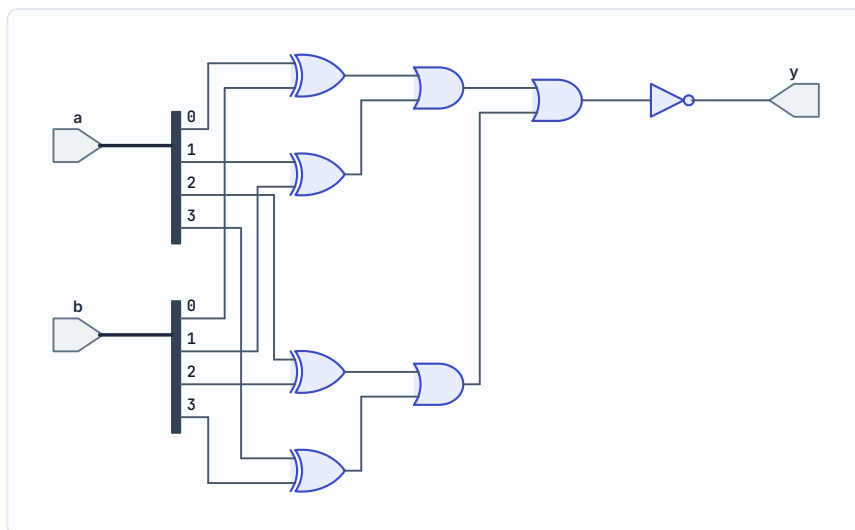


Figure E.4. The = circle at gate level (4 bits): XORs, a two-level OR tree and a NOT.

`YOSYS → NETLISTSVG`

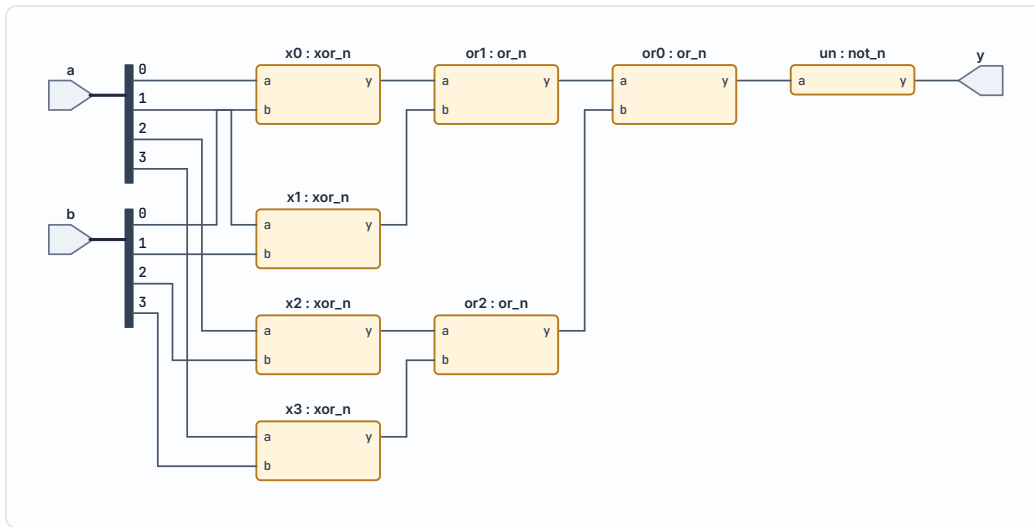


Figure E.5. NAND-structural equality, 4 bits: *xor_n* leaves, an *or_n* tree, and *not_n*. YOSYS → NETLISTSVG

E.5 Comparison (<, ≤, >, ≥)

For unsigned numbers, $a < b$ exactly when the subtraction $a - b$ needs a borrow, i.e. when the carry-out of $a + \bar{b} + 1$ is 0. A less-than cell is therefore a subtractor that keeps only its carry chain and discards the sum bits. Figure E.6 shows Yosys doing precisely that: the inverters on b , the generate/propagate gates and the carry logic remain, and no sum XORs are built.

For signed numbers, one correction is needed. If the operands have different signs, the negative one is smaller, whatever the magnitudes. If they have the same sign, the unsigned comparison is already right:

$$\langle a \rangle < \langle b \rangle = \begin{cases} a_{N-1} & a_{N-1} \neq b_{N-1} \\ a <_u b & \text{otherwise.} \end{cases}$$

`lts_n` (Figure E.7) implements this with an XOR on the sign bits and a 2:1 multiplexer in front of the unsigned comparator. The ALU of Section 4.5 reaches the same result from the other direction, correcting the sign of $a - b$ with the overflow bit. The remaining orderings need no new hardware: $a > b$ is $b < a$, $a \geq b$ is $\overline{a < b}$, and $a \leq b$ is $\overline{b < a}$.

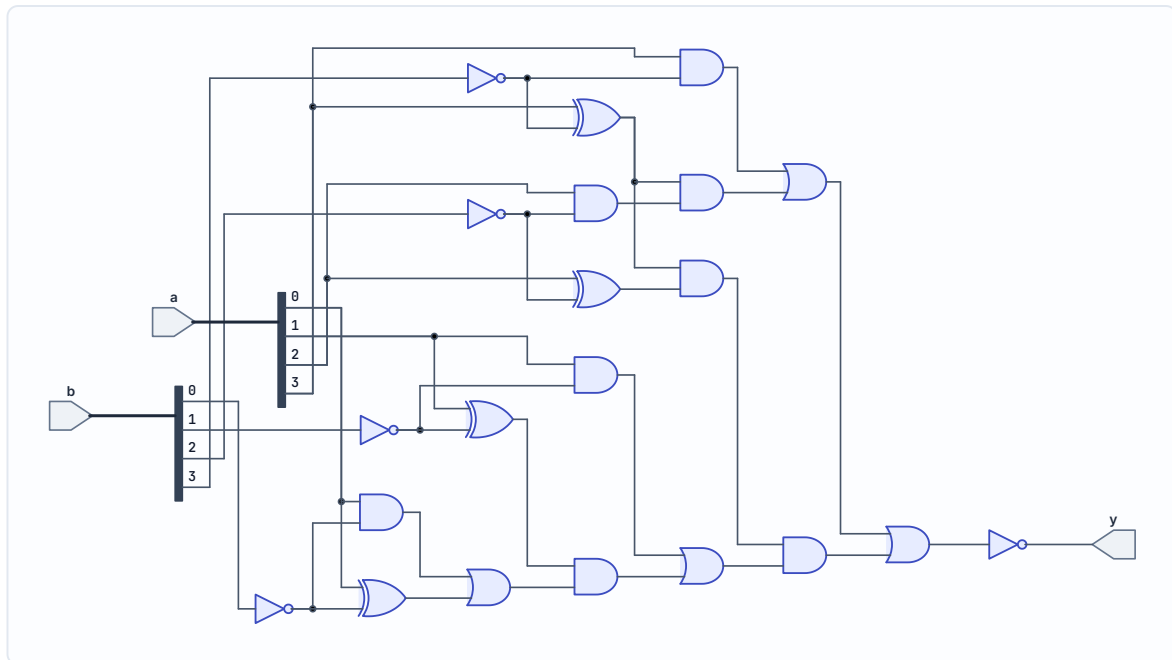


Figure E.6. The < circle at gate level (4 bits, unsigned): the carry network of the subtraction $a - b$, ending in a NOT that turns "no borrow" into "less than".
YOSYS → NETLISTSVG

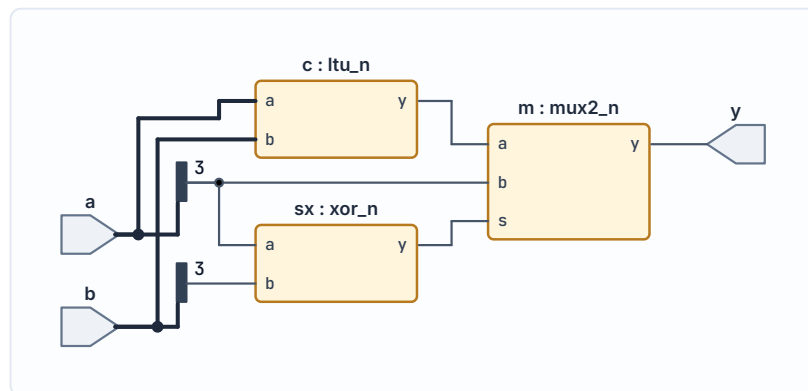


Figure E.7. NAND-structural signed comparison: the unsigned comparator *ltu_n* plus a sign-bit XOR and a 2:1 multiplexer.
YOSYS → NETLISTSVG

E.6 Cells drawn with ordinary gate symbols

Four more word-level cells appear in the figures without a circle, and each looks like a single gate:

- **Reduction OR** (`$reduce_or`, `$reduce_bool`). Drawn as an OR gate with a bus input, it means "any bit is 1", and lowers to an OR tree (Figure E.8). The hazard unit's " $\neq 0$ " guards use it (Figure 8.6).
- **Logical NOT** (`$logic_not`). Drawn as an inverter on a bus, it means "all bits are 0", which is the same OR tree followed by one inverter (Figure E.9). The ALU's *zero* output is one (Figure 4.6).
- **Word multiplexer** (`$mux`). One trapezoid on buses is one 2:1 multiplexer per bit, all sharing the select line (Figure E.10). At NAND level each is the 4-gate multiplexer of Section 2.5.
- **Parallel multiplexer** (`$pmux`). A trapezoid with a one-hot select bus, as in the 4×4 register file (Figure 5.7) and the ALU (Figure 4.6). It lowers to an AND-OR network: each input bit is ANDed with its select bit, and the results are ORed. Then one 2:1 multiplexer per bit, controlled by the OR of all select bits, supplies the default value (here 0) when no select bit is set (Figure E.11).

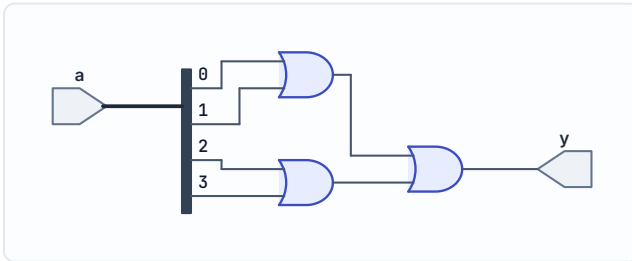


Figure E.8. Reduction OR (4 bits) → OR tree.

YOSYS → NETLISTSVG

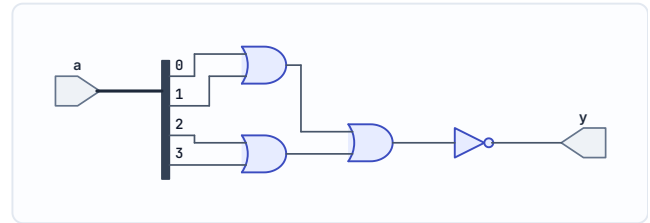


Figure E.9. Logical NOT (4 bits) → OR tree + NOT.

YOSYS → NETLISTSVG

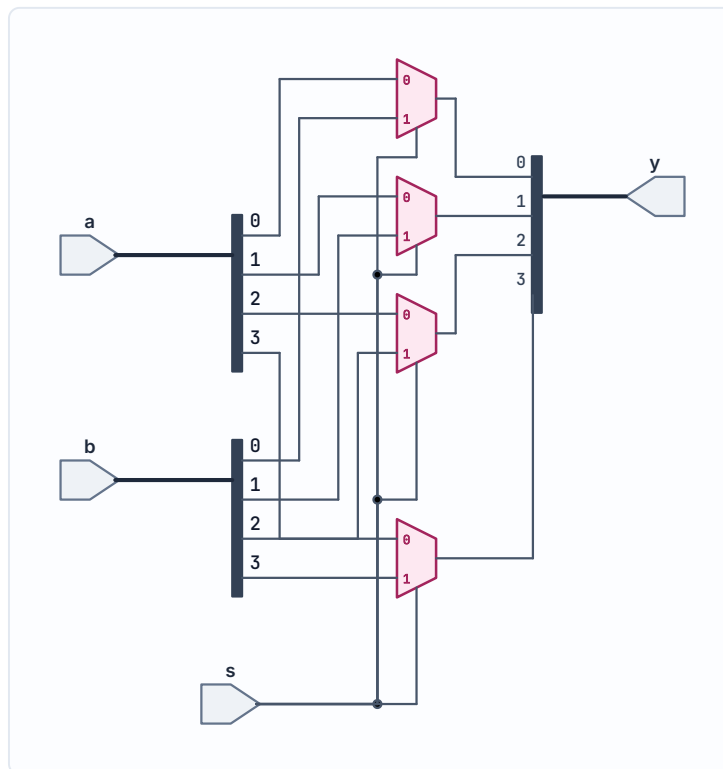


Figure E.10. Word multiplexer (4 bits) → four 2:1 multiplexers with a shared select.

YOSYS → NETLISTSVG

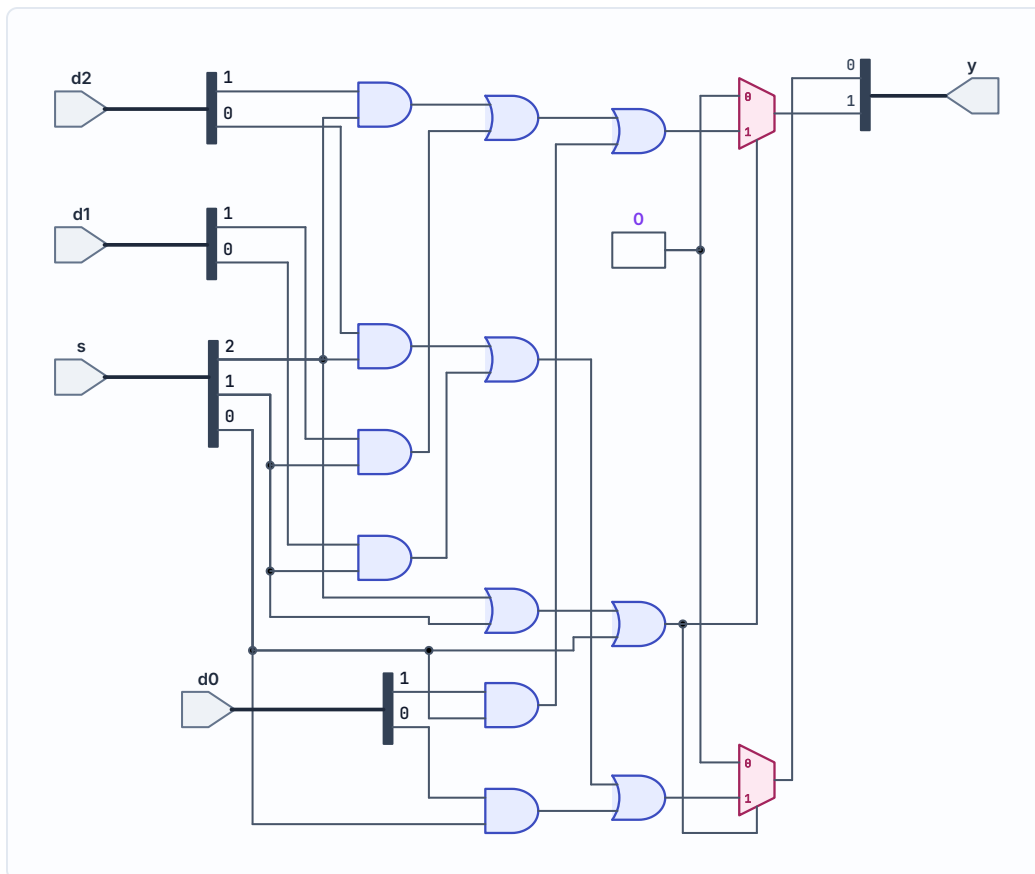


Figure E.11. One-hot multiplexer (three 2-bit inputs $d0$ – $d2$, select s) → per-bit AND-OR selection, then one default multiplexer per bit driven by the OR of the select bits.

YOSYS → NETLISTSVG

E.7 Cost and verification

Table E.2 compares the two realizations at four widths, with every gate costed in NAND gates and NAND levels. Yosys' lowered gates are converted to NAND-equivalent counts using Table 2.1: AND = 2, OR = 3, XOR = 4, NOT = 1. The equality cell is cheap and shallow at every width. Addition, subtraction and comparison inherit the adder's carry problem:

- **Ripple carry** (the NAND-only versions) keeps the gate count lowest, but its depth grows linearly with width.
- **Brent–Kung** (Yosys' carry network) grows only logarithmically in depth, although at 4 and 8 bits it is still one or two levels deeper than ripple carry and it only pulls ahead from 16 bits. It also costs more gates as the width grows: at 32 bits, a Brent–Kung adder takes 618 NAND-equivalent gates and 38 levels, against the ripple adder's 288 gates and 68 levels.
- **Signed comparison** adds 8 gates and 2 levels to the unsigned one at every width (`lts_n`: 37·15 / 65·23 / 121·39 / 233·71 gates·levels for 4/8/16/32 bits).

Table E.2. NAND-equivalent gates / NAND levels of each operator. "Yosys" is its own lowering (Brent–Kung carry network); "NAND" is the structural construction (ripple carry).

Operator	4 bits		8 bits		16 bits		32 bits	
	Yosys	NAND	Yosys	NAND	Yosys	NAND	Yosys	NAND
+ (add)	44 / 14	36 / 12	118 / 22	72 / 20	280 / 30	144 / 36	618 / 38	288 / 68
– (subtract)	52 / 17	40 / 13	130 / 25	80 / 21	300 / 33	160 / 37	654 / 41	320 / 69
= (equal)	26 / 8	26 / 8	54 / 10	54 / 10	110 / 12	110 / 12	222 / 14	222 / 14
< (unsigned less-than)	49 / 15	29 / 13	103 / 19	57 / 21	213 / 23	113 / 37	435 / 27	225 / 69

Both views are verified.

- **Yosys' lowering.** Each lowered netlist was written back out as gate-only Verilog and simulated against the original operator over every input combination. The arithmetic and comparison cells were checked at 4 and 8 bits, the reduction cells at 8 bits and the multiplexers at 4 bits; the one-hot multiplexer used every select value that is one-hot or all-zero. In total: 280 576 vectors, 0 failures.
- **NAND constructions.** These were checked on all 65 536 operand pairs at 8 bits, and property F10 of Section 10.3 proves them equal to $+$, $-$, $==$, and signed and unsigned $<$ for all 16-bit operands (equality also at 32 bits).

Listing E.1. NAND-structural subtractor, equality and comparison cells.

opgates.sv

```
// Subtractor:  $a - b = a + \sim b + 1$ . The carry-out is 1 exactly when  $a \geq b$  (no borrow).
module sub_n #(parameter int N = 4) (input logic [N-1:0] a, b,
                                     output logic [N-1:0] y, output logic nborrow);
    logic [N-1:0] nb;
    for (genvar i = 0; i < N; i++) begin : g_inv
        not_n u (.a(b[i]), .y(nb[i]));
    end
    rca #(N) add (.a(a), .b(nb), .cin(1'b1), .s(y), .cout(nborrow));
endmodule

// Equality: one XOR per bit marks a difference; an OR tree collects them; NOT gives "equal".
// N must be a power of two (the OR tree is a complete binary tree).
module eq_n #(parameter int N = 4) (input logic [N-1:0] a, b, output logic y);
    logic [2*N-2:0] t; // heap-ordered tree: leaves N-1 .. 2N-2
    for (genvar i = 0; i < N; i++) begin : g_leaf
        xor_n ux (.a(a[i]), .b(b[i]), .y(t[N-1+i]));
    end
    for (genvar k = N-2; k >= 0; k--) begin : g_node
        or_n uo (.a(t[2*k+1]), .b(t[2*k+2]), .y(t[k]));
    end
    not_n un (.a(t[0]), .y(y));
endmodule

// Unsigned less-than:  $a < b$  exactly when  $a - b$  borrows (carry-out of  $a + \sim b + 1$  is 0).
module ltu_n #(parameter int N = 4) (input logic [N-1:0] a, b, output logic y);
    logic [N-1:0] diff; logic nborrow;
    sub_n #(N) s (.a, .b, .y(diff), .nborrow);
    not_n u (.a(nborrow), .y(y));
endmodule

// Signed less-than: if the signs differ,  $a < b$  exactly when  $a$  is negative;
// if they agree, the unsigned comparison is correct.
module lts_n #(parameter int N = 4) (input logic [N-1:0] a, b, output logic y);
    logic u, d;
    ltu_n #(N) c (.a, .b, .y(u));
    xor_n sx (.a(a[N-1]), .b(b[N-1]), .y(d));
    mux2_n m (.a(u), .b(a[N-1]), .s(d), .y(y));
endmodule
```

APPENDIX F

Reproducing the Results

Every artefact in this primer comes from the accompanying source archive, laid out as follows:

- `hdl/`: the SystemVerilog sources `gates.sv`, `seq.sv`, `rv_blocks.sv`, `rv_single.sv`, `rv_pipe.sv`, `arith.sv`, `mem16.sv`, `cache.sv`, `fpu.sv`, `cpu_fpu.sv`, `microcode.sv`, `rv_multi.sv`, `rv_pipe_bp.sv`, `prefix.sv`, `kmap.sv`, `opgates.sv`, `formal.sv`, `legend.sv`, and the generated and analysis multipliers `dadda8.sv`, `dadda8_rca.sv`, `dadda8_ks.sv` and `mul_array8.sv`;
- `sim/`: the testbenches, the assembler and reference simulator, the result checker, the random co-simulation (`cosim.py`) and mutation (`mutate.py`) drivers;
- `rend/`: the netlist rendering, static-timing and formal-proof scripts (`formal.py`, `formal_neg.py`), with the netlistsvg skins;
- `fig/`: the generators for the hand-drawn figures and the timeline charts.

Listing F.1 gives the commands that produce the verification results and every netlist figure. The hand-drawn figures and timelines come from `fig/figs.py` and `fig/timeline.py`. Yosys was run as the WebAssembly build `yowasp-yosys` from PyPI; a native Yosys works identically.

Listing F.1. Build and verification commands.

```
# tools
pip install yosys-yosys # Yosys 0.69 (WebAssembly build)
npm install netlistsvg # 1.0.2
apt install iverilog # 12.0

# 1. unit tests (exhaustive where stated)
cd sim
iverilog -g2012 -DSIM -o units.vvp tb_units.sv ../hdl/gates.sv ../hdl/seq.sv \
    ../hdl/rv_blocks.sv ../hdl/arith.sv
vvp -n units.vvp # → unit checks: 267576, failures: 0

# 2. processors vs. reference simulator
python3 asm.py # writes prog.hex, expected.json, listing.txt
iverilog -g2012 -DOUTFILE="single.log" -o single.vvp tb_cpu.sv ../hdl/rv_blocks.sv ../hdl/rv_single.sv
iverilog -g2012 -DPIPE -DOUTFILE="pipe.log" -o pipe.vvp tb_cpu.sv ../hdl/rv_blocks.sv ../hdl/rv_pipe.sv
vvp -n single.vvp; vvp -n pipe.vvp
python3 check.py # regs match: True mem match: True (both)
iverilog -g2012 -o halt.vvp tb_halt.sv ../hdl/rv_blocks.sv ../hdl/rv_single.sv && vvp -n halt.vvp
iverilog -g2012 -o ng.vvp ng.sv && vvp -n ng.vvp # Booth negative control: 255 failures
iverilog -g2012 -DSIM -o d.vvp tb_dadda.sv ../hdl/dadda8.sv ../hdl/gates.sv && vvp -n d.vvp
iverilog -g2012 -o mem.vvp tb_mem.sv ../hdl/mem16.sv ../hdl/rv_blocks.sv && vvp -n mem.vvp
python3 ../fig/microasm.py # assemble microcode.hex, dispatch1/2.hex
iverilog -g2012 -s tb_micro -o mc.vvp tb_micro.sv ../hdl/microcode.sv ../hdl/rv_blocks.sv && vvp -n mc.vvp
iverilog -g2012 -s tb_micro_comb -o mcc.vvp tb_micro.sv ../hdl/microcode.sv ../hdl/rv_blocks.sv && vvp -n
mcc.vvp
iverilog -g2012 -DMULTI -DMICRO=1 -DCYCLES=400 -DOUTFILE="multi1.log" -o mu1.vvp tb_cpu.sv
    ../hdl/rv_blocks.sv ../hdl/microcode.sv ../hdl/rv_multi.sv && vvp -n mu1.vvp
iverilog -g2012 -DPIPE -DBP -DOUTFILE="bp.log" -o bp.vvp tb_cpu.sv ../hdl/rv_blocks.sv ../hdl/rv_pipe.sv
    ../hdl/rv_pipe_bp.sv && vvp -n bp.vvp
python3 check.py single pipe bp multi1
iverilog -g2012 -DSIM -o dc.vvp tb_dffchar.sv ../hdl/gates.sv ../hdl/seq.sv && vvp -n dc.vvp # flip-flop
timing
python3 numdemo.py # number-system program (then run tb_cpu.sv in numdemo/)
python3 fpvec.py 300000 && iverilog -g2012 -o fpu.vvp tb_fpu.sv ../hdl/fpu.sv && vvp -n fpu.vvp
python3 fpudemo.py # FPU program; run tb_cpu_fpu.sv in fpudemo/ with -DCPU=0..3
iverilog -g2012 -o ops.vvp tb_ops.sv ../hdl/opgates.sv ../hdl/gates.sv && vvp -n ops.vvp
iverilog -g2012 -o ks.vvp tb_ks.sv ../hdl/prefix.sv ../hdl/gates.sv && vvp -n ks.vvp
iverilog -g2012 -o cache.vvp tb_cache.sv ../hdl/cache.sv ../hdl/mem16.sv ../hdl/rv_blocks.sv && vvp -n
cache.vvp
python3 cosim.py 2000 # random co-simulation, 5 configurations (~4 min)
python3 mutate.py # mutation testing, 6 mutants x 200 programs
iverilog -g2012 -o m8.vvp tb_mul8.sv ../hdl/dadda8_rca.sv ../hdl/mul_array8.sv ../hdl/gates.sv && vvp -n m8.vvp

# 3. netlists and gate-depth analysis
cd ../rend
python3 render.py # yosys → JSON → netlistsvg (*.raw.svg)
python3 style.py # apply colour skin → out/*.svg
python3 sta.py # NAND depth of N-bit ripple adders
python3 sta_mul.py # array vs Dadda critical path: 70 vs 42
python3 sta_ks.py # ripple vs Kogge-Stone depth and size
python3 sta_struct.py # clock periods of the four CPUs (structural mapping)
python3 sta_sweep.py; python3 sta_abc.py # memory-delay sweep; ABC-mapping comparison
python3 sta_fpu.py fp_add32; python3 sta_fpu.py fp_mul32; python3 sta_fpu_breakdown.py
python3 sta_ops.py; python3 lower_check.py # operator costs; verify Yosys' gate-level lowering
cp ../sim/*.hex . && python3 formal.py && python3 formal_neg.py # 10 proofs + 4 negative controls

# 4. hand-drawn figures and pipeline timelines (reads sim/pipe.log)
cd ../fig && python3 gen_dadda.py && python3 figs.py && python3 figs2.py && python3 figs3.py && python3
figs4.py && python3 figs5.py && python3 timeline.py
python3 cachesim.py ../sim/cache_traces.txt && python3 fetchcache.py # cache model and fetch-trace study
python3 depth_mul.py # cell-level delay model (reproduces 70 / 42; tree = 24)
```

Each rendering job is a Yosys script of the form shown in Listing F.2. For gate-level figures, `nand2` is kept as a black box and renamed to Yosys's internal NAND cell type so that netlistsvg draws a NAND symbol. For hierarchical figures, the listed submodules are black-boxed instead.

Listing F.2. Representative Yosys script (full adder).

```
read_verilog -sv ../hdl/gates.sv
hierarchy -top full_adder
blackbox nand2
proc; opt_clean
write_json full_adder.json
```

APPENDIX G

Glossary and Notation

This appendix collects the terms and abbreviations (Table G.1) and the notation (Table G.2) used in the primer, with a pointer to where each is introduced.

G.1 Terms and abbreviations

Table G.1. Glossary.

Term	Meaning
ABC	Logic optimizer and technology mapper called by Yosys; restructures logic for area unless given a timing library (Section 8.11).
ALU	Arithmetic-logic unit: the datapath block that adds, subtracts, ANDs, ORs and compares (Section 4.5).
BCD	Binary-coded decimal: each decimal digit coded in 4 bits; codes 10–15 unused (Section 2.7).
Bubble	A pipeline slot holding no instruction; created by clearing a pipeline register (Section 8.2).
BTB	Branch-target buffer: a small table predicting whether a fetched instruction branches and where to (Section 8.10).
Canonical SoP / PoS	Sum of all minterms where the function is 1 / product of all maxterms where it is 0 (Section 2.6).
Carry-in, carry-out	The carry entering the least-significant and leaving the most-significant position of an adder.
CLA	Carry-lookahead adder: computes carries directly from generate/propagate signals (Section 4.4).
Clock-to-Q (t_{pcq})	Delay from the clock edge until a flip-flop output is valid (Section 5.4).
CMOS	Complementary metal-oxide-semiconductor: logic built from paired p- and n-channel transistors (Section 2.3).
Co-simulation	Running the design and a reference model on the same stimulus and comparing results (Section 10.1).
Counterexample	An input (sequence) for which a claimed property fails; produced by a SAT solver when a proof fails.
CPA	Carry-propagate adder: an ordinary adder that resolves all carries, e.g. the final adder of a multiplier tree.
CPI	Cycles per instruction: average clock cycles needed to execute one instruction (Section 7.3).
CPU	Central processing unit; in this primer, one of the four RV32I-subset processors.
Critical path	The slowest register-to-register path; it sets the minimum clock period.
Datapath / control	The part of a processor that stores and transforms data / the part that decides what the datapath does each cycle.
Don't-care (×)	An input combination that never occurs, so the output may be chosen freely during minimization.
DRAM, SRAM	Dynamic / static random-access memory: capacitor-based / cross-coupled-latch-based storage (Section C.8).
Dispatch	A microcode branch that uses the opcode to select the next microinstruction (Appendix D).
FA, HA	Full adder (three input bits), half adder (two input bits) (Section 4.2).
Flush	Clearing a pipeline register to discard a wrongly fetched instruction (Section 8.6).
Forwarding	Routing a newer value from a later pipeline stage to an earlier one instead of the stale register-file value (Section 8.4).

FPGA	Field-programmable gate array: a reconfigurable chip built from lookup tables and flip-flops.
FSM	Finite-state machine: a state register plus next-state and output logic (Appendix D).
Gate level, NAND level	The unit of delay used throughout: one level is one NAND gate; other gates count their NAND depth (Table 2.1).
Gray code	Ordering in which neighbouring codes differ in one bit (00, 01, 11, 10); used to label K-map rows and columns.
Hardwired control	Control signals produced by logic gates synthesized from the specification (Appendix D).
Harvard memory	Separate instruction and data memories, as used by all four processors here.
Hazard	A situation where the next instruction cannot proceed correctly in the next cycle: data, load-use or control (Chapter 8).
HDL	Hardware description language; this primer uses SystemVerilog.
Hold time (t_{hold})	How long a flip-flop input must stay stable after the clock edge (Section 5.4).
Implicant, prime implicant	A product term covering only 1s (and don't-cares); a prime implicant cannot be enlarged further (Section 2.7).
Induction (proof by)	Showing a property holds initially and is preserved by every step, hence always (F7, Section 10.3).
ISA	Instruction set architecture: the programmer-visible contract of a processor (Chapter 6).
ISS	Instruction-set simulator: the Python reference model that executes programs instruction by instruction.
K-map	Karnaugh map: a truth table arranged so adjacent cells differ in one variable (Section 2.7).
KS, BK	Kogge–Stone and Brent–Kung parallel-prefix carry networks (Sections 4.6, 8.11).
Latch, flip-flop	Level-sensitive / edge-triggered one-bit storage (Chapter 5).
Load-use hazard	An instruction needs a value that a <code>lw</code> immediately ahead of it has not yet read; fixed by a one-cycle stall (Section 8.5).
Metastability, MTBF	An unresolved flip-flop state after a setup/hold violation; mean time between such failures (Section 5.4).
Microcode, μPC	Control words stored in a memory; the micro-program counter that addresses them (Appendix D).
Minterm, maxterm	A product (sum) containing every variable once; 1 (0) for exactly one input row (Section 2.6).
Misprediction	A branch prediction that proves wrong in Execute; costs a two-instruction flush (Section 8.10).
MSB, LSB	Most / least significant bit.
Mutation testing	Injecting a deliberate bug to check that the tests detect it (Section 10.2).
Netlist	A circuit described as cells and the wires between them; every figure tagged Yosys $\rightarrow$ netlistsvg is a rendered netlist.
Technology mapping (techmap)	The synthesis step that replaces word-level cells such as \$add by single-bit gates (Appendix E).
Word-level cell	A synthesis cell operating on whole buses, e.g. \$add or \$eq, drawn as a circle (Appendix E).
One-hot	A bit vector with exactly one bit set, e.g. decoder outputs (Appendix C).
Opcode, funct3, funct7	Instruction fields that select the operation (Section 6.2).
Overflow (V), carry (C)	A signed result that does not fit in the word; an unsigned result that wraps (Section 3.5).
Fixed point ($Qm.n$)	An integer read with an implied binary point n bits from the right (Section 3.8).
Sign / zero extension	Widening a signed / unsigned value by copying the sign bit / inserting zeros (Section 3.4).
IEEE 754, binary32	The floating-point standard; its 32-bit format with sign, 8-bit biased exponent and 23-bit fraction (Section 9.1).
Significand, exponent bias	The digits of a floating-point number ($1.f$); the constant (127) subtracted from the stored exponent (Section 9.1).

Subnormal (denormal)	A floating-point number below the normal range, stored with exponent field 0 (Section 9.1).
NaN, canonical NaN	Not-a-number, the result of invalid operations; RISC-V always returns 0x7FC00000 (Section 9.2).
Guard, round, sticky bits	The first, second and OR of all remaining discarded bits, used for correct rounding (Section 9.3).
ulp	Unit in the last place: the gap between a floating-point number and its successor (Section 9.2).
FPU	Floating-point unit: hardware for floating-point arithmetic (Chapter 9).
fcsr, frm, fflags	RISC-V floating-point control/status register, its rounding-mode field and its sticky exception flags (Section 9.7).
Memory-mapped I/O	Accessing a peripheral through ordinary load/store instructions at reserved addresses (Sections C.8, 9.6).
Parallel-prefix adder	An adder computing all carries with an associative operator arranged as a tree, in $O(\log N)$ levels (Section 4.6).
PC	Program counter: the address of the current instruction.
Pipeline register	The register between two pipeline stages, named after them (F/D, D/E, E/M, M/W).
RAM, ROM	Random-access (read/write) memory; read-only memory.
RCA	Ripple-carry adder: a chain of full adders (Section 4.3).
RISC-V, RV32I, M extension	An open instruction-set architecture; its 32-bit base integer set; its multiply/divide extension (Chapter 6, Appendices A–B).
RTL	Register-transfer level: HDL written in terms of registers and word-level operations rather than gates.
SAT solver	A program that decides whether a Boolean formula can be made true; used here to prove properties (Section 10.3).
Setup time (t_{setup})	How long before the clock edge a flip-flop input must be stable (Section 5.4).
Sign extension	Copying an immediate's sign bit into the upper bits of a wider word (Section 6.2).
SLT	Set-less-than: outputs 1 if $a < b$ as signed numbers (Section 4.5).
SoP, PoS	Sum of products (AND-OR), product of sums (OR-AND) (Section 2.6).
SR latch	Set-reset latch built from two cross-coupled gates (Section 5.1).
SRT division	Division with a redundant quotient-digit set, named after Sweeney, Robertson and Tocher (Section B.3).
Stall	Holding pipeline registers for a cycle so an instruction waits (Section 8.5).
Static timing analysis	Computing worst-case path delays from the netlist without simulation (Sections 4.3, 8.11).
Testbench	Simulation-only code that drives a design and checks its outputs.
Two's complement	Signed binary representation in which the MSB has weight -2^{n-1} (Section 3.3).
XLEN	Register width of a RISC-V processor: 32 for RV32.

G.2 Notation

Table G.2. Notation.

Symbol	Meaning
$\overline{x}, x'$	NOT x (complement). Both forms appear: the overbar in equations, the prime in K-map expressions.
$x \cdot y, xy$	AND
$x + y$	OR (in Boolean expressions); addition in arithmetic context
$x \oplus y$	XOR
$\sum m(\dots), \prod M(\dots)$	Sum of the listed minterms; product of the listed maxterms (Section 2.6)
$\langle x \rangle$	Signed (two's-complement) value of bit vector x
$(g, p) \circ (g', p')$	Carry-combine operator of prefix adders (Section 4.4)
<code>a[31:2], {a, b}</code>	SystemVerilog bit range (bits 31 down to 2); concatenation
<code>32'd4, 2'b10, 0x9</code>	Sized literals: 32-bit decimal 4, 2-bit binary 10; hexadecimal 9
<code>~, &, , ^</code>	SystemVerilog bitwise NOT, AND, OR, XOR
suffix F, D, E, M, W	Pipeline stage a signal belongs to: <code>pcF</code> , <code>rdE</code> , <code>aluresultM</code> (Section 8.2)
<code>x0 ... x31</code>	RISC-V registers; <code>x0</code> always reads 0
<code>S0 ... S10</code>	States of the multicycle controller (Figure D.2)
<code>F1 ... F10</code>	Formal properties of Section 10.3
$T_c, t_{pd}, t_{cd}, t_{seq}$	Clock period; worst-case (propagation) and best-case (contamination) logic delay; flip-flop overhead $t_{pcq} + t_{setup}$
levels	Delay in NAND gates (unit gate delay), used for all timing figures
$\times$ (in K-maps)	Don't-care cell
12 345	Thousands are separated by a thin space

References

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Morgan Kaufmann, 2020 (© 2021).
- [2] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021 (© 2022).
- [3] N. Nisan and S. Schocken, *The Elements of Computing Systems: Building a Modern Computer from First Principles*, 2nd ed. MIT Press, 2021.
- [4] H. M. Sheffer, "A set of five independent postulates for Boolean algebras, with application to logical constants," *Transactions of the American Mathematical Society*, vol. 14, no. 4, pp. 481–488, 1913.
- [5] C. E. Shannon, "A symbolic analysis of relay and switching circuits," *Transactions of the AIEE*, vol. 57, no. 12, pp. 713–723, 1938.
- [6] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017 (© 2019).
- [7] A. D. Booth, "A signed binary multiplication technique," *Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, no. 2, pp. 236–240, 1951.
- [8] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Addison-Wesley, 2011.
- [9] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed. Oxford University Press, 2010.
- [10] P. M. Kogge and H. S. Stone, "A parallel algorithm for the efficient solution of a general class of recurrence equations," *IEEE Transactions on Computers*, vol. C-22, no. 8, pp. 786–793, 1973.
- [11] T. J. Chaney and C. E. Molnar, "Anomalous behavior of synchronizer and arbiter circuits," *IEEE Transactions on Computers*, vol. C-22, no. 4, pp. 421–422, 1973.
- [12] C. S. Wallace, "A suggestion for a fast multiplier," *IEEE Transactions on Electronic Computers*, vol. EC-13, no. 1, pp. 14–17, 1964.
- [13] A. Waterman and K. Asanović (eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213. RISC-V Foundation, 2019.
- [14] L. Dadda, "Some schemes for parallel multipliers," *Alta Frequenza*, vol. 34, pp. 349–356, 1965.
- [15] O. L. MacSorley, "High-speed arithmetic in binary computers," *Proceedings of the IRE*, vol. 49, no. 1, pp. 67–91, 1961.
- [16] C. Wolf et al., *Yosys Open SYnthesis Suite*, version 0.69. <https://github.com/YosysHQ/yosys>
- [17] N. Turley, *netlistsvg*, version 1.0.2. <https://github.com/nturley/netlistsvg>
- [18] S. Williams, *Icarus Verilog*, version 12.0. <https://github.com/steveicarus/iverilog>
- [19] A. Weinberger and J. L. Smith, "A logic for high-speed addition," *National Bureau of Standards Circular 591*, pp. 3–12, 1958.
- [20] J. E. Robertson, "A new class of digital division methods," *IRE Transactions on Electronic Computers*, vol. EC-7, pp. 218–222, 1958.
- [21] K. D. Tocher, "Techniques of multiplication and division for automatic binary computers," *Quarterly Journal of Mechanics and Applied Mathematics*, vol. 11, no. 3, pp. 364–384, 1958.
- [22] A. Edelman, "The mathematics of the Pentium division bug," *SIAM Review*, vol. 39, no. 1, pp. 54–67, 1997.
- [23] M. D. Ercegovac and T. Lang, *Digital Arithmetic*. Morgan Kaufmann, 2004.
- [24] M. V. Wilkes, "The best way to design an automatic calculating machine," in *Report of the Manchester University Computer Inaugural Conference*, Manchester, July 1951.
- [25] M. V. Wilkes, "The growth of interest in microprogramming: A literature survey," *ACM Computing Surveys*, vol. 1, no. 3, pp. 139–145, 1969.
- [26] W. V. Quine, "The problem of simplifying truth functions," *The American Mathematical Monthly*, vol. 59, no. 8, pp. 521–531, 1952.
- [27] M. Karnaugh, "The map method for synthesis of combinational logic circuits," *Transactions of the AIEE, Part I: Communication and Electronics*, vol. 72, no. 5, pp. 593–599, 1953.
- [28] E. J. McCluskey, "Minimization of Boolean functions," *Bell System Technical Journal*, vol. 35, no. 6, pp. 1417–1444, 1956.

- [29] R. K. Brayton, G. D. Hachtel, C. T. McMullen and A. L. Sangiovanni-Vincentelli, *Logic Minimization Algorithms for VLSI Synthesis*. Kluwer Academic, 1984.
- [30] J. K. F. Lee and A. J. Smith, "Branch prediction strategies and branch target buffer design," *IEEE Computer*, vol. 17, no. 1, pp. 6–22, 1984.
- [31] R. A. DeMillo, R. J. Lipton and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *IEEE Computer*, vol. 11, no. 4, pp. 34–41, 1978.
- [32] J. E. Smith, "A study of branch prediction strategies," in *Proceedings of the 8th Annual Symposium on Computer Architecture*, pp. 135–148, 1981.
- [33] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019. IEEE, 2019.
- [34] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 3, pp. 5–48, 1991.